

Understanding simpleKafka from Scratch

A source-driven, incremental Java course

simpleKafka teaching project

Java 21 · 31 executable steps

Before you begin

This book develops the course through one map chapter and eight implementation chapters, covering 31 steps. You will build record formats, logs, network requests, consumer offsets, consumer groups, controlled replica failover, and consistency experiments across producers, brokers, and consumers. This is neither “implementing Kafka” by connecting to a real Kafka cluster nor reducing production Kafka to a few empty terms. Every step maps to an explicit Java method, behavior tests, failure boundaries, and a reproducible command. Start at Step 1 and proceed in order; the book, course manifest, and test classes share the same numbering. This book is for readers starting with no Kafka, message-queue, or distributed-systems knowledge. The only prerequisites are basic Java reading (classes, methods, collections, and exceptions) and running terminal commands. The book explains binary data, files, networking, concurrency, and clocks when they first become necessary.

The course deliberately separates Java sources into three roots: **provided/**, **exercises/**, and **reference/**. The student entry point compiles the first two; the reference entry point compiles the first and third. The two implementations have identical fully qualified class names, so never compile both answer roots together. The reference implementation is for comparison with the contract after you try a step, not a back door onto the student test classpath. Chapter 1 explains the development environment, how to read test failures, and the progression rules.

Contents

1	Course Map, Tools, and How to Learn	1
1.1	How a Message Moves Through a System	1
1.2	What This Course Builds—and What It Does Not	2
1.3	Preparing Java 21+ and Course Commands	2
1.4	Understanding Binary Data and File I/O	3
1.5	How to Read Test Failures	3
1.6	Concepts Already Introduced and Where to Go Next	4
2	Records and Single-Segment Persistence	5
2.1	Step 1: Record Encoding and Decoding	5
2.2	Step 2: Log Append	8
2.3	Step 3: Offset-Based Reads	10
2.4	Step 4: Crash Recovery	11
3	Segments, Indexing, Retention, and Truncation	14
3.1	Step 5: Sparse Index	14
3.2	Step 6: Segmented Log	16
3.3	Step 7: Log Retention	19
3.4	Step 8: Tail Truncation	20
4	Partitions and a Broker on a Real Network	23
4.1	Step 9: Partition Routing	23
4.2	Step 10: Topic Metadata	25
4.3	Step 11: TCP Frame Protocol	27
4.4	Step 12: Single-Broker Requests	29
5	Producing, Consuming, Offsets, and Redelivery	33
5.1	Step 13: Batched Producer	33
5.2	Step 14: Pull-Based Consumer	36
5.3	Step 15: Consumer Offset Persistence	38
5.4	Step 16: At-Least-Once Processing	40
6	Consumer Groups, Rebalancing, and Commit Fencing	42
6.1	Step 17: Partition Assignment	42

6.2	Step 18: Membership and Rebalancing	44
6.3	Step 19: Heartbeats and Expiration	46
6.4	Step 20: Consumer Group Fencing	48
7	Replication, ISR, and High Watermarks	51
7.1	Step 21: Fetch-Based Replication	51
7.2	Step 22: ISR and High Watermark	54
7.3	Step 23: Produce Acknowledgments	56
7.4	Step 24: Concurrent Replicated Broker	57
8	Election, Log Repair, and Replica Readmission	60
8.1	Step 25: Clean Election and Leader Epochs	60
8.2	Step 26: Divergent-Log Reconciliation	62
8.3	Step 27: Metadata Routing Refresh	64
8.4	Step 28: Replica Readmission to ISR	66
9	Consistency Across Producers, Brokers, and Consumers	69
9.1	Step 29: Cross-Layer Consistency Experiments	69
9.2	Step 30: Idempotent Producer	73
9.3	Step 31: Idempotent Consumer Effects	77
A	Formats, Error Codes, and Interface Index	80
A.1	Disk and Network Formats	80
A.1.1	Records and Indexes	80
A.1.2	TCP Frames and Request Bodies	81
A.1.3	Message Body Field Order	81
A.1.4	Identifiers and Value Types	82
A.2	Idempotent Append and Durable Effects	83
A.3	Fixed Error Codes	83
A.4	The 31 Steps and Edit Entry Points	84
A.5	Course Simplifications and Differences from Kafka	85
A.6	Official References and Further Reading	86
A.7	Typesetting and Build Instructions	86
	Glossary	88

Chapter 1

Course Map, Tools, and How to Learn

1.1 How a Message Moves Through a System

Start with an order: after a customer pays, the shop needs to tell a later program, such as inventory, “the order is paid.” If the payment program calls inventory directly, the message may be lost while the receiver is down. A messaging system lets the sender hand off a message for storage, so the receiver can read it at its own pace. Apache Kafka is a distributed messaging and event-streaming platform; this book uses a runnable small teaching system to explain the underlying problems. No Kafka knowledge is assumed, and the course protocol is not presented as Kafka’s official protocol.

This book calls one storable, readable piece of data a **message** or **record**. A **producer** sends records; a **broker** accepts requests and stores them; a **consumer** actively requests and processes them. A messaging system is software that transfers and stores records between these roles. A successful send request, a record being read, and successful business processing are three distinct events; later chapters store and inspect their states separately.

A **topic** is a named category for related messages, such as **orders**. A topic can be split into ordered **partitions**; this course uses partitions so different records can be appended and read independently. A record’s **key** can associate or route it, its **value** is its content, and its **timestamp** is a time value stored with the record. The timestamp does not replace the **offset**, which determines order within a partition. Each offset is the record’s number within its own partition, starting at 0. Offsets from different partitions cannot be compared, and there is no single global order for the whole topic.

Topic	Partition	Record offsets in order
orders	0	0, 1
orders	1	0, 1

The two offset-0 entries are different records. This book guarantees order within each partition, not an order between the two partitions.

A record’s lifecycle is: the producer chooses a partition; the broker appends the record to that partition’s log; a consumer actively fetches it; the program runs a processing callback; and only after successful processing does it save its consumption progress. Reading does not delete the log record, so another consumer or a restarted consumer can read it again. Log retention determines when old records are deleted; consumption progress says where a consumer resumes. They are separate states. Chapter 4 explains the failure window between processing and saving progress.

Chapter and steps	Question answered
Chapter 1, 1–4	How do we encode, append, and read records, and identify a crash tail?
Chapter 2, 5–8	How do indexes speed lookup, how are logs segmented, and how do head retention and tail truncation differ?
Chapter 3, 9–12	How do we route to partitions, discover broker locations, and recover TCP bytes into complete requests?
Chapter 4, 13–16	How do we batch sends, read records, save consumption progress, and understand replay after failure?
Chapter 5, 17–20	How do consumers share partitions, and how are stale-member requests rejected?
Chapter 6, 21–24	How do brokers replicate logs, and which records may ordinary consumers see?
Chapter 7, 25–28	How do we elect a replica after failure, repair divergence, and admit a replica again?
Chapter 8, 29–31	How do we observe cross-layer failures, identify repeated sends, and make local effects replay-safe?

1.2 What This Course Builds—and What It Does Not

This project makes the path above runnable: partition logs on disk, TCP requests, producing and consuming, consumer offsets, a single-topic consumer group, and pull-based replication with controlled failover among local brokers. It is not an official Kafka client and does not use the Kafka wire protocol; do not point the official `KafkaProducer` or `KafkaConsumer` at this program.

The three Java source roots are part of the course design. `provided/` contains data types, I/O, transport, serialization scaffolding, lifecycle management, and test fixtures, keeping the focus on algorithms rather than repeated setup. `exercises/` is the only workspace students need to complete step by step. All step types and public signatures are visible in advance; methods not yet reached throw `ExerciseNotImplementedException` with the step number. `reference/` contains an isolated complete implementation for comparison after trying the contract and for course verification. Its class names match the exercise sources, so student and reference source roots must never be compiled together. Tests reuse the same black-box contract in both modes; the student runtime classpath excludes reference classes.

This is not production-grade Kafka: it has no Kafka wire-protocol compatibility, transactions, compression, compaction, complete controller consensus, or controller HA. Chapter 8’s limited idempotent-producer protocol and durable local effect ledger are neither Kafka production protocols nor end-to-end transactions. They use a trusted teaching control plane within one JVM; although multiple brokers have TCP servers, threads, and independent disk directories, their replicas still share that control plane. The course can demonstrate controlled failover, but cannot establish consensus under arbitrary network partitions or production-grade availability. Relevant steps explain the specific differences.

1.3 Preparing Java 21+ and Course Commands

The repository uses the Gradle Wrapper `./gradlew`; Gradle 9.8.1 runs on Java 17+, while Java compile, test, and demo tasks require a Java 21+ JDK and compile with `--release21`. Maven, Docker, and a running Kafka are not required. The scripts use Bash 3.2+, `find`, `awk`, `curl`, and either `sha256sum` or `shasum`; `tar` and `install` prepare Tectonic, while macOS Pandoc extraction also needs `unzip`. EPUB diagrams require Poppler’s `pdftoppm`; `./gradlew :setupBook` checks for it but does not install system packages. Install Poppler on Linux/WSL with `sudo apt-get install poppler-utils`, or on macOS with `brew install poppler`. First check `java -version`, `javac -version`, and `./gradlew --version`. You may use any suitable JDK through

JAVA_HOME or PATH; SDKMAN is optional. The project does not silently change SDKMAN, system packages, or shell configuration.

`./gradlew :setup` resolves the pinned JUnit 1.11.4 dependency and verifies its checksum through Gradle; it does not store JUnit under `.tools/`. `./gradlew :setupBook` prepares checksum-verified Tectonic 0.17.0 and Pandoc 3.12.1 under `.tools/`, and checks Poppler without installing system software. `./gradlew :doctor` checks Gradle, Java, managed JUnit, and optional book tools. `./gradlew :compile` compiles the student skeleton and tests; successful compilation does not mean the exercises are implemented. The default course and list language is English; `COURSE_LANG=zh` `./gradlew :list` displays Chinese step titles. `./gradlew :book` builds the English PDF and EPUB at `build/book/en/simpleKafka.pdf` and `build/book/en/simpleKafka.epub`; `COURSE_LANG=zh` `./gradlew :book` builds the Chinese pair at `build/book/zh/simpleKafka-zh.pdf` and `build/book/zh/simpleKafka-zh.epub`. Each command also updates that language's checked-in PDF and EPUB under `docs/book/dist/`. Readers can open those distributed files without Java or build tools. The book task starts the Wrapper JVM but does not compile Java or resolve JUnit. The first build needs a network connection for the TeX bundle; after warming both languages, use `BOOK_OFFLINE=1` `./gradlew :book --offline` and `COURSE_LANG=zh BOOK_OFFLINE=1 ./gradlew :book --offline`. Gradle `--offline` controls Gradle dependencies; `BOOK_OFFLINE=1` selects the cached TeX bundle for both Tectonic builds.

1.4 Understanding Binary Data and File I/O

A file stores bytes; it does not remember field boundaries for a program. The UTF-8 character “值” occupies 3 bytes, so its value length in the course example is 3, not its character count 1. An ordinary record has $\text{length}=28+3=31$ and occupies $4+31=35$ bytes in total. An encoding must specify field order, integer width, and byte order. This course uses big-endian order: for example, the four bytes of integer 1 are 00000001, with the most significant byte first.

A `ByteBuffer` is a byte view with a capacity, position, and limit. Position is the next read/write location; limit is the end of the current readable/writable region. In write mode, `flip()` resets position to zero and sets limit to the position just written, so those bytes can then be read; flip does not clear the data. A buffer position is a byte cursor, while a record offset in later exercises is a logical number within a partition; they cannot be substituted. A decoder should advance only past bytes it has successfully consumed.

`FileChannel` can read or write at a specified file byte position and can request that written data be forced to storage. One `channel.read` may fill only part of a buffer, and one `write` may write only part of it; the provided `ChannelIO.readFully/writeFully` loops to handle this. Step 2 uses a lock to make concurrent appends to one log complete one at a time, so batches cannot interleave. The lock protects a short shared-state operation; do not hold it while waiting for network I/O. After each batch, `force(false)` is a simplified request to persist file data, not a mathematical guarantee against power loss, device caches, media damage, or filesystem semantics. CRC32C detects accidental changes to record bytes; it does not repair errors or prove that data came from a trusted source.

1.5 How to Read Test Failures

First read the failed Step test name and assertion, then return to the contract in this chapter. The first exception in a test report is usually more useful than the last part of the stack trace: an unimplemented method identifies its step and method; course errors should preserve `ErrorCode`; an unexpected `IOException` may be wrapped as `STORAGE_ERROR` with its cause preserved. Do

not map errors to an empty list or a successful default value just to make tests green. A single-step diagnostic command runs only that step and does not mean its cumulative dependencies are satisfied; the cumulative command reevaluates every earlier step. The course runner prints named tests grouped by step, per-step and overall counts, failure assertions with exact method navigation, and rerun commands. The report is always in English; legacy JUnit XML is saved under `build/student/reports/` or `build/reference/reports/`. These custom JavaExec tasks print plain text. Standard Gradle `:exercises:test` and `:reference:test` tasks support IDE debugging and exact `--tests` selection; their XML/HTML reports use `build/<mode>/gradle-test-results/` and `build/<mode>/gradle-test-reports/`. The runner retains the `NO_COLOR` and `TERM=dumb` color gate.

Command	Meaning
<code>./gradlew :list</code>	Show all 31 steps, their editable methods, and prerequisites.
<code>./gradlew :stepTest -Pstep=12</code>	Diagnose Step 12 only; useful for locating the current problem when it fails.
<code>./gradlew :test -Pstep=12</code>	Run cumulative tests from Step01Test through Step12Test.
<code>./gradlew :referenceTest -Pstep=31</code>	Run the complete 31-step contract against the isolated reference implementation.
<code>./gradlew :referenceDemo</code>	Run the completed reference TCP/failover scenario.
<code>./gradlew :referenceConsistencyDemo</code>	Run the real TCP and disk consistency scenarios for Chapter 8.

Step numbers accept decimal values from 1 through 31, with optional leading zeroes (for example, `-Pstep=1` or `-Pstep=01`). Chapter boundaries are Steps 4, 8, 12, 16, 20, 24, 28, and 31: a chapter is complete only when cumulative tests through that boundary all pass. Passing `./gradlew :test -Pstep=8` means Steps 1–8 all pass; passing `./gradlew :stepTest -Pstep=8` means only the Step 8 diagnostic ran and does not authorize progression. Full-course tests must not put reference sources on the student classpath, and reference mode cannot substitute for exercise progress.

1.6 Concepts Already Introduced and Where to Go Next

Keep three ideas in mind: a topic names a category, its partitions are independently ordered logs, and each record's offset has meaning only within its own partition; sending, reading, and business processing are distinct events, and reading does not delete a record; log retention determines when records are deleted, while consumption progress determines where one consumer resumes. Every later step introduces its new terms and invariants before relying on them; no Kafka abbreviation is prior knowledge.

Use these questions to navigate the next steps: [Step 1 \(Record encoding and decoding\)](#) explains how a record becomes bytes with clear boundaries; [Step 4 \(Crash recovery\)](#) shows how to distinguish an unfinished file tail from complete damaged data; [Step 14 \(Pull-based consumer\)](#) demonstrates how a consumer's read position advances; [Step 22 \(ISR and high watermark\)](#) explains which records ordinary consumers may see when copies of a log have different progress; [Step 25 \(Clean election and leader epochs\)](#) shows how to reject requests from an old broker role after another broker takes over writing.

Chapter 2

Records and Single-Segment Persistence

This chapter starts with a practical question: after a process exits, how can a message be read back from a file unchanged? Memory disappears with the process, while a file stores only bytes and does not remember which bytes belong to one record. The course first defines binary record boundaries, then appends records to an append-only log; later steps use a logical offset to identify a record and a byte position to find it in a file. LEO (log end offset) is the next offset available for append, not the last record's offset and not the file's byte length. These are course concepts, not Kafka's disk format.

length	CRC32C	offset	timestamp	key/value len	key value
4 B	4 B	8 B	8 B	8 B	variable

Figure 2.1: A disk record. length does not include its own leading 4 bytes.

2.1 Step 1: Record Encoding and Decoding

Goals and prerequisites

Goal: Define stable on-disk boundaries so writers and readers assign the same meaning to every byte.

Before you begin: Understand lengths, fixed-width integers, UTF-8 bytes, and CRC32C; prior knowledge of binary formats or file scanning is not required.

Background and reasoning

Numbers and fields in a file are only bytes; a format must specify their order, width, and byte order before those bytes can be decoded. This course uses big-endian (most-significant byte first) fields: `length:int32|crc32c:int32|offset:int64|timestamp:int64|keyLength:int32|valueLength:int32|key|value`. `length` is counted from the CRC field and excludes its own leading 4 bytes. The fixed portion occupies 32 total bytes before payload, of which the `length` value's fixed portion is 28 bytes. CRC32C checks whether bytes from `offset` through the end of `value` match the written checksum; it can detect common accidental corruption, but is not authentication or a security signature.

Field	Width	Meaning
<code>length</code>	int32, 4	Excludes its leading 4 bytes; legal values are 28–1,048,576.
<code>crc32c</code>	int32, 4	Checks offset through the end of value, excluding length and CRC itself.
<code>offset</code>	int64, 8	Nonnegative logical number within a partition.
<code>timestamp</code>	int64, 8	Nonnegative time value stored with the record.
<code>keyLength</code>	int32, 4	-1 means null key, 0 means empty key, otherwise the key byte count.
<code>valueLength</code>	int32, 4	Nonnegative value byte count; value cannot be null.
<code>key, value</code>	Variable	Follow their length fields; count encoded bytes.

Starting point and edit location.

Already provided: The `LogRecord` type represents a record with a logical offset; `RecordData` stores key, value, and timestamp; `CourseException`, `ErrorCode`, big-endian fields, and CRC32C utilities are provided.

Implement in: At repository root, implement `publicstaticByteBuffer encode(LogRecord record)` and `publicstaticLogRecord decode(ByteBuffer source)` in `exercises/src/main/java/io/simplekafka/storage/RecordCodec.java`. Do not edit provided types, the reference implementation, or tests.

Prerequisites

This is the first course step; you only need to read basic Java classes, methods, and exceptions. A byte is the unit used for file lengths (8 bits): one UTF-8 character may occupy multiple bytes, so lengths count encoded bytes rather than characters. The next step writes these encoded bytes to a file.

Inputs, outputs, and state

- `encode` accepts a `LogRecord(offset, data)`; a null record reports `INVALID_REQUEST`. `offset` and `timestamp` must be nonnegative `long` values; `value` must be present (it may contain zero bytes); `key` may be null, an empty array, or a nonempty byte array. Success returns a new buffer with `position=0` and `limit` exactly at the end of this record. It reads or changes no file state.
- `decode` starts at the source's current position; a null source throws `NullPointerException` without consuming input. It interprets fields as big-endian, independent of the source's byte order. Heap, direct, read-only, and slice buffers are valid inputs. Success consumes exactly one record and advances position to that record's end; `limit` is unchanged and bytes after it remain for the next call.
- The legal length range is 28 through 1,048,576 inclusive; total record space is `4+length=32+keyBytes+valueBytes`. A null key is written as `keyLength=-1`, while an empty key is 0; `valueLength` must be nonnegative. Ordinary records have no additional identity fields.

Behavioral contract and invariants. Incomplete input (fewer than the 4-byte length prefix, or fewer than the body declared by a legal length) reports `INVALID_REQUEST`. A complete but out-of-range length, invalid field structure, negative `offset/timestamp`, or CRC mismatch reports `CORRUPT_RECORD`. An invalid record passed to `encode` or a record over the size limit reports `INVALID_REQUEST`. `decode` must not change `limit` on success or failure; on failure it must not advance source position.

A complete hand-worked record

First use the independent golden fixture from the tests: `offset=7`, `timestamp=23`, `key=null`, and an empty value, with every integer big-endian. The hexadecimal fields are `0000001cdf159f32`

0000000000000000070000000000000017ffffffff00000000: length=28; CRC32C is `df159f32`; `ffffffff` is keyLength=-1; valueLength=0. The 4-byte prefix is outside length, so the total record is 32 bytes.

A separate size example uses a null key and the UTF-8 value “值” (assume its bytes are `e580bc`, 3 bytes): length=28 + 0 + 3 = 31, and total space is 4 + 31 = 35 bytes. An empty key is encoded with keyLength=0, not -1; this is not the empty-value golden fixture above.

Observable examples and test points. `Step01Test` compares encode/decode with independent byte goldens and covers null versus empty keys, zero-length values, consecutive decoding, several `ByteBuffer` views, every incomplete prefix, out-of-range lengths, CRC/structure corruption, and the maximum size. Key methods: `encodeAndDecodeMatchIndependentBigEndianGoldenRecords` checks the golden bytes; `decodesOneRecordFromHeapDirectReadOnlyAndSliceBuffers` checks supported views; `rejectsEveryIncompletePrefixAndOutOfRangeRecordLength` and `rejectsCorruptionInEveryStoredField` cover errors; `enforcesLengthLimitAndSupportsLargestNonnegativeLogicalValues` checks the maximum. The largest ordinary record has length=1,048,576 and total size=1,048,580; one more value byte must be rejected.

Common pitfalls and differences from real Kafka

Do not count the length prefix in length or include the CRC field in its own CRC range. Do not treat a Unicode character count as valueBytes. A matching CRC32C only says the protected bytes agree with the checksum; it does not prove the source is trusted.

Tiered hints (not the answer)

Level 1

List the 4-byte length, 4-byte CRC, two 8-byte integers, two 4-byte lengths, and variable tail separately.

Level 2

Calculate the length range, the fields it covers, and total allocation as separate quantities; convert UTF-8 to bytes before counting.

Level 3

For each failed decode, decide whether input is incomplete or complete but invalid, then verify the successful position/limit and a trailing sentinel.

Commands and progression. Use `./gradlew :stepTest -Pstep=1` to diagnose this step; `./gradlew :test -Pstep=1` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must a null key and a zero-byte empty key use different representations?
2. What does a successful CRC check tell you, and what does it not tell you?

Self-check explanation A null key means no key is present; an empty key means a key exists with length zero. Collapsing them loses caller data. CRC32C can detect an accidental change in protected bytes, but cannot establish sender identity or replace structural validation.

2.2 Step 2: Log Append

Goals and prerequisites

Goal: Implement continuous offset allocation for a single-segment log and complete the course's required writes before returning success.

Before you begin: Reuse record encoding and understand a lock around shared state, full-write loops, and file byte positions.

Background and reasoning

Appending adds new records to the end of an existing log instead of overwriting old records. A partition log keeps shared state for the next offset and file-tail byte position; a lock is a mutually exclusive section protecting shared state, so only one append at a time can read or update those values. Without that coordination, two threads could assign the same offset or interleave batch bytes. A single `FileChannel.write` may write only part of a buffer; the provided `ChannelIO.writeFully` keeps writing until it is complete. The course requires a complete batch write and `force(false)` before publishing its result.

Starting point and edit location.

Already provided: `SegmentLog.create(Path,long)` creates a segment. The segment, `FileChannel`, `lock`, `baseOffset`, `nextOffset`, `close()`, `RecordCodec.encodedSize(RecordData)`, and `ChannelIO.writeFully(...)` are provided.

Implement in: Implement `public AppendResult append(List<RecordData> records)` in `exercises/src/main/java/io/simplekafka/storage/SegmentLog.java`.

Prerequisites

Complete [Step 1 \(record encoding and decoding\)](#), especially how to calculate a record's complete encoded byte size and reject invalid records before writing. Tests start from a nonzero `baseOffset`, so do not assume every log starts at offset 0.

Inputs, outputs, and state

- Input is a nonempty `List<RecordData>` in append order; each item must meet Step 1's value, timestamp, and maximum-size rules. Output `AppendResult(firstOffset,nextOffset)` is a half-open interval: it includes `firstOffset` and excludes `nextOffset`.
- An empty or null batch, null element, invalid data, oversized record, or an offset that cannot be incremented is rejected with `INVALID_REQUEST`. Known-invalid input must be detected before any disk write.
- On success, the file grows at its tail by the entire batch and LEO advances to `nextOffset`. A write or force I/O failure reports `STORAGE_ERROR`. I/O failure may occur after some or all bytes have been written; the course does not promise rollback, so do not claim that disk is unchanged after every failure. A closed segment cannot be appended to.

Behavioral contract and invariants. Allocate consecutive offsets from the current LEO while holding the lock; keep each batch adjacent and prevent concurrent batches from interleaving. Precheck every record before writing. Return the interval and publish the new LEO only after all writes complete through `writeFully` and `force` succeeds. Invalid input leaves the file and LEO unchanged.

A complete hand-worked append

Create an empty segment with `baseOffset=5`, so its initial file is empty and `LEO=5`. Append three ordinary records, each with a null key and a one-byte UTF-8 value, so each occupies 33 bytes: `append([a,b])` writes offsets 5 and 6, returns `[5,7)`, leaves the file at 66 bytes and `LEO=7`; then `append([c])` writes offset 7, returns `[7,8)`, leaves the file at 99 bytes and `LEO=8`. If a later invalid record is rejected, the file and `LEO` remain 99 and 8, and the next valid record starts at 8.

Observable examples and test points. `Step02Test: appendsBatchesFromNonzeroBaseAndPreservesIndependentDiskEncoding` verifies `[5,7)`, `[7,8)`, and disk fields at `base=5`; `rejectsInvalidBatchesAtomicallyWithoutCreatingOffsetGaps` checks empty/null/null-containing/oversized rejection, unchanged file/`LEO`, and the next offset; `rejectsAppendAtMaximumOffsetWithoutChangingEmptyFile` checks overflow; `concurrentBatchesReceiveDisjointRangesWithoutInterleaving` checks disjoint, noninterleaved ranges.

Common pitfalls and differences from real Kafka

If an invalid third record is detected only after writing the first two, a partial batch can remain in the log. Do not substitute raw value length for `RecordCodec`'s complete encoded size. A successful force is the course's condition for returning success, not proof that a physical medium can never lose data during a power failure.

Tiered hints (not the answer)

Level 1

Write down the current `LEO` before the call and use it as the batch's first offset; `nextOffset` starts the next batch.

Level 2

Separate prechecking the entire input from reading current `LEO` under the lock, and use `writeFully` for partial writes.

Level 3

In a concurrent test, check both `AppendResult` intervals and every file offset in order, not just the final `LEO`.

Commands and progression. Use `./gradlew :stepTest -Pstep=2` to diagnose this step; `./gradlew :test -Pstep=2` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must the entire batch be prevalidated before the first disk write?
2. If a write throws an I/O exception, why can't you automatically conclude that it wrote nothing?

Self-check explanation Whole-batch prevalidation prevents predictable parameter errors from leaving half a batch or consuming offsets. An I/O error can occur after the operating system accepted some bytes, so a failed return does not imply a disk rollback; recovery must identify a valid boundary.

2.3 Step 3: Offset-Based Reads

Goals and prerequisites

Goal: Read complete records from a logical offset while honoring both record-count and encoded-byte budgets.

Before you begin: Distinguish logical offsets, file byte positions, hints, and the append cursor.

Background and reasoning

A caller uses an offset to name the first logical record it wants; a file reader needs a byte position from which to scan. An offset numbers records in a partition log, while a byte position counts bytes within this segment; they have different units and cannot be substituted for one another. `bytePositionHint` only reduces searching: it must be a record boundary and cannot skip the target. Results still start at the requested offset. A record cannot be split, and `maxBytes` counts each complete encoded record including its 4-byte length prefix. Explicit-position file reads also avoid changing the append position.

Starting point and edit location.

Already provided: Segment creation, LEO, locking, and record decoding are provided; `read(long, int, int)` supplies the segment start as the default hint.

Implement in: Implement `public List<LogRecord> readFrom(long offset, long bytePositionHint, int maxRecords, int maxBytes)` in `exercises/src/main/java/io/simplekafka/storage/SegmentLog.java`.

Prerequisites

Complete [Step 2 \(log append\)](#) and understand that successful appends assign consecutive offsets while the file grows by complete encoded bytes. Without a sparse index, the target can be found by scanning from the segment start; this step's hint is a physical search position, not a replacement for the logical request.

Inputs, outputs, and state

- `offset` must be in `[baseOffset, LEO]`; exactly `LEO` is a valid empty read. `maxRecords` and `maxBytes` must both be positive. Returned records have consecutive offsets and do not exceed either budget.
- `bytePositionHint` is a nonnegative byte position in this segment. It must be a real record boundary and no later than the physical position for the requested offset; an EOF hint is valid only when `offset=LEO`. A negative, skipping, interior, or past-EOF hint reports `INVALID_REQUEST`; an out-of-range offset reports `OFFSET_OUT_OF_RANGE`; a corrupt complete stored record reports `CORRUPT_RECORD`; a closed segment or underlying I/O failure reports `STORAGE_ERROR`.
- If the first complete record alone exceeds `maxBytes`, return an empty list rather than part of a record. Reads, successful or not, do not append, change `LEO`, or advance persistent consumer progress.

Behavioral contract and invariants. Scan forward from the verified hint boundary, skipping complete records whose offsets are before the target; output begins at the requested offset. Count each record and its complete encoded bytes only after the entire record is available. Reading does not move the append cursor.

Hand calculation: full records, budget, and hint

Assume base=0 and append five records in order. Each has a null key and a 1-byte value in the ordinary format, so each occupies $32 + 1 = 33$ bytes. Their offsets are 0–4, their file starting positions are 0, 33, 66, 99, 132, and EOF is 165. Calling `readFrom(2,33,2,66)` uses hint=33, the valid boundary for offset 1; the reader skips offset 1 and still returns offsets 2 and 3. Budgets of 32, 33, 65, and 66 bytes return 0, 1, 1, and 2 records. offset=5 with hint=165 returns an empty list.

Observable examples and test points. `Step03Test: validatesRecordBoundaryHints` checks 33-byte positions, hints -1/1/67/166, target-skipping hint 165, budgets, and errors; `enforcesNonzeroBaseBoundsAndKeepsReadPagingSeparateFromAppendPosition` checks nonzero base and paging without moving the append position; `rereadsCompleteRecordsAndRejectsDamageWrittenThroughAnotherHandle` checks `CORRUPT_RECORD` after external damage.

Common pitfalls and differences from real Kafka

Treating offset=2 as file byte 2 starts in the middle of a record; treating the hint as the first returned offset changes the request’s meaning. Byte budget is not just payload size: lengths, CRC, and field headers use disk space too.

Tiered hints (not the answer)**Level 1**

Draw a two-row mapping from offsets to file starting positions and label the units.

Level 2

Check that a hint is not after the target and is a record boundary; results start at the requested offset, not the hint.

Level 3

Add complete record sizes for budgets 32, 33, 65, and 66, and separately test an empty result when the first record exceeds the budget.

Commands and progression. Use `./gradlew :stepTest -Pstep=3` to diagnose this step; `./gradlew :test -Pstep=3` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. If a hint points to offset 1’s boundary, why must a read at offset 2 not return offset 1?
2. When the first record does not fit maxBytes, why is returning part of it not reasonable?

Self-check explanation The hint only locates a place to begin searching; the requested offset defines the result. A partial record cannot be decoded because it contains length and checksum fields, so an empty result tells the caller that no complete record fits this budget.

2.4 Step 4: Crash Recovery**Goals and prerequisites**

Goal: Recover a segment’s continuous LEO after restart and truncate only a file tail that can be proven incomplete.

Before you begin: Distinguish a partly written record from a complete but corrupt record.

Background and reasoning

A process may exit while writing the length, fixed fields, or value. Recovery starts from `baseOffset` and compares each complete record's offset with the next expected offset. Record boundaries require both length and complete fields: when EOF cannot form the next record, returning to that record's start is safe; a complete record with an invalid CRC, field, or continuous offset is corruption and must not be silently discarded.

Starting point and edit location.

Already provided: `SegmentLog.open(Path, long)` opens an existing file and invokes recovery. File positions, `baseOffset`, the LEO field, and `codec/read/write` paths from Steps 1–3 are established.

Implement in: Implement `public long recover()` in `exercises/src/main/java/io/simpl/ekafka/storage/SegmentLog.java`.

Prerequisites

Complete [Step 3 \(offset-based reads\)](#) and be able to identify record boundaries from the length prefix and fixed fields. This method is called explicitly and also when `open` opens an old file; calling `recover` again after `open` succeeds must give the same result.

Inputs, outputs, and state

- Input is an open segment and its `baseOffset`; an empty file may have any nonnegative base. The result is recovered LEO, the next offset for append; for an empty segment `LEO=baseOffset`.
- Fewer than 4 remaining bytes means an incomplete length prefix: truncate at the current record start. A complete length outside `[28,1,048,576]` is `CORRUPT_RECORD`, even if no body is present. A legal length whose declared body is incomplete is truncated at the current record start.
- A complete record with bad CRC/fields or an offset other than the next expected offset is `CORRUPT_RECORD`; reject it and leave the original file bytes unchanged. A closed segment or underlying I/O failure reports `STORAGE_ERROR` and preserves the I/O cause.

Behavioral contract and invariants. After successful recovery, the file contains only a continuous, complete, valid prefix, and LEO points to its end. Only an incomplete tail record may be repaired; never skip or rewrite a complete corrupt record. Repeating recovery or closing and reopening must be idempotent.

Hand calculation: recover, then append

Assume `base=0` and the file has two complete ordinary records, each with a null key and 1-byte value. Offsets 0 and 1 occupy 33 bytes each, so the valid prefix is 66 bytes and the next expected offset is 2. The third record at offset 2 declares the same total size of 33 bytes; only its first 17 bytes are in the file (including its complete length=29 but an incomplete body), making the file 83 bytes long. `recover` truncates to byte position 66 and returns `LEO=2`; appending another one-byte value returns `[2,3)`. In contrast, if that third record has only a complete length prefix declaring length=1,048,577, the length itself is illegal, so recovery reports `CORRUPT_RECORD` without truncation.

Observable examples and test points. `Step04Test: truncatesEveryIncompleteTailPrefixAndKeepsACompleteThirdRecord` and `truncatesAnIncompleteFirstRecordAtZeroAndNonzeroBases` verify truncation only for incomplete tails; `recoversEmptyAndNonzeroBaseFilesIdempotentlyAndRejectsWrongBase` checks empty/nonzero-base recovery; `rejectsCompleteCorruptionAndOffsetDiscontinuitiesWithoutChangingFileBytes` checks CRC, fields, illegal length, and offset gaps leave complete corrupt files unchanged.

Common pitfalls and differences from real Kafka

Treating every parse failure as a tail fragment swallows complete-record corruption; treating every EOF as a corrupt file prevents recovery from an interrupted write. Only after a legal length tells you the expected body size can you distinguish a short body from an illegal length.

Tiered hints (not the answer)

Level 1

The scan needs only the current valid byte boundary and the next expected offset.

Level 2

Classify “fewer than 4 prefix bytes / illegal length / legal length but short body / complete valid or complete corrupt.”

Level 3

Reopen two valid records plus a partial tail; then separately use a complete record with a bad CRC and compare error codes and file bytes.

Commands and progression. Use `./gradlew :stepTest -Pstep=4` to diagnose this step; `./gradlew :test -Pstep=4` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can't an illegal number in a complete length prefix be treated as a body that was only partly written?
2. Why does recovery return LEO=2 rather than the last record's offset=1?

Self-check explanation An illegal length is a complete declaration that the record format does not accept, not a legal record that lacks bytes, so it is corruption. LEO is the next append position; after a record at offset 1, the next offset is 2.

Chapter 3

Segments, Indexing, Retention, and Truncation

As a single log file grows, restart scans become longer; deleting old data from the middle would otherwise require rewriting everything after it. A partition log is therefore made of consecutive segment files: each segment contains a range of offsets, the active segment receives appends, and closed segments are no longer appended to. A byte position within a segment counts file bytes, not partition offsets. A sparse index stores only some offset-to-byte-position hints and can be rebuilt from the main log; it speeds lookup but is not record truth. Retention deletes a complete old segment prefix, while truncation deletes a tail after a nextOffset—their directions and purposes differ.

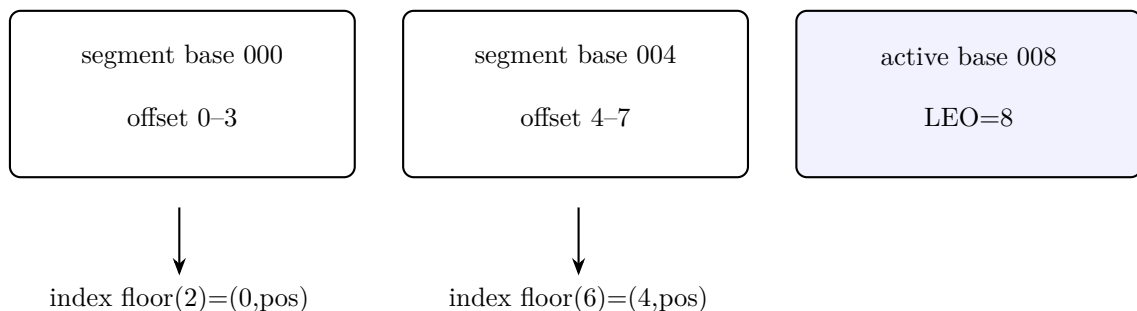


Figure 3.1: Logical ranges in a segmented log and the sparse index within each segment. The base offset in a segment filename is its logical starting point.

3.1 Step 5: Sparse Index

Goals and prerequisites

Goal: Build a persistent, rebuildable sparse mapping from offsets to byte positions.

Before you begin: Understand strictly increasing observations, floor lookup, and scanning the primary log.

Background and reasoning

An index entry is two big-endian int64 values, **offset**, **position**, for 16 bytes total. Rather than index every record, persist a point every intervalRecords records according to the logical distance from the segment base; always persist the first record. floor(target) chooses the greatest indexed

offset not larger than the target, so a later scan can begin at a known record boundary. Even with no index entry, scanning safely from `(baseOffset,0)` is possible. A missing or damaged index can be rebuilt from the primary log, but an index cannot repair a damaged log.

Starting point and edit location.

Already provided: The `SparseIndex` path, `baseOffset`, `intervalRecords`, persisted file, loading, and close lifecycle are provided; `SegmentLog` can rescan the primary log.

Implement in: Implement `public synchronized void add(long offset, long position)`, `public synchronized IndexEntry floor(long offset)`, and `public synchronized void rebuild(Path logFile, long requestedBaseOffset)` in `exercises/src/main/java/io/simplekafka/storage/SparseIndex.java`.

Prerequisites

Complete [Step 4 \(crash recovery\)](#) and know how to validate complete records, CRCs, and continuous offsets in the main log. An index entry's position is a byte count within the segment file; its offset is a record number in the partition. They cannot be interchanged.

Inputs, outputs, and state

- `add` receives a record's observed logical offset and its byte position within the segment. The first observation must be `(baseOffset,0)`; every later observation must strictly increase both offset and position, and position must be nonnegative. An observation that is not persisted still participates in the next ordering check.
- Persist an observation only when `(offset-baseOffset) mod intervalRecords=0`; the first `(baseOffset,0)` is always persisted. A rejected observation reports `INVALID_REQUEST` and leaves the index file and ordering state unchanged. A closed index or I/O failure reports `STORAGE_ERROR`; `floor` on a closed index also reports `STORAGE_ERROR`.
- `floor` returns the greatest indexed entry at or before target; if none qualifies (including when target is below base), return `(baseOffset,0)`. `rebuild` receives the main-log path and required base, clears old entries, scans and validates the log, and recreates all points that should be persisted; it does not change log contents. An invalid path or mismatched base reports `INVALID_REQUEST`; corrupt log contents report `CORRUPT_RECORD`; a closed index or underlying I/O failure reports `STORAGE_ERROR`.

Behavioral contract and invariants. Each index-file entry stores an 8-byte offset followed by an 8-byte position, both big-endian. Use every observation to validate strict increase; calculate persistence frequency from the offset's distance from base, not from the number of add calls or a fixed byte interval. Missing, truncated, stale, or invalid indexes must be rebuildable from the current log; rebuilding must not make an invalid log appear valid.

Hand calculation: index points and floor

Assume `baseOffset=8` and `intervalRecords=2`. The segment contains offsets 8, 9, 10, 11; each record has a null key and a 1-byte value, so each occupies 33 bytes and starts at positions 0, 33, 66, 99. Observe all four in order: the first offset 8 writes `(8,0)`; offset 9 is checked for order but not persisted; offset 10 is two records from base, so it writes `(10,66)`; offset 11 is checked but not persisted. The index file contains exactly two 16-byte entries; `floor(11)=(10,66)`, `floor(9)=(8,0)`, and the no-candidate convention makes `floor(7)=(8,0)` too. If the index is deleted, scanning the four intact log records reconstructs the same two entries.

Observable examples and test points. `Step05Test: writesBigEndianEntriesAtIntervalAndFloorsSafely` checks 16-byte entries, interval, and floor; `supportsNonzeroBasesAn`

`dReopensPersistedPoints` checks nonzero bases and reopening; `rejectedObservationsLeaveTheIndexAndOrderingStateUnchanged` checks state preservation; `rebuildsMissingTruncatedAndStaleIndexesFromTheCurrentLog` and `rebuildRejectsMalformedLogsWithoutChangingTheirBytes` check index rebuilding and rejection of malformed logs.

Common pitfalls and differences from real Kafka

floor is not “any nearest entry” : it must not be after the target. An index position must point to a record start or a reader may interpret payload bytes as a header. The index can be discarded and rebuilt; the main log cannot be replaced with a guess.

Tiered hints (not the answer)

Level 1

Write the candidate condition $\text{offset} \leq \text{target}$, then define the result when there are no candidates: `(base,0)`.

Level 2

Separate the ordering state for every observation from whether that point is persisted; the interval is based on `offset-base`.

Level 3

Delete the index file, create `SparseIndex` again, and observe rebuilding through a real log read rather than checking only an in-memory floor.

Commands and progression. Use `./gradlew :stepTest -Pstep=5` to diagnose this step; `./gradlew :test -Pstep=5` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must the primary log still be scanned after finding a floor entry?
2. Why can a bad index be rebuilt while a bad primary log cannot use the same recovery strategy?

Self-check explanation The index is only a search start that does not pass the target; it usually has no exact point for the target, so the records still need to be validated and scanned. An index is redundant guidance that can be recalculated from the record facts; corrupted log contents have no other trusted source from which to infer them.

3.2 Step 6: Segmented Log

Goals and prerequisites

Goal: Combine single-segment logs into a capacity-rolled partition log and make reads continuous across segments.

Before you begin: Understand active segments, complete-record capacity, index hints, and global LEO.

Background and reasoning

`PartitionLog` holds multiple `SegmentLog` instances in `baseOffset` order. An active segment is the final segment that can still receive appends; if it cannot fit the next whole record, close

it and create a new segment at the partition's current LEO. Capacity is based on complete encoded record bytes, not value length. When the segment is empty, one legal record larger than `segmentBytes` may occupy it; otherwise that record could never be appended. A cross-segment read finds the segment containing the target offset, uses that segment's index floor as a hint, then scans forward; `maxRecords`/`maxBytes` are the total budgets for the whole read and do not reset at a segment boundary.

Starting point and edit location.

Already provided: Each segment's create/open/recover, sparse index, file discovery, and close path are provided. The `PartitionLog` skeleton supplies `mutationVersion()` and private `markMutation()`.

Implement in: Implement `public synchronized AppendResult append(List<RecordData> records)` and `public List<LogRecord> read(long offset, int maxRecords, int maxBytes)` in `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`. The `PartitionLog(Path, long, int)` constructor is provided.

Prerequisites

Complete [Step 5 \(sparse index\)](#); understand `SegmentLog` append/read and the index's floor hint. Segment discovery recognizes only regular files named with 20 digits and the `.log` suffix; the paired `.index` is index data. Other files and directories are not segments to clean up.

Inputs, outputs, and state

- append takes an ordered batch and returns one global half-open range [`firstOffset`, `nextOffset`); offsets remain continuous across segment boundaries. An empty/invalid batch or offset overflow reports `INVALID_REQUEST` and must not create a segment first. A closed log or underlying I/O failure reports `STORAGE_ERROR`; failure after a valid append starts does not promise to roll back files.
- read accepts an offset in the retained range [`logStartOffset`, `LEO`]; `LEO` returns empty. `maxRecords`/`maxBytes` must be positive. Results are ordered and continuous, and share total record-count and complete encoded-byte budgets across all segments. Out-of-range offset is `OFFSET_OUT_OF_RANGE`; a segment gap or corrupt record is `CORRUPT_RECORD`; I/O/closed-log failure is `STORAGE_ERROR`. If the first record exceeds the byte budget, return empty.
- Each fully prevalidated append batch increments `mutationVersion` on this open object once before its first file write or roll; a batch spanning segments still increments once. Invalid input does not increment it. This in-memory mutation generation is not persisted; an I/O failure after writing starts also means an earlier recovery proof is no longer trustworthy. `LEO` and segment files change on successful append; reads do not change them.

Behavioral contract and invariants. Roll only when a nonempty active segment cannot fit the next complete record; an empty segment may hold one legal oversized record on its own. Precheck the full batch before first create/roll so invalid input cannot leave an empty segment. Reopening scans registered segments and recovers `LEO`; unfamiliar files and directories must remain untouched.

Hand calculation: segment rolling and a cross-segment budget

Case A: `segmentBytes`=64, with three ordinary records each having a null key and empty value, for 32 bytes each. Offsets 0 and 1 exactly fill the base-0 segment at 64 bytes; before offset 2, roll to base 2, whose size is 32. One append returns `[0,3)`, `bases`=[0,2], `sizes`=[64,32]. Case B: three records in order occupy 32, 132, and 32 bytes, with `segmentBytes` still 64. The 132-byte record cannot fit in the existing nonempty segment, so it occupies a segment by itself at base 1;

the following 32-byte record starts a new segment at base 2. Segment sizes are [32,132,32]. A segment may exceed its target only because one legal record itself is larger and the segment was empty.

For case A, `read(0,3,64)` returns only 0 and 1; a 96-byte budget returns 0, 1, and 2. When moving into the second segment, the remaining budget is 32; it must not reset to 64.

Observable examples and test points. `Step06Test: rollsAtExactByteBoundaries` and `WritesIndependentRecords` checks segment capacity; `appliesReadBudgetsAcrossSegmentsAndValidatesBoundsAndLimits` checks cross-segment budgets; `invalidBatchDoesNotMutateAnExistingMultiSegmentLog` checks no directory mutation for invalid batches; `acceptsAnOversizedRecordBetweenRecordsInOneBatchAndReopensEverySegment` checks a legal oversized record alone; `recoversOnlyAnIncompleteActiveTailAndRebuildsIndexesOnReopen` and `refusesCompleteCrcCorruptionAndSegmentBaseGapsWithoutChangingLogs` check recovery boundaries; `concurrentBatchesStayContiguousAndSurviveReopen` checks continuity after concurrent appends and reopen.

Common pitfalls and differences from real Kafka

Counting only payload misses the length, CRC, and fixed fields; resetting `maxBytes` for each segment exceeds the caller's total budget. A single-segment append does not roll segments—rolling belongs to `PartitionLog`.

Tiered hints (not the answer)

Level 1

Use `RecordCodec.encodedSize` for each complete record, then compare with the active segment's remaining bytes.

Level 2

Walk the batch while tracking one global `nextOffset` and current segment size; when a read enters a new segment, carry forward the same remaining budget.

Level 3

Check directory bases, each segment's exact size, `mutationVersion`, and record-by-record results after reopen.

Commands and progression. Use `./gradlew :stepTest -Pstep=6` to diagnose this step; `./gradlew :test -Pstep=6` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must an invalid batch be rejected before creating the next segment?
2. What budget state must a read carry from one segment to the next?

Self-check explanation Prevalidation prevents an invalid request from creating an empty segment or changing the directory. A byte budget is accumulated for one call; switching physical files does not restore the bytes already used.

3.3 Step 7: Log Retention

Goals and prerequisites

Goal: Delete expired closed segments at the retained prefix while preserving the active segment, LEO, and actual new start.

Before you begin: Distinguish physical retention from consumer progress.

Background and reasoning

The current retained range is `[logStartOffset, LEO)`. `deleteBefore` is an explicit synchronous call; it does not run automatically when a consumer commits an offset. A segment cannot be split by individual record: only closed segments can be deleted, and the last segment must remain active so the log has an append target. The new start is the base of the first remaining segment; it may be lower than the requested threshold because segment boundaries need not match that threshold.

Starting point and edit location.

Already provided: The `PartitionLog` skeleton owns multi-segment discovery, the start/end offsets, and closing paths, and provides `mutationVersion()` and private `markMutation()`.

Implement in: Implement `public synchronized long deleteBefore(long offset)` in `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`.

Prerequisites

Complete [Step 6 \(segmented log\)](#); understand the offset ranges determined by a segment base and the next segment base, and why the active segment cannot be deleted. A consumer commit is separate state and is not an input to this method.

Inputs, outputs, and state

- `offset` is a nonnegative long retention bound. A negative value reports `INVALID_REQUEST` and changes no state. Return the actual `logStartOffset`, not the requested threshold.
- Delete only a closed segment whose `endOffset` (the next segment base) is at or below the threshold, together with its `.index` at the same base. Keep the active segment and at least one segment. Do not delete unfamiliar files or directories. Deletion does not move LEO, alter remaining segment contents, or adjust consumer positions.
- If at least one segment is actually deleted, increment `mutationVersion` once before the first deletion (at most once per call). If no segment qualifies, return the current start and leave the version unchanged. A read below the new start reports `OFFSET_OUT_OF_RANGE`; do not silently seek to the start. A closed log or underlying I/O failure reports `STORAGE_ERROR`; if deletion fails midway, the operation does not promise to roll back deletions that already happened.

Behavioral contract and invariants. Delete only complete closed segments as units and delete their log/index pair; never remove the active or final segment. Reopen must recover the same `logStartOffset` and LEO; a no-op does not increment `mutationVersion`.

Hand calculation at retention boundaries

Assume nine records at offsets 0–8, each with a null key and empty value, occupying 32 bytes; `segmentBytes=96` makes segments at bases 0, 3, and 6, each with three records and 96 bytes.

LEO=9, and base 6 is active. `deleteBefore(4)` deletes only base 0, whose `end=3`, so the actual start is 3. `deleteBefore(9)` then deletes base 3, whose `end=6`, so the start is 6. `deleteBefore(1000)` still keeps the active base-6 segment, so start remains 6. LEO stays 9 for all three calls; the third call deletes nothing and does not increment `mutationVersion`. The threshold 4 does not become the returned start 3 because deletion operates on whole segments.

Observable examples and test points. `Step07Test: deletesOnlyEligibleClosedSegmentsAndKeepsTheActiveSegment` checks `end<=threshold`, paired indexes, start/LEO, and unfamiliar files; `retentionKeepsTheOnlyActiveSegmentWhetherEmptyOrNonempty` checks that the sole segment remains; `deletesClosedSegmentAtItsExactEndOffset` checks deletion at the inclusive threshold boundary.

Common pitfalls and differences from real Kafka

Deleting only `.log` leaves an orphaned index; treating the threshold as “delete all offsets below it” would split a segment or delete active data. Consumer commits and log-file deletion are independent.

Tiered hints (not the answer)

Level 1

For each segment, write its base, exclusive end, and active status before checking `end≤threshold`.

Level 2

Separately consider a single empty segment and a single nonempty segment to ensure one active segment remains.

Level 3

List the `.log/index` files after deletion, reopen, and read before the new start, at the boundary, and at LEO.

Commands and progression. Use `./gradlew :stepTest -Pstep=7` to diagnose this step; `./gradlew :test -Pstep=7` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can `deleteBefore(4)` return `logStartOffset=3` rather than 4?
2. Even if a consumer has committed 9, why does that alone not trigger deletion?

Self-check explanation Retention uses whole segments, and base 3 is the next actual boundary. A commit records processing progress; it does not automatically delete files or prove no other consumer needs the old data.

3.4 Step 8: Tail Truncation

Goals and prerequisites

Goal: Remove a divergent log tail at `nextOffset` and make the log appendable again at that point.

Before you begin: Understand half-open ranges, segment boundaries, and index consistency.

Background and reasoning

`truncateTo(nextOffset)` means “the offset to write next” : retain all records whose offset is less than `nextOffset` and remove the remaining tail. It is opposite to retention—retention removes an old prefix, truncation removes a newer suffix. If the target lies within a segment, truncate that file at the target record’s byte boundary and rebuild its index; if it equals a segment base, keep an empty active segment at that base. This local log operation by itself does not establish a safe replica switch or change consumer positions.

Starting point and edit location.

Already provided: Earlier steps established segment close, file truncation, index rebuilding, and the global segment table; `PartitionLog` provides `mutationVersion()` and private `markMutation()`.

Implement in: Implement `public synchronized void truncateTo(long nextOffset)` in `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`.

Prerequisites

Complete [Step 7 \(log retention\)](#); know that actual `logStartOffset` may exceed zero and that LEO is an exclusive end. Truncation must stay within the retained range and cannot restore a prefix already deleted by retention.

Inputs, outputs, and state

- `nextOffset` must be in the closed range `[logStartOffset, LEO]`. An out-of-range value reports `OFFSET_OUT_OF_RANGE` and leaves files, indexes, segment table, LEO, and `mutationVersion` unchanged. The method returns no value.
- On success, retain every record with `offset < nextOffset`; delete later records/segments and their indexes; set LEO to `nextOffset`. `nextOffset = LEO` is a no-op. A valid change increments `mutationVersion` once before the first file write/delete; invalid and no-op calls do not increment it. Unfamiliar files remain. A closed log or underlying I/O failure reports `STORAGE_ERROR`; an I/O failure after a valid change starts does not promise rollback, and the incremented `mutationVersion` does not revert.
- After success, append can run immediately and its first record must receive exactly `nextOffset`. Reopen must recover an active segment and index consistent with the truncated log.

Behavioral contract and invariants. The truncation boundary always means the next record’s offset; there is no interpretation “keep record 5 and write 5.” At a segment boundary, preserve an active segment for future appends; inside a segment, adjust the log tail and its index together.

Hand calculation: four independent truncations

Reuse Step 7’s fixture: assume records 0–8 each occupy 32 bytes, `segmentBytes=96`, three full segments at `bases=[0,3,6]` with `sizes=[96,96,96]`, `LEO=9` and `start=0`. Each row below starts from that same fixture and applies one operation:

1. `truncateTo(5)`: retain offsets 0–4; `bases=[0,3]`, `sizes=[96,64]`, `LEO=5`.
2. `truncateTo(3)`: retain 0–2; `bases=[0,3]`, `sizes=[96,0]`. The empty base-3 segment receives the next offset 3.
3. `truncateTo(0)`: retain an empty log; only base 0 remains, size 0, and the next append starts at 0.
4. `truncateTo(9)`: target equals LEO, so nothing changes; `bases/sizes` remain `[0,3,6]/[96,96,96]` and `mutationVersion` does not increment.

Observable examples and test points. `Step08Test: truncatesAtBoundariesWithinSegmentsAtLeoAndAtLogStart` checks segment interiors, boundaries, LEO, and log start; `truncateRebuildsMixedSizeSegmentStateAndIndexPoints` checks log/index consistency; `invalidAndRepeatedTruncationsPreserveTheRetainedFiles` checks invalid and no-op calls; `truncatesThenRetainsAcrossRestartAndContinuesAtTheReopenedEnd` checks reopen, retention, and further append; `fixedSeedOperationsMatchAnIndependentSegmentAndOffsetModel` checks repeated state transitions.

Common pitfalls and differences from real Kafka

`truncateTo(5)` keeps offset 4, not offset 5; 5 is the next position. Changing the log file without trimming/rebuilding its index can make a reader jump to a deleted divergent record.

Tiered hints (not the answer)

Level 1

Translate `nextOffset` into “retain offsets below it” before deciding what stays.

Level 2

Draw the active-file state for a target inside a segment, at a segment base, at LEO, and at `logStart`.

Level 3

Check disk files and indexes, close and reopen, then append one record and verify its offset equals the target.

Commands and progression. Use `./gradlew :stepTest -Pstep=8` to diagnose this step; `./gradlew :test -Pstep=8` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. After `truncateTo(5)`, why is the next record written at 5 rather than 4?
2. Why must the sparse index be handled after truncating the record-file tail?

Self-check explanation The half-open range retains every `offset < 5`, so the last retained offset is 4 and the next is 5. Index entries are physical record-boundary hints; stale tail entries could send a read into deleted data rather than the current log.

Chapter 4

Partitions and a Broker on a Real Network

The first two chapters encoded records into recoverable single- and multi-segment logs. This chapter gives a topic (a named category of messages) multiple partitions (each partition is its own ordered log), then connects clients to a broker (a process that stores partition data and handles requests) over loopback TCP. Every partition has its own offset sequence: offset 0 in orders-0 and offset 0 in orders-1 identify different records, and they cannot be compared to establish a topic-wide order. Metadata tells clients which partitions exist, their leader broker, and connection endpoint. TCP delivers an ordered byte stream, not Java write boundaries, so an application protocol needs frame lengths. The course protocol is for teaching only and is not compatible with Kafka's wire protocol.

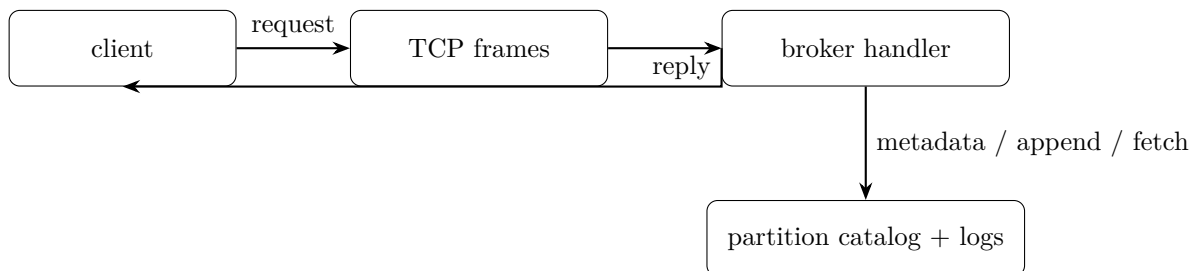


Figure 4.1: Request/response path over one connection. Provided code handles message-body encoding and decoding; students implement frames and business dispatch.

4.1 Step 9: Partition Routing

Goals and prerequisites

Goal: Route the same key to the same partition consistently while making null-key rotation observable.

Before you begin: Understand key bytes, an unsigned CRC32C value, and modulo by a positive partition count.

Background and reasoning

A producer chooses a partition before appending so that it knows which ordered log receives a record. The course rule is to compute CRC32C over a non-null key's bytes, interpret the 32-bit

result as an unsigned value, and take it modulo the partition count; the same byte string and count therefore choose the same partition. A null key has no bytes to hash, so one Partitioner instance rotates among available partitions. An empty byte array is a present key of length zero and is not the same as null.

Starting point and edit location.

Already provided: The no-argument `Partitioner()` constructor is provided, but the round-robin cursor state is not a provided answer; add whatever instance state the implementation needs.

Implement in: Implement `public int choose(byte[] key, int partitions)` in `exercises/src/main/java/io/simplekafka/client/Partitioner.java`.

Prerequisites

This step depends on [Step 8 \(tail truncation\)](#); the previous chapter established the continuous offsets held by a partition log. The chapter introduction defines topic, partition, and broker: this method chooses an existing partition for a write, but does not create a topic or log.

Inputs, outputs, and state

- key is null or any non-null byte array, including an empty array; partitions is the number of available partitions and must be >0 . Return an int partition id in $[0, \text{partitions})$.
- A non-null key uses `unsigned(CRC32C(key)) mod partitions` and does not read or write the null-key cursor; an empty array has `CRC32C=0`. A null key uses this instance's cursor: `normalize cursor mod partitions` to select a partition, then set the cursor to `(selected partition + 1) mod partitions`; a changed partition count is applied before selection.
- `partitions ≤ 0` reports `INVALID_REQUEST` and does not advance the cursor. Different Partitioner instances have independent cursors. This is in-memory state, with no disk or network side effect.

Behavioral contract and invariants. Validate that the partition count is positive first. Do not use a signed-int remainder when the CRC is meant to be an unsigned 32-bit value. Only a null-key call advances the rotation cursor; keyed and empty-array selections do not. When the partition count changes, each null-key result must remain in range.

Hand calculation: keyed routing and cursor

Assume key string “123456789” is encoded as ASCII bytes. Its CRC32C is 0xe3069283 (unsigned value 3,808,858,755); modulo 7 partitions gives partition 2. An empty array has `CRC32C=0`, so with 7 partitions it returns partition 0 without changing the null-key cursor. On a fresh instance, null-key calls with partition counts 3, 3, 1, 3, 3, 2, 2 return 0, 1, 0, 0, 1, 0, 1: the count-1 call normalizes the cursor to 0, and later calls use the new partition count.

Observable examples and test points. `Step09Test: routesByKnownUnsignedCrc32cVectorsAndKeepsEmptyKeyAsAKey` checks CRC32C and empty keys; `keyedAndEmptyKeysDoNotAdvanceTheNullKeyCursor` checks cursor preservation; `nullKeyCursorsAreIndependentAndSinglePartitionAlwaysSelectsZero` and `nullKeyCursorStaysWithinPartitionsWhenTopicSizesChange` check instance isolation and changing partition counts; `requiresAPositivePartitionCount` checks invalid counts.

Common pitfalls and differences from real Kafka

Do not use an object's `hashCode()` instead of CRC32C over key bytes, and do not take a remainder directly from a possibly negative signed int. Real Kafka has a different key hash and

a sticky-batching policy for null keys; the course’s per-instance rotation is not Kafka’s default behavior.

Tiered hints (not the answer)

Level 1

Draw three inputs—null, empty array, and nonempty array—and note which ones change the cursor.

Level 2

Interpret CRC as an unsigned number from 0 to $2^{32} - 1$, then use a positive partition count for modulo.

Level 3

Track the null cursor call by call; normalize when count changes and insert a keyed call to verify it does not advance the cursor.

Commands and progression. Use `./gradlew :stepTest -Pstep=9` to diagnose this step; `./gradlew :test -Pstep=9` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why should partition hashing use key bytes rather than a `RecordData` object’s Java hashCode?
2. How do null and a zero-length key differ in routing and state updates?

Self-check explanation Hashing uses stable key bytes; Java object hashCode is not the course rule and does not promise the same cross-restart meaning. A null key rotates and updates the instance cursor; an empty array has the actual CRC32C value 0, routes deterministically, and does not update the cursor.

4.2 Step 10: Topic Metadata

Goals and prerequisites

Goal: Register a fixed set of partitions in the local catalog and return deterministic metadata for client routing.

Before you begin: Distinguish disk directories from in-memory topic registration.

Background and reasoning

A topic is a logical name used by readers; its partition directories contain log files. Metadata is a structured description a broker returns to clients, not a guess based on arbitrary folders on disk. Once the course catalog creates a fixed partition count, a single broker describes each partition with its `TopicPartition`, current broker as leader, epoch, replicas, ISR, and endpoint. In single-broker mode the epoch is 0, and replicas and ISR contain only that broker.

Starting point and edit location.

Already provided: The `PartitionCatalog` constructor fixes root, broker id, and endpoint; `TopicPartition`, `PartitionMetadata`, log registration, and catalog-root containment are provided.

Implement in: `Implement public synchronized void createTopic(String topic, int partitionCount) and public synchronized List<PartitionMetadata> metadata(String topic)` in `exercises/src/main/java/io/simplekafka/broker/PartitionCatalog.java`.

Prerequisites

Complete [Step 9 \(partition routing\)](#); understand that a partition id is scoped within a topic. Also use the `PartitionLog` from [Step 6 \(segmented log\)](#) as the storage for each partition. Metadata registers/describes partitions; it does not choose a producer's key.

Inputs, outputs, and state

- topic must contain 1–255 ASCII bytes from `[A-Za-z0-9._-]`, and cannot be `.` or `...` partitionCount must be positive. Perform these checks before creating topic or partition paths. An invalid name/count reports `INVALID_REQUEST` and must not change the directory.
- Successful `createTopic` returns no value, registers partitions 0 through count-1, and opens their logs. Repeating the same name/count is idempotent and does not erase existing data. A different count for the same name reports `INVALID_REQUEST` and leaves existing metadata, files, and logs unchanged. Underlying create/open I/O errors report `STORAGE_ERROR`. Calling after the catalog is closed throws `IllegalStateException`.
- `metadata` returns a list ordered by partition id; in each single-node entry `leaderId=current brokerId`, `epoch=0`, `replicas=[brokerId]`, `isr=[brokerId]`, and `endpoint` is the configured address. An unregistered topic reports `UNKNOWN_TOPIC_OR_PARTITION`. A new catalog starts with an empty registry, so explicitly call `createTopic` to register and reopen the directories; an unexpected numeric directory is not automatically included. Calling after the catalog is closed throws `IllegalStateException`.

Behavioral contract and invariants. Validate the identifier and positive count before creation. Repeating the same definition does not change disk; a different count is rejected without changing the existing directory. Metadata order is stable and does not expose paths outside root. The model permits a topic name of up to 255 bytes; a file system that rejects a 255-byte path component may still fail directory creation, although `TopicPartition` accepts the identifier itself.

Hand calculation: create, restart, and register

Assume `brokerId=7`, a configured endpoint, and an empty root. After `createTopic(orders,2)`, metadata is ordered `orders-0` then `orders-1`, and both entries have `leader=7`, `epoch=0`, `replicas=isr=[7]`. Append records 0 and 1 to `orders-0`, close the catalog, and create a new one: `metadata(orders)` now reports `UNKNOWN_TOPIC_OR_PARTITION` because the topic registry is not persistent metadata. Calling `createTopic(orders,2)` again opens the same directory; `orders-0` still reads offsets 0 and 1, and a new append returns `[2,3)`. Even an extra on-disk `orders/9` directory does not add a third registered entry.

Observable examples and test points. `Step10Test: createsIndependentFixedTopicsAndReopensOnlyWithExplicitPartitionCounts` checks metadata, persisted logs, and explicit re-registration; `rejectsInvalidCountsAndTopicNamesWithoutMutatingTheFilesystemOrEscapingTheRoot` checks validation and unchanged directories; `acceptsA255ByteTopicIdentifierWhereTheFilesystemAllowsIt` checks the identifier boundary.

Common pitfalls and differences from real Kafka

Do not concatenate an unvalidated user topic into a path, or enumerate directories to infer a partition count and register manual leftovers. The course catalog does not model Kafka's topic

control plane or partition-expansion process.

Tiered hints (not the answer)

Level 1

Separate the “topic in the in-memory registry” from “a log directory exists on disk.”

Level 2

Write the resulting state for same-name/same-count, same-name/different-count, and unregistered-after-restart calls.

Level 3

Compare directory snapshots around invalid creates and restart; after explicit re-registration, check that offsets continue without a gap.

Commands and progression. Use `./gradlew :stepTest -Pstep=10` to diagnose this step; `./gradlew :test -Pstep=10` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can't the same topic name with a different partition count be treated as idempotent success?
2. After restart, what can an existing numeric directory prove, and what can it not prove?

Self-check explanation Partition count determines the fixed set of routable partitions; accepting another count silently would make topic metadata and logs disagree. A directory proves some files exist, but does not prove they are registered as the current topic set or reveal the count of partitions that are absent.

4.3 Step 11: TCP Frame Protocol

Goals and prerequisites

Goal: Split a TCP byte stream into bounded frames and exchange requests or replies one frame at a time.

Before you begin: Understand short reads, length prefixes, EOF position, and correlation ids.

Background and reasoning

TCP provides an ordered byte stream but does not preserve each application's write boundary: one write may arrive through multiple reads, while several writes may be combined in one read. A frame carries a length telling the reader how many bytes make up this protocol unit; the fixed header supplies version, API, request correlation id, and error code, while the payload is the business body. The correlationId is the value used to match a response to its request.

Starting point and edit location.

Already provided: The `Frame` value type, API ids, `ErrorCode` wire ids, `Messages` body types, and `MessageCodec` body codec are provided; this step does not rewrite business bodies.

Implement in: Implement `public static void write(OutputStream output, Frame frame) throws IOException` and `public static Frame read(InputStream input) throws IOException` in `exercises/src/main/java/io/simplekafka/protocol/FrameCodec.java`.

Prerequisites

Complete [Step 10 \(topic metadata\)](#) to know that APIs send requests to a broker, and understand the big-endian fixed-width integers from [Step 1 \(record encoding\)](#). A frame is the course's TCP wrapper, not a disk record or Kafka's official request header.

Inputs, outputs, and state

- Wire order is big-endian `length:int32|version:int16(1)|api:int16|correlationId:int32|error:int16|payload`. `length` excludes its own 4 bytes but includes the remaining 10-byte header and payload; legal values are [10,8,388,608], maximum payload is 8,388,598 bytes, and the maximum physical frame is 8,388,612 bytes.
- `write` writes one Frame. Null arguments, unknown APIs, or a total length outside the range report `INVALID_REQUEST` before output begins. Success emits exactly one frame. A stream I/O failure propagates the same `IOException`; once actual I/O begins, the external stream is not promised to roll back.
- `read` returns at most one frame per call. A null input reports `INVALID_REQUEST`. EOF before the first length byte returns null; EOF after only part of the length prefix or frame body reports `INVALID_REQUEST`. An out-of-range length, version other than 1, unknown API, or unknown error wire id also reports `INVALID_REQUEST`; validate length before reading/allocating its declared size. Input stream I/O failures propagate as the original `IOException`.

Behavioral contract and invariants. A read loop must fill the required bytes rather than assuming one read fills a header or body. Only EOF at a clean frame boundary returns null; a partial frame is not an empty result. Consecutive frames consume their own lengths without swallowing the next frame. For an invalid/oversized prefix, consume no more than its already-read 4-byte length; do not read its suffix or allocate memory based on a malicious size. A known error id in a frame only means its error field is valid; the provided server also rejects a request frame with nonzero error as `INVALID_REQUEST` before dispatch.

Decode the golden frame field by field

The `Step11Test` request/reply golden is 00000000d00010001000000290008010203. The first 4 bytes are 13, the length after the prefix: a 10-byte header plus a 3-byte payload. `version=0001=1`, `API=0001=METADATA`, `correlationId=00000029=41`, `error=0008=REQUEST_TIMEOUT`, and payload is 010203. Thus the physical frame is 17 bytes. With no payload, length remains 10 and the full physical frame is 14 bytes.

Observable examples and test points. `Step11Test: writesAndReadsTheIndependentGoldenFrameAndHandlesEmptyPayload` checks the golden bytes; `acceptsMaximumFrameAndRejectsOneByteOverBeforeWritingAnything` checks limits and zero-byte rejection; `rejectsEveryIncompletePrefixOfTheGoldenFrame` checks EOF; `readsBackToBackFramesFromStreamsDeliveringOneTwoOrThreeBytesAtATime` checks short reads and consecutive frames; `rejectsInvalidLengthsWithoutConsumingBytesAfterTheHeader` and `rejectsUnknownVersionApiAndErrorIds` check format errors; `propagatesTheSameUnderlyingIOExceptionFromReadAndWrite` checks the original I/O exception is propagated.

Common pitfalls and differences from real Kafka

One read is not guaranteed to fill a length or header; allocating from an unvalidated length can amplify memory use for invalid input. The course's `version=1`, API list, and error ids are teaching wire rules, not a connection to an official Kafka client.

Tiered hints (not the answer)**Level 1**

Sum the fixed header field widths to get the minimum length 10, and decide whether length includes its own prefix.

Level 2

Distinguish “no new frame has started” from “a frame has started but bytes are missing,” and loop to fill short reads.

Level 3

Feed the golden one byte at a time to confirm each call consumes only one frame; append a sentinel after an oversized length and verify it is not read.

Commands and progression. Use `./gradlew :stepTest -Pstep=11` to diagnose this step; `./gradlew :test -Pstep=11` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why should a response echo the correlationId for its request?
2. Why is it useful to distinguish clean EOF from EOF halfway through a frame?

Self-check explanation The correlation id lets a client pair each response with the correct call. Boundary EOF means the stream ended normally; a partial frame is truncated protocol data and cannot be disguised as “no more requests.”

4.4 Step 12: Single-Broker Requests

Goals and prerequisites

Goal: Connect metadata, produce, and fetch business requests to the local catalog and verify the full path over real loopback TCP.

Before you begin: Connect frame handling, the provided body codec, and log input/output boundaries.

Background and reasoning

A broker is the request-semantics boundary: it dispatches a Request to metadata, produce, or fetch, validates partition, parameters, and epoch, calls the matching backend, and returns a success body or an ErrorBody with an error code. produce appends records to one partition; fetch actively reads records by offset and budget and does not delete the log. A single broker has no follower, so this step’s high watermark (HW, the visible committed boundary) equals LEO. A real TCP test exercises client framing, the provided body codec, the server handler, and response framing together.

Starting point and edit location.

Already provided: The `BrokerHarness.single`, TCP server/client, `MessageCodec` request/reply bodies, request/reply types, log registration, and service lifecycle are provided; `SingleBrokerMain` uses loopback and an OS-assigned port.

Implement in: Implement `publicMessages.Replyhandle(Messages.Requestrequest)` in `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`. This step implements METADATA, PRODUCE, and FETCH dispatch only; do not rewrite transport or the body codec.

Prerequisites

Complete [Step 11 \(TCP frame protocol\)](#) and the topic/partition concepts used in [Step 9 \(partition routing\)](#) and [Step 10 \(topic metadata\)](#). This step fetches with single-broker HW=LEO; the replication chapter will teach HW again, so do not generalize the single-node rule.

Inputs, outputs, and state

- Metadata input is `MetadataRequest(topic)`; success returns `MetadataBody(List<PartitionMetadata>)`. A null or malformed topic returns `INVALID_REQUEST`; an unknown but valid topic returns `UNKNOWN_TOPIC_OR_PARTITION`. Metadata reads the catalog and does not change logs.
- Produce input is `ProduceRequest(tp, epoch, acks, timeoutMillis, records)`; single-broker mode accepts only `epoch=0`, `acks` must be `Acks.LEADER` (`wire=1`) or `Acks.ALL` (`wire=-1`), `timeoutMillis` must be nonnegative, and `records` must be nonempty. The single-broker backend ignores the acknowledgment mode, and a nonnegative timeout does not cause a wait; acknowledgment behavior with replicas is taught in [Step 23 \(produce acknowledgments\)](#). Check order: resolve the partition, validate produce parameters, obtain the backend, check epoch/leader, then call backend produce. Unknown partition returns `UNKNOWN_TOPIC_OR_PARTITION`; invalid parameters return `INVALID_REQUEST`; a mismatched epoch returns `FENCED_EPOCH`. Success appends and returns `ProduceBody(AppendResult)`. Precondition failures do not append; an I/O failure after writing starts does not guarantee there was no physical side effect.
- Fetch input is `FetchRequest(tp, epoch, offset, maxRecords, maxBytes, token)`; use `token=null` here, before the later consumer-group fencing step. Check order: resolve the partition, validate positive `maxRecords`/`maxBytes`, obtain the backend, check epoch/leader, then fetch from the backend. Single-broker mode accepts `epoch=0`; an offset outside the retained range `[logStartOffset, LEO]` returns `OFFSET_OUT_OF_RANGE`, and a request rejected by these checks does not change the log. Success returns `FetchBody(records, logStartOffset, logEndOffset, highWatermark, epoch)`; records fit both budgets and are never split. Here HW=LEO and `offset=LEO` is an empty successful read. Fetch does not change the log.
- The handler returns a `Reply`: success has `error=NONE` and its matching success body; failure carries a course error code and safe `ErrorBody`. A null or unsupported request reports `INVALID_REQUEST`. The handler maps a `CourseException` to a `Reply` preserving its code; parameter `IllegalArgumentException/NullPointerException` maps to `INVALID_REQUEST`; unexpected runtime failures map to `STORAGE_ERROR` without exposing their cause. `ExerciseNotImplementedException` passes through so the course runner can locate unfinished work. Provided transport handles invalid frames, nonzero request-frame errors, body format/trailing bytes, and reply-frame encoding; the handler must not reimplement these parsing tasks.

Behavioral contract and invariants. Unknown partitions, stale epochs, and out-of-range offsets retain their own error codes; a request rejected before backend access must not append. Single-broker metadata has `epoch=0`, and a successful fetch does not pass `LEO=HW`. An error affects only its request: do not send Java stack traces or private exception details over the wire, and keep handling later valid TCP requests. The provided `MessageCodec` caps fetch `maxBytes` at 8,388,566 disk bytes to fit a single response frame; an oversized complete record is not split, and remaining records are fetched in a later request starting at the next offset.

Hand calculation: produce, fetch, and page a large reply

Assume `orders-0` exists, `epoch=0`, `LEO=0`, and `produce` uses `acks=Acks.LEADER` and `timeout=Millis=0`. Append three records in order: key `k` (1 byte)/value `v0` (2 bytes), key `k`/value `v1`, and null key/value `v2`. Ordinary record totals are 35, 35, and 34 bytes. Success returns `[0,3)` and `LEO` becomes 3. A fetch at `offset=0` with `maxRecords=2` and `maxBytes=4096` returns offsets 0 and 1; `FetchBody` has `start=0`, `LEO=3`, `HW=3`, `epoch=0`. Fetch at `offset=3` returns empty. If `maxBytes` is 69, one byte below the total for two 35-byte records, only offset 0 is returned.

Now assume 9 records, each with a null key and a 1,048,548-byte value; each ordinary disk record is 1,048,580 bytes. A request for `maxBytes=10,000,000` is capped by the provided codec to 8,388,566: 7 records use 7,340,060 bytes and fit, so the first reply contains offsets 0–6; 8 records use 8,388,640 bytes and exceed the cap, so offset 7 is not included. A later request starting at offset 7 returns 7 and 8. Records are never split to fill a budget.

Observable examples and test points. `Step12Test: servesPreciseMetadataRecordsAndFailuresWithoutMutatingTheLog` checks metadata, `produce`/`fetch`, and rejected requests leaving the log unchanged; `boundsLargeFetchRepliesAndPreservesRecordContinuation` and `returnsAnErrorReplyWhenAHandlerProducesAnOversizedFetchBody` check large-response budgets; `sanitizesHandlerRuntimeExceptionsAndKeepsBothConnectionsUsable` checks error sanitization and continued service; `isolatesExistingTopicPartitionsAndServesFetchesFromNonzeroLogStart` checks partition/start isolation; `rawTcpRequestsAndMalformedBodiesStayIsolatedToTheirConnections` checks failure isolation over real TCP.

Common pitfalls and differences from real Kafka

Calling only the handler directly misses TCP boundaries, correlation ids, and reply-codec errors; swallowing an exception as an empty success makes a caller believe a write/read succeeded. One business error must not stop the broker. A successful `produce` and the client receiving its reply are separate network events: a reply may be unreachable after a network failure.

Tiered hints (not the answer)**Level 1**

List input type, prechecks, and success body for `METADATA`, `PRODUCE`, and `FETCH` as three separate paths.

Level 2

Keep the responsibility and error code of each frame/`MessageCodec`, handler-validation, and backend failure layer distinct.

Level 3

Use loopback TCP for metadata, `produce`, and `fetch`, then send a known failing request and `fetch` again; compare the log and replies.

Commands and progression. Use `./gradlew :stepTest -Pstep=12` to diagnose this step; `./gradlew :test -Pstep=12` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must an error Reply not send a Java stack trace or internal exception message to a client?
2. Why does single-broker `HW=LEO` not define the rule for replica mode?

Self-check explanation Internal exceptions may disclose implementation details and are not a stable protocol contract; return only a coarse error code and short safe message. A single broker has no replication progress to distinguish; with replicas, LEO is only a local tail and HW separately defines the visible range.

Chapter 5

Producing, Consuming, Offsets, and Redelivery

This chapter distinguishes four kinds of progress that are easy to confuse. A partition's log end offset (LEO) is the next offset available for append. During replication, the high watermark (HW) is a next-offset boundary; only records with offset $< \text{HW}$ are within the replication-visible range. In the current single-broker path, HW equals LEO, and replication later in the book can separate them. A consumer's position is the in-memory offset for its next fetch; its committed next offset is progress stored per group and partition so it can be restored after restart. `poll` advances only the in-memory position, while `commit` writes progress to the offset log; neither deletes messages. A producer receipt reports the partition and half-open offset range `[firstOffset, nextOffset)` for an append; it is not a consumer commit. A directly supplied `RpcClient` transfers to the client and closes with `close`; a shared `MetadataRouter` remains owned by its creator.

5.1 Step 13: Batched Producer

Goals and prerequisites

Goal: Implement per-partition ordered batch buffering, synchronous sends at the threshold, and return all receipts on explicit flush.

Before you begin: Understand what a producer, partition routing, batch, acknowledgement, and unknown request outcome mean.

Background and reasoning

A producer is a client that writes records to a broker; each record is routed to a topic (a named category) and partition (one ordered log). A batch is a group of records temporarily held by the client for one send. Each partition buffers independently, so reaching the threshold for one partition does not send another partition's records. A receipt is the actual offset range returned by the broker; ACK is the requested acknowledgement mode. This step's single-broker backend ignores ACK; it also does not wait for a nonnegative `ProduceRequest.timeoutMillis`. The producer waits synchronously for the broker reply, while the actual TCP wait limit is configured separately by `RpcClient`; this is not multi-replica confirmation. Replica ACK semantics arrive in [Step 23 \(produce acknowledgments\)](#). An error or timeout does not prove that no append occurred: the request may have reached the broker and been written while its reply was lost. The course's ordinary `SimpleProducer` does not retry or use a background timer.

Read first

Review [Step 3](#) (partitions and offsets within a partition), then [Step 9](#) (choosing a partition by key), [Step 10](#) (listing partitions through metadata), and [Step 12](#) (broker PRODUCE handling). In particular, `Partitioner.choose` is student work completed in Step 9, not a provided answer; this step currently uses the direct `RpcClient` path. Client routing through `MetadataRouter` comes in Step 27.

Starting point and edit location.

Already provided: `RpcClient`, message/record value types, and the broker metadata/produce protocol are provided; reuse the Step 9 `Partitioner` implementation.

Implement in: `exercises/src/main/java/io/simplekafka/client/SimpleProducer.java`
 a: `void send(String topic, byte[] key, byte[] value, long timestamp)` and `List<ProduceReceipt> flush()`. Implement only these exercise methods; do not change `Partitioner`, the protocol, or tests.

Inputs, outputs, and state

- Constructor argument `batchRecords` must be positive or reports `INVALID_REQUEST`; a null client/partitioner throws `NullPointerException`. Default ACK is `LEADER`, and the request field `timeoutMillis` is 5,000 ms. Existing `setAcks(Acks, long)` changes ACK/timeout fields on later requests; timeout must be nonnegative, and policy changes are allowed only with no pending batch. Null acks throws `NullPointerException`.
- In `send`, topic must be a valid course identifier; key may be null; value must be non-null; timestamp is a nonnegative long, and the course does not require it to represent a real-world clock. A successful call returns no receipt: it appends the record to that `TopicPartition`'s FIFO batch and synchronously sends that partition when the threshold is reached.
- `flush()` sends every nonempty partition batch, returns an unmodifiable list of receipts accumulated since the previous flush, and clears that accumulation. With no pending records it sends no empty request and returns an empty list.
- A metadata/parameter error before sending causes no broker append. An error after entering the produce RPC/reply path may follow an append; the producer saves the first error and enters an unknown-outcome state. Later `send`, `flush`, and `setAcks` calls report `REQUEST_TIMEOUT`; close the producer and create a new instance.

Behavioral contract and invariants. Preserve send order within a partition; different partitions have independent batches. Keep receipts from automatic sends until the next explicit flush. Receipts use half-open offset ranges. Null keys use the partitioner's per-instance rotation, and non-null keys use the Step 9 routing implementation.

Separately checkable boundaries

- A null value, negative timestamp, or invalid topic reports `INVALID_REQUEST` before entering produce; an unknown topic reports `UNKNOWN_TOPIC_OR_PARTITION`. These checks do not append or put the producer into an unknown-outcome state.
- An oversized ordinary record (more than 1,048,580 bytes total; with a null key, the value limit is 1,048,548 bytes) reports `INVALID_REQUEST`; the broker receives no produce frame and appends nothing. If the batch threshold has not been reached, `send` may return successfully first; the local encoder discovers the size only on automatic send or `flush()`. At that point the producer marks the produce-path exception as an unknown outcome, and later operations report `REQUEST_TIMEOUT`. This is the course client's conservative state behavior, not evidence that the broker may have appended; do not resend the batch.

- If any exception occurs on the first synchronous batch-send path, the call propagates its original error code and latches the producer's unknown-outcome state; later `send`, `flush`, and `setAcks` calls report `REQUEST_TIMEOUT`, and the batch must not be resent. Whether an append actually occurred depends on where the exception arose: a network/reply loss may follow an append, while a local encoding error proves no frame was sent but still latches this state in the course implementation.
- Calling `setAcks` with buffered records or a negative timeout reports `INVALID_REQUEST`. Calling a producer operation after close reports `IllegalStateException`.

Observable examples and test points. Work it by hand: `orders-0` starts at `LEO=0`, and the producer is configured with `batchRecords=2`. Two records with the same key buffer in order; the second triggers the send, so the broker appends offsets 0 and 1 and returns `[0,2)`. A third record remains buffered and is not yet readable. The next `flush()` sends it and returns two receipts, `[0,2)` and `[2,3)`; a second flush returns an empty list. Assume the topic has the target partition or the key is known to route there.

`Step13Test.flushesAtTheBatchThresholdAndReturnsAutomaticAndExplicitRanges` checks threshold behavior and one-time receipt delivery; `automaticFlushForOnePartitionLeavesAnotherPartitionsPendingBatchInvisible` checks independent partition batches; `rejectsInvalidSendsWithoutPersistingAnyRecords` and `doesNotRetryAProduceWhoseAppendSucceededBeforeItsResponseTimedOut` check pre-append rejection and no retry after append/reply loss.

Tiered hints (not the answer)

Level 1

First separate the times when a request is sent, the broker appends, the client receives a reply, and the caller receives a receipt; they need not coincide.

Level 2

Use `TopicPartition` as the batch boundary; work through one partition reaching its threshold while another remains below it.

Level 3

Use the test case where the broker appends and then delays/drops its reply. Check whether the client preserves the first error and avoids sending the old batch again.

Check your understanding

1. Why can a timeout not prove that the record was not written?
2. If an automatic send succeeds, why must the receipt still be returned by a later flush?

Self-check explanation. The reply can be lost after append, so the client cannot know the result; the receipt lets the caller identify the exact offset range, and dropping it makes an append impossible to track.

Common pitfalls and differences from real Kafka

Treating “received an error” as “definitely not written” and retrying may create duplicates. Real Kafka also has asynchronous sends and more acknowledgement choices; this course's `SimpleProducer` is synchronous, explicitly flushed, and does not retry.

Commands and progression. Use `./gradlew :stepTest -Pstep=13` to diagnose this step; `./gradlew :test -Pstep=13` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

5.2 Step 14: Pull-Based Consumer

Goals and prerequisites

Goal: Build a single-threaded, single-round pull consumer with explicit assignment and in-memory positions.

Before you begin: Understand “next offset,” replay with seek, empty fetches, and the shared cross-partition budget.

Background and reasoning

A consumer actively pulls records from a broker; reading does not delete them from the log. Its position is the offset for the next fetch in one partition, not persisted progress. Manual assign establishes the set of partitions this client will read; seek changes only the local read position. poll issues one bounded fetch round rather than waiting continuously.

Read first

Review [Step 3 \(partitions and their local offsets\)](#) and [Step 12 \(broker FETCH requests\)](#). Step 13 is the preceding course step, but this lesson does not call the producer or read OffsetStore. It starts with purely in-memory positions at offset 0; restoration and commit arrive in Step 15.

Starting point and edit location.

Already provided: The broker’s FETCH dispatch, RpcClient, and the LogRecord and TopicPartition value types are provided.

Implement in: `exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java`
 a: `void assign(Collection<TopicPartition> partitions)`, `void seek(TopicPartition tp, long offset)`, and `Map<TopicPartition, List<LogRecord>> poll(int maxRecords, int maxBytes)`. Implement these three methods; the `read-only position(TopicPartition)` accessor is already provided.

Inputs, outputs, and state

- `assign` replaces the old assignment: deduplicate and sort by topic/partition; preserve positions for retained partitions, remove revoked ones, and initialize new ones at 0. A null collection throws `NullPointerException`; a null element reports `INVALID_REQUEST`; rejection leaves the old assignment unchanged.
- `seek(tp, offset)` requires an assigned partition and a nonnegative next offset. A null `tp` throws `NullPointerException`; an unassigned partition reports `NOT_ASSIGNED`; a negative offset reports `OFFSET_OUT_OF_RANGE`; a rejected seek leaves positions unchanged. seek does not commit.
- Both `maxRecords` and `maxBytes` in `poll` must be positive; otherwise report `INVALID_REQUEST` without advancing positions. They are total budgets shared across partitions, scanned in topic/partition order. An empty assignment sends no fetch; an empty partition spends no budget and is omitted from the returned map. A successful returned map is unmodifiable. After close, consumer operations throw `IllegalStateException`; `position(tp)` on an unassigned partition reports `NOT_ASSIGNED`.
- At this stage an ordinary record costs `32+keyBytes+valueBytes`; a record cannot be split. Each successful partition reply must be contiguous from its requested position and fit the remaining record/byte budgets. Otherwise that partition reports `INVALID_REQUEST` and does not advance. If seek starts before a deleted log start or after the current LEO, the broker reports `OFFSET_OUT_OF_RANGE` and the failed partition does not advance. Other server errors

propagate unchanged; positions of earlier successful partitions may already have advanced while the failed partition remains at its old position.

Behavioral contract and invariants. A partition that returns records advances to the last returned offset plus one. A partition with no returned records, an empty fetch, or a failed fetch does not advance. poll does not commit or delete log data. The broker caps oversized positive maxBytes to a conservative single-frame budget; a later round continues at the first unreturned offset.

Observable examples and test points. Assume `orders-0` and `orders-1` each contain two ordinary records, each with null key and empty value, so each record costs 32 bytes; both positions start at 0. After assigning both partitions, `poll(4,64)` scans first `orders-0`, returns offsets 0 and 1, and produces a map containing only that partition; positions become (2,0) and the budget is exhausted. Seek both back to 0, then `poll(4,63)`: the first partition can return only offset 0, since the remaining 31 bytes cannot fit another record, so positions become (1,0). In a scripted test, if the broker replies `UNKNOWN_TOPIC_OR_PARTITION` for the second partition (simulating that request's failure), the first position may be 1 while the failed one stays 0. `Step14Test.pollsInPartitionOrderWithinRoundBudgetsAndPreservesManualPositions` checks ordering, budgets, seek, and assign; `pollContinuesPastEmptyPartitionsWithoutS` `pendingTheSharedBudget` checks empty partitions; `leavesFailedPartitionPositionUnc` `hangedForFetchErrorsAndMalformedSuccessReplies` checks failures, discontinuous offsets, and oversized replies.

Tiered hints (not the answer)

Level 1

Track assignment, each in-memory position, and persisted committed offset separately; this step changes only the first two.

Level 2

Subtract each whole record's encoded bytes and one record from the same poll's remaining budget; do not reset it per partition.

Level 3

Let an earlier-sorted partition succeed and a later one fail. Inspect both positions separately instead of assuming the whole round rolls back atomically.

Check your understanding

1. After poll reads offsets 0 and 1 and sets position to 2, why might a rebuilt consumer still read from 0?
2. Why must a later partition's position not advance when its fetch fails?

Self-check explanation. The position lives only in the old client's memory; without resume/-commit, a new assignment defaults to 0. A failed partition was not successfully read, so advancing it could skip data, while an earlier successful partition may retain its own progress.

Common pitfalls and differences from real Kafka

A successful read is not a durable commit; an empty read at the log end is no reason to increment position. Real Kafka has continuous polling, heartbeats, and a more complex assignment lifecycle; this step implements only single-threaded bounded rounds.

Commands and progression. Use `./gradlew :stepTest -Pstep=14` to diagnose this step; `./gradlew :test -Pstep=14` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

5.3 Step 15: Consumer Offset Persistence

Goals and prerequisites

Goal: Persist each group's next partition offset in a local durable log and restore it in a new consumer.

Before you begin: Distinguish consumer progress from `OffsetStore`'s physical log offsets and its group key.

Background and reasoning

A consumer position is an in-memory read cursor; `commit` stores the “next offset to read” as consumer progress. `OffsetStore` is itself an append log, but its physical offsets number the store records; the `nextOffset` in each payload is the consumer progress. The key is `(group, TopicPartition)`, so a different group or partition has separate progress. Reading or committing neither deletes business records nor changes HW.

Read first

You need [Step 1 \(log records and recovery\)](#), [Step 12 \(broker request dispatch\)](#), and [Step 14 \(position, assign, and poll\)](#). A manual consumer uses a null group token; membership fencing does not arrive until Step 20.

Starting point and edit location.

Already provided: Offset-entry encoding in `MessageCodec`, the backing `PartitionLog`, record recovery, and `COMMIT_OFFSET/FETCH_OFFSET` request types are provided.

Implement in: `exercises/src/main/java/io/simplekafka/group/OffsetStore.java`: `void commit(OffsetKey key, long nextOffset)`, `OptionalLong fetch(OffsetKey key)`; `exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java`: `void resume(Collection<TopicPartition> partitions)`, `void commitSync()`; also extend `handle(Messages.Request)` in `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java` with `COMMIT_OFFSET/FETCH_OFFSET` branches.

Inputs, outputs, and state

- `OffsetKey` identifies a group and one topic-partition. `commit`'s `nextOffset` must be nonnegative; moving it backward is allowed. A null commit key, negative value, or null fetch key reports `INVALID_REQUEST`; a closed store or storage failure reports `STORAGE_ERROR`. `fetch` returns the latest `OptionalLong`; no commit is empty, not a committed offset of 0.
- Each commit appends and forces an encoded offset entry with a null record key and timestamp 0. On reopen, the last record for each key wins. The store's physical log offset need not equal the payload's `nextOffset`. A complete corrupt record reports `CORRUPT_RECORD` without changing the file; a valid but incomplete tail may be truncated by the backing recovery code.
- `resume` deduplicates and sorts partitions, replaces the assignment, and sets each position to its committed value or 0 if missing. It does not clamp a committed value to the current LEO. A lookup failure propagates after the assignment has changed and is not rolled back: new partitions remain at 0 and retained partitions keep their old positions. `commitSync` sends one request per assigned partition, including position 0; an empty assignment sends no commit request. `poll` does not commit, and commits for several partitions are not one atomic transaction.
- Protocol success bodies: `CommitOffsetRequest(OffsetKey, nextOffset, null)` returns `EmptyBody`; `FetchOffsetRequest(OffsetKey)` returns an `OffsetBody` with an `OptionalLong`. A negative offset is also `INVALID_REQUEST` through the broker.

Behavioral contract and invariants. The most recently appended value for the same key is current, and it may move backward; changing either group or topic-partition isolates the value. `resume` restores durable progress but does not repair an out-of-range commit or write the consumer's current position to the store automatically.

Observable examples and test points. Start with no `workers/orders=0` value. Commit 3, close and reopen, and fetch returns 3; commit 1 and reopen again, and fetch returns 1, demonstrating latest-wins and rewind. The same partition under another group is still empty. A new consumer resumes at position 1. If the stored value is 99 while the partition LEO is 3, resume still sets 99; poll later reports `OFFSET_OUT_OF_RANGE` and does not change the stored value.

`Step15Test.offsetStorePersistsNextOffsetsPerGroupAndPartitionAndAllowsRewind` checks isolation, rewind, and reopen; `rawOffsetRequestsReturnExactBodiesAndIsolateGroupTopicAndPartitionKeys` checks protocol bodies; `resumeCommitsAllPositionsAndDoesNotClampPastLogEnd` checks resume; `offsetStoreTruncatesAnIncompleteTailAndRejectsCompleteBadCrcWithoutMutation` checks recovery and complete corruption.

Tiered hints (not the answer)

Level 1

First treat consumer progress as the next business-record offset, not as the physical offset in `OffsetStore`'s log.

Level 2

One `OffsetKey` can have several historical commits; use the final append, not an assumed map order.

Level 3

Test two separate facts: whether the value remains after reopening the store, and whether a fresh `consumer.resume` starts at that value.

Check your understanding

1. After processing offset 2, why commit `nextOffset=3` instead of 2?
2. If `nextOffset` is 99 and LEO is 3, should resume silently change it to 3?

Self-check explanation. `nextOffset` marks the next record to process, so 3 skips already processed offsets 0–2. The store restores the caller's committed value faithfully; a later fetch reports the out-of-range condition instead of silently rewriting progress.

Common pitfalls and differences from real Kafka

Committing the last processed offset makes it replay after recovery; confusing committed offset with a replication visibility boundary is also wrong. Real Kafka stores offsets in a replicated/-compacted internal topic; this course uses a local single-coordinator append log and provides no high availability.

Commands and progression. Use `./gradlew :stepTest -Pstep=15` to diagnose this step; `./gradlew :test -Pstep=15` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

5.4 Step 16: At-Least-Once Processing

Goals and prerequisites

Goal: Order polling, business side effects, and offset commits safely, and work out replay after failure.

Before you begin: Understand at-least-once delivery, in-memory positions versus durable commits, and non-atomic requests.

Background and reasoning

A business side effect changes state while processing a record, for example writing a local ledger or sending a notification. One call to `ProcessingLoop` polls first, invokes callbacks in ascending `TopicPartition` order and ascending offset within each partition, and calls `commitSync` only after every callback succeeds. Side effects and the offset log are not one transaction: if processing succeeds but commit fails, recovery must read the record again and may repeat the side effect. At-least-once means preferring a possible repeat of successful work over silently skipping work that did not succeed.

Read first

You need [Step 14 \(poll and position\)](#) and [Step 15 \(durable next offsets and commitSync\)](#). The callback interface is provided; no transaction joining external effects to the offset store is required.

Starting point and edit location.

Already provided: The `RecordProcessor.process(TopicPartition, LogRecord)` callback interface and the one-round poll/commit lifecycle are provided.

Implement in: `exercises/src/main/java/io/simplekafka/client/ProcessingLoop.java`
 a: `intrunOnce(intmaxRecords, intmaxBytes) throws Exception`. Implement the one-round controller only; do not change the consumer, store, or callback interface.

Inputs, outputs, and state

- `maxRecords` and `maxBytes` are positive total budgets across the poll's partitions; a nonpositive limit is rejected by poll with `INVALID_REQUEST`. On success return the count of records processed and committed. An empty poll still calls `commitSync` and returns 0.
- The callback receives a partition and one `LogRecord`, returns no value, and may throw any `Exception`. Its original exception propagates, and the round commits no partitions; poll may already have advanced the consumer's in-memory positions.
- An exception from poll, callback, or commit marks this `ProcessingLoop` failed; a later `runOnce` throws `IllegalStateException`. Close the old consumer, create a new consumer and loop, and resume from durable committed positions. If a side effect happened before its offset was committed, that effect may repeat.
- After all callbacks succeed, `commitSync` commits each assigned partition separately; this is not a cross-partition transaction, so a later commit failure may follow successful earlier commits. The failure propagates and the method does not return a successful count.

Behavioral contract and invariants. Invoke callbacks in stable partition order and offset order; begin committing only after every record returned by this poll has been processed successfully. If any callback fails, do not commit any partition. After any failed round, do not reuse the loop or treat its advanced in-memory position as durable progress.

Observable examples and test points. Initially `orders-0` and `orders-1` each contain offsets 0 and 1, and group `workers` has no commits. `poll(4,8192)` returns all four. Callbacks first cause effects `p0:0,p0:1`, then throw at `p1:0`; neither partition has a committed offset, although the old consumer’s in-memory positions reached 2. Rebuild and resume: all four records are read from 0/0, so the two earlier `p0` effects happen again. In a different failure, all callbacks finish but commit requests fail: effects already happened, while commits may be unconfirmed; recovery may replay, and earlier partition keys may already have committed. `Step16Test.laterPartitionFailureLeavesBothOffsetsUncommittedAndReplaysEarlierSideEffects` covers a later-partition callback failure; `commitFailureAfterSideEffectsReplaysTheSameRecordsAtLeastOnce` covers commit failure; `boundedRoundsProcessInTopicPartitionOffsetOrderAndCommitFinalPositions` covers ordering, budgets, and empty rounds; `failedPollProcessesNothingAndResumeRetriesThePreviouslyFetchedRecord` covers poll failure.

Tiered hints (not the answer)

Level 1

Write `poll`, each side effect, each offset request, and crash/rebuild in temporal order.

Level 2

List callbacks in the required order, then decide whether failure happened in a callback or during commit; they have different side-effect histories.

Level 3

For each failure in `Step16Test`, record the store value, consumer position, and whether the loop can be reused separately; do not collapse them into one “progress” value.

Check your understanding

1. If `p0` processing succeeds but `p1` processing fails, why can `p0`’s successful side effects also replay?
2. Is `commitSync` all-or-nothing across three partitions?

Self-check explanation. The contract waits for all callbacks before beginning commits; if a callback fails, no partition is committed, so recovery reads from the old durable offsets. `commitSync` sends separate requests, not a cross-partition transaction, so an earlier partition may have committed already.

Common pitfalls and differences from real Kafka

Swallowing a callback exception and then committing can lose a failed record; continuing to use a failed loop can skip uncommitted data. At-least-once is not exactly-once. An application must make its side effects deduplicable; `Step 31` implements local durable business deduplication.

Commands and progression. Use `./gradlew :stepTest -Pstep=16` to diagnose this step; `./gradlew :test -Pstep=16` runs cumulatively from `Step 1` through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Chapter 6

Consumer Groups, Rebalancing, and Commit Fencing

A consumer group is a set of consumers cooperating to read one topic: a member is a participant identifier, an assignment is the current list of partitions owned by each member, and one partition is assigned to at most one member in the group at a time. The coordinator stores group state and answers join/assignment requests; membership changes make it recompute the whole assignment, a change called a rebalance. A generation is the group version incremented on each topology change, not a clock or consumer offset. Later, a GroupToken carries group, member, and generation together so the server can reject stale members. Heartbeats indicate that a member is still active; expired members are removed. The course models one topic, immediate rebalancing, and a trusted in-process coordinator, not the full Kafka group protocol or highly available consensus.

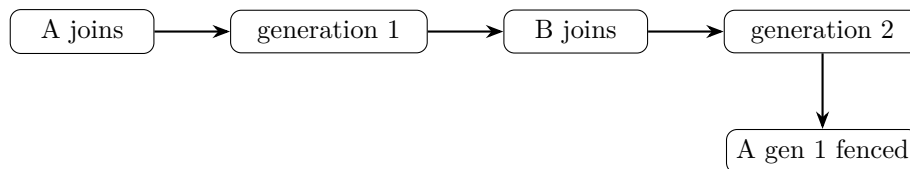


Figure 6.1: A membership topology change increments the generation; an old token cannot overwrite the current commit.

6.1 Step 17: Partition Assignment

Goals and prerequisites

Goal: Implement deterministic and balanced Round Robin assignment with exactly one owner per partition.

Before you begin: Understand deduplication, stable sorting, immutable results, and quotient/remainder distribution.

Background and reasoning

Partitions can be processed in parallel by consumers, but two members processing one partition at the same time can conflict over progress. Round Robin cycles through sorted members to assign sorted partitions; its result is repeatable and partition counts differ by at most one. This

assignor is a pure calculation and does not determine whether a member has actually joined a group.

Read first

Review [Step 3 \(TopicPartition and partition ordering\)](#). The assignor is called by the [Step 18 \(membership coordinator\)](#); it allocates only the partition/member lists it receives and does not query a catalog or modify a log.

Starting point and edit location.

Already provided: The `TopicPartition` value type and group result type are provided; the `RoundRobinAssignor` constructor is stateless and does not provide the assignment algorithm.

Implement in: `exercises/src/main/java/io/simplekafka/group/RoundRobinAssignor.java`: `Map<String,List<TopicPartition>>assign(List<TopicPartition>partitions,List<String>members)`. Implement only `assign`.

Inputs, outputs, and state

- `partitions` is the list of `TopicPartition` values to assign; `members` is the list of member identifiers. Neither list nor any element may be null. Keep only one copy of duplicate partitions or members.
- Sort partitions by topic then partition number, and members by Java `String` natural order; do not preserve input order or sort numeric-looking strings by numeric value.
- The returned map contains every distinct member, including members with empty lists; both the map and its lists are unmodifiable. The assignor does not change inputs and has no state between calls.
- Null inputs/elements, or a member identifier outside 1–255 UTF-8 bytes, report `INVALID_REQUEST`. With no members, return an empty map; with members but no partitions, retain every member with an empty list.

Behavioral contract and invariants. When there is at least one distinct member, every distinct partition appears exactly once and is owned by a member; every distinct member appears in the result. The maximum and minimum partition counts among members differ by no more than one. With no members, return the empty map as stated above; no partition has an owner.

Observable examples and test points. Start with the explicit partition list [`orders-0`, `orders-1`, `orders-2`, `orders-3`, `orders-4`] and members in input order [`b`, `a`, `a`]. After deduplication and sorting, members are [`a`, `b`]; `a` gets 0, 2, 4 and `b` gets 1, 3. Shuffling input does not change the result. With five partitions and members sorted by Java string order as [`a`, `b`, `c`, `d`, `e`, `f`, `g`], the first five get one each and `f/g` remain with empty lists.

`Step17Test.sortsAndDeduplicatesPartitionsAndMembersBeforeRoundRobinAssignment` checks deduplication and deterministic order; `assignsFivePartitionsToTheFirstFiveOfSevenSortedMembersAndKeepsEmptyMembers` checks empty assignments; `sortsCrossTopicPartitionsAndMemberIdsLexicallyAfterDeduplication` checks cross-topic and lexical order, where "10" precedes "2".

Tiered hints (not the answer)

Level 1

Break “one owner” into two independently checkable properties: complete coverage and no duplicates.

Level 2

Sort members and partitions first, then assign by index modulo member count; the input List order is not the course rule.

Level 3

Work out five partitions with two members and five partitions with seven members, then add duplicate inputs and verify no partition appears twice.

Check your understanding

1. Why is stable sorting observable contract rather than output polish?
2. When there are more members than partitions, why return the members with empty lists?

Self-check explanation. Stable sorting gives the same owner for the same sets on every call, which makes coordination and verification reproducible; an empty list says a member remains in the group but currently owns no partition.

Common pitfalls and differences from real Kafka

Returning only members that own partitions loses explicit empty assignments; depending on incidental map iteration breaks reproducibility. Real Kafka has several assignors and cooperative rebalancing; this course implements only a simple single-topic Round Robin and does not claim it is Kafka's default algorithm.

Commands and progression. Use `./gradlew :stepTest -Pstep=17` to diagnose this step; `./gradlew :test -Pstep=17` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

6.2 Step 18: Membership and Rebalancing

Goals and prerequisites

Goal: Manage group members, generation, and the full assignment, rebalancing correctly when membership changes.

Before you begin: Understand what join, leave, generation, and committed offset each represent.

Background and reasoning

A member is a consumer instance identified by a member id; in this course a group subscribes to one topic. The coordinator stores the member set and its shared generation, and uses the Step 17 assignor to calculate the complete assignment. A new member joining or an existing member leaving changes the topology, increments the generation, and immediately recomputes partitions for every member. Repeating join for the same member and topic refreshes only its heartbeat; it does not change topology. A generation is neither a timestamp nor consumer progress.

Read first

Review [Step 10 \(topic metadata and the partition catalog\)](#), then [Step 17 \(deterministic assignment\)](#). You need to know which `TopicPartition` values belong to a topic and distinguish the coordinator's in-memory membership from the durable offsets in Step 15.

Starting point and edit location.

Already provided: `PartitionCatalog`, group lock, member-state container, `TimeSource`, and the Step 17 assignor are provided; message request value types are already encoded.

Implement in: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `GroupAssignmentJoin(Stringgroup, Stringmember, Stringtopic)`, `GroupAssignmentLeave(Stringgroup, Stringmember)`, and `GroupAssignmentAssignment(Stringgroup, Stringmember)`; extend `handle(Messages.Request)` in `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java` with `JOIN_GROUP/LEAVE_GROUP/GROUP_ASSIGNMENT`.

Inputs, outputs, and state

- group/member identifiers must be nonempty and at most 255 UTF-8 bytes; topic must be a valid course topic. A first join creates the group only after verifying that the topic exists, then advances its initial generation from 0 to 1.
- A new member joining or a known member successfully leaving increments generation exactly once and returns a `GroupAssignment` for all current members. Repeating join for the same member and topic updates only `lastHeartbeat` and returns the existing assignment; it must not duplicate the member or increment generation.
- A topic conflict in an existing group takes priority and reports `INVALID_REQUEST`; an unknown topic reports `UNKNOWN_TOPIC_OR_PARTITION` and does not create a group or consume a generation. Invalid identifiers report `INVALID_REQUEST`. leave or assignment for an unknown group/member reports `UNKNOWN_MEMBER`; operations after coordinator close throw `IllegalStateException`.
- When the group becomes empty, retain its generation history in the coordinator and retain committed offsets; offset storage is separate from member lifetime. Membership itself is process-local and does not become a durable group protocol just because offsets persist.

Behavioral contract and invariants. Each real topology change increments one generation and recalculates assignments for every remaining member; a repeated join does not change the generation. Assignment queries succeed only for a currently known member and return the same full-group mapping.

Observable examples and test points. Initially `workers` does not exist, and `orders` has partitions 0, 1, and 2. a joins and receives generation 1 with all three partitions; a joins the same topic again and generation remains 1. b joins, generation 2, with a assigned 0 and 2 and b assigned 1. c joins at generation 3 and each member receives one partition. c leaves at generation 4; the new assignment is a: 0,2 and b: 1. If the group later becomes empty, its generation remains and its existing group offsets can still be fetched.

`Step18Test.membershipChangesRebalanceEverySurvivorAndRetainEmptyGroupGenerationOverTcp` uses real TCP to check join, repeated join, full-group rebalance, topic/unknown-member errors, empty-group offset retention, and a later rejoin.

Tiered hints (not the answer)

Level 1

First decide whether a request actually changes the member set; a repeated join, a rejected join, and a new member join are different state transitions.

Level 2

After each topology change, check that every member sees the same generation and full assignment, not only that the joiner was updated.

Level 3

Starting from no members, list the generation and every owner after a joins, a repeats, b joins, c joins, and c leaves.

Check your understanding

1. Why must a receive an assignment for a new generation when b joins?
2. Why should group offsets not be deleted when the last member leaves?

Self-check explanation. A partition may have moved from a to b, so a's old assignment no longer agrees with server ownership; consumer progress belongs to a group/topic-partition, not to an online member, so an empty group can retain it.

Common pitfalls and differences from real Kafka

Advancing a separate version per member makes tokens disagree; an empty group does not mean its consumer progress is invalid. Real Kafka uses multi-phase join, assignment synchronization, cooperative revocation, and other protocol steps; the course rebalances immediately and does not simulate the full classic barrier or KIP-848.

Commands and progression. Use `./gradlew :stepTest -Pstep=18` to diagnose this step; `./gradlew :test -Pstep=18` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

6.3 Step 19: Heartbeats and Expiration

Goals and prerequisites

Goal: Detect idle members with a controllable clock and process all same-group expirations as one topology change.

Before you begin: Understand `lastHeartbeat`, session timeout, and the inclusive equality boundary.

Background and reasoning

A member process may crash without leaving, so the coordinator cannot immediately remove it just because it is silent. A successful heartbeat records the current millisecond value from the injected clock and indicates that the member is active. A member expires once elapsed time reaches the session timeout. A deterministic clock makes boundary tests independent of sleep; the provided periodic worker is the runtime trigger, while a test's direct expiry hook is a separate mechanism.

Read first

You need [Step 18 \(members, generation, and assignment\)](#). Be able to explain that members in a group share a generation and that a removed member's old token is no longer a valid heartbeat.

Starting point and edit location.

Already provided: The `TimeSource`, group lock, positive session timeout, and periodic scheduling lifecycle are provided.

Implement in: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `void heartbeat(GroupToken token), Set<String> expire()`; extend `handle(Messages.Request)` in `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java` with `HEARTBEAT`.

Inputs, outputs, and state

- The coordinator's `sessionTimeoutMillis` must be positive at construction or reports `INVALID_REQUEST`. A null heartbeat token reports `INVALID_REQUEST`; check member existence first, reporting `UNKNOWN_MEMBER` when absent; then check generation, reporting `ILLEGAL_GENERATION` for a stale or future value. Failures do not refresh time; only a valid token updates that member's `lastHeartbeat` to the current clock. Operations after coordinator close throw `IllegalStateException`.
- `expire()` takes no argument and uses the injected current clock. It returns the sorted set of removed-member identifiers formatted as `group/member`. Expiration uses `now-lastHeartbeat >= sessionTimeoutMillis`; a member below the threshold remains, and a future heartbeat timestamp does not expire early.
- One expire call can remove several expired members from one group, but increments that group's generation once and recalculates assignment once; non-expired members remain. An empty result means no member was removed; repeating expiry makes no further change. Committed offsets are not deleted.
- The broker's periodic worker triggers checks automatically. For deterministic tests, move the `TimeSource` and call the provided `expireGroups()` hook or `expire()`, rather than guessing timing with real sleep.

Behavioral contract and invariants. The expiry boundary is $\geq$; an invalid/stale heartbeat does not renew membership. Results sort by group and then member. Each group changed by one expiry pass increments generation once, and an empty group retains its generation.

Observable examples and test points. Set `clock=1000` and `timeout=100`; a and b join `workers` at 1000, making `generation=2`, so a's generation-1 heartbeat is already stale. At 1099, a's old token and future generation 99 report `ILLEGAL_GENERATION`, and an unknown member reports `UNKNOWN_MEMBER`; none refreshes time. b's valid generation-2 heartbeat updates its time to 1099. `expire` at 1099 returns empty; at 1100 it removes only `workers/a`, exactly 100 ms after a's last heartbeat. Generation becomes 3 and b receives all three partitions.

`Step19Test.rejectsStaleAndFutureHeartbeatsAndExpiresOnlyTheUnrefreshedMemberAtTheBoundary` checks 1099/1100 and failed heartbeats; `expirationOrdersAllIdentifiersAndAdvancesEachChangedGroupOnce` checks sorting and one increment per group; `idempotentJoinRefreshesHeartbeatButRejectedJoinDoesNot` checks renewal; `replicatedClusterExpiresIdleMembersWithoutIncomingRequests` and `expireGroupsProvidesDeterministicClusterExpiryAndChecksLifecycle` check the periodic worker and deterministic hook.

Tiered hints (not the answer)

Level 1

Use the current clock, check elapsed time, and remove members within one consistent state observation; a failed heartbeat must not write `lastHeartbeat`.

Level 2

The return value should identify the exact group/member, not only the number of expirations.

Level 3

With `timeout=100` and `lastHeartbeat=1000` compare 1099 and 1100; refresh B at 1099 and confirm that this pass removes only A.

Check your understanding

1. Why must the equality boundary be explicit in the timeout contract?
2. If A and C in one group expire in the same call, why not increment generation twice?

Self-check explanation. Without an equality rule, ownership at the exact deadline would be ambiguous; one expiry pass observes one topology change for that group, so removing its expired members together triggers one full rebalance.

Common pitfalls and differences from real Kafka

Using system time and sleep for an exact threshold is scheduler-sensitive; use an injected clock for boundaries and test the worker separately as integration behavior. Updating lastHeartbeat before validating a token lets an expired member renew illegally. Real Kafka heartbeat behavior also depends on configuration, network, and coordination protocol.

Commands and progression. Use `./gradlew :stepTest -Pstep=19` to diagnose this step; `./gradlew :test -Pstep=19` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

6.4 Step 20: Consumer Group Fencing

Goals and prerequisites

Goal: Combine assignments with fetch and offset commit, using generation tokens to reject stale operations from zombie members.

Before you begin: Understand the token triple, revoked partitions, assignment refresh, and the boundary around external side effects.

Background and reasoning

An old consumer thread may still run after a rebalance. A `GroupToken=(group,member,generation)` is the group-membership version carried by a request; before allowing tokenized FETCH or COMMIT, the broker must confirm that the member exists, the generation is current, and the member owns the target partition. Fencing can reject stale requests, but it cannot recall records already delivered to the old client or undo business side effects already performed. Manual mode has no token and continues to use the earlier ordinary consumer path.

Read first

You need [Step 14 \(fetch positions\)](#), [Step 15 \(durable offset keys\)](#), [Step 17 \(assignment algorithm\)](#), [Step 18 \(generation and assignment\)](#), and [Step 19 \(heartbeat and expiry\)](#). This step combines them; do not treat a member generation as a consumer offset.

Starting point and edit location.

Already provided: Group request/response types, `RpcClient`, the `SimpleConsumer` pull/offset path, and member-lifecycle skeleton are provided.

Implement in: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `void validate(GroupToken token, TopicPartition tp)`; `exercises/src/main/java/io/simplekafka/client/GroupConsumer.java`: `GroupConsumer(RpcClient client, String group, String member)` (takes ownership and closes the client), `GroupConsumer(MetadataRouter router, String group, String member)` (borrows the router and does not close it); both constructors require group/member identifiers of 1–255 UTF-8 bytes. Implement only `subscribe(String topic)`, `Map<TopicPartition, List<LogRecord>> poll(int maxRecords, int maxBytes)`, and `commitSync()`; extend `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java` with token fencing for FETCH/COMMIT.

Inputs, outputs, and state

- `GroupCoordinator.validate(token, tp)` requires a current member, current generation, and ownership of the target partition. Unknown member reports `UNKNOWN_MEMBER`; a mismatched generation, including a future token, reports `ILLEGAL_GENERATION`; a partition not assigned to the member reports `NOT_ASSIGNED`. A null `tp` reports `INVALID_REQUEST`; a rejected validation does not write `OffsetStore`.
- `subscribe(topic)` explicitly joins and obtains an assignment/token. A newly assigned partition starts at its committed offset or 0; a retained partition keeps its position; a revoked partition is dropped. Poll/commit before subscribing reports `INVALID_REQUEST`; unknown-topic and other errors propagate. Operations after consumer close throw `IllegalStateException`.
- `poll(maxRecords, maxBytes)` requires positive total budgets or reports `INVALID_REQUEST`, and heartbeats before fetch requests; its returned map is unmodifiable. If the heartbeat reports `ILLEGAL_GENERATION`, it only queries the current assignment, updates the local token/positions, then fetches with the new token: revoked positions are removed and new partitions start at committed/0. On `UNKNOWN_MEMBER`, it does not rejoin automatically; the caller must explicitly subscribe. Other errors propagate. A successful fetch advances the relevant in-memory position but does not itself commit.
- `commitSync()` sends the current token with each assigned partition's position. It neither heartbeats nor refreshes the assignment. A stale token/non-owner rejection leaves the store unchanged; the per-partition requests are not atomic, so an earlier commit may succeed before a later request fails. The broker COMMIT branch requires a nonnegative `nextOffset`, and the `OffsetKey`'s group and partition must match the token and request; otherwise it reports `INVALID_REQUEST` without append. The server validates and appends under the coordinator lock, with lock order coordinator then store; this is not a partition-log transaction. A null token remains valid on the ordinary manual requests.
- Successful token FETCH returns `FetchBody`, token COMMIT returns `EmptyBody`, and assignment lookup returns `GroupBody`. Previously sent records and client side effects are not rolled back by a later rejection. Operations after coordinator close throw `IllegalStateException`.

Behavioral contract and invariants. Both FETCH and COMMIT fence member, generation, and partition owner. The COMMIT `OffsetKey` must match the token's group and the requested partition; any rejection must not append that offset. After a generation changes, an old member cannot commit a revoked partition's cached position.

Observable examples and test points. Initially `orders-0` contains only offset 0; `orders-1` contains offsets 0-8, and `workers/orders-1` is committed at 7. a subscribes at generation 1 and owns both partitions; `poll(1, 4096)` reads `p0/offset0`, leaving positions `p0=1` and `p1=7`. b subscribes, causing generation 2; a now owns only `p0`, and b owns `p1`. a's old-token `commitSync` reports `ILLEGAL_GENERATION`; `p0` is still uncommitted and `p1` is still 7. Append `p0/offset1`. a polls, detects the old generation, obtains the new assignment, retains `p0=1`, drops revoked `p1`, and reads `p0/offset1`. Its commit stores `p0=2` while `p1` remains 7. b's first poll starts at committed `p1=7`. If the broker fixture restarts, in-memory membership disappears but the log and offsets remain (`p0=2`, `p1=9`). Create a new `GroupConsumer` a and explicitly subscribe; the new generation starts at 1, and its positions resume from committed offsets 2 and 9.

`Step20Test.coordinatorFencesCommitsByGenerationOwnershipAndMatchingOffsetKey` checks the coordinator and mismatched keys; `rawTcpFetchAndCommitTokensFenceInvalidOwnersWithoutChangingOffsets` checks stale/future/unknown/non-owner tokens over real TCP; `groupConsumerPreservesRetainedPositionDropsRevokedPartitionsAndRecoversAfterExpiryAndRestart` checks refresh, takeover, and restart recovery; `groupConsumerClosesItsOwnedRpcClientAfterPollingARecord` checks resource ownership.

Tiered hints (not the answer)**Level 1**

Request validation must check member, generation, and partition owner together; comparing only the group name is not fencing.

Level 2

After rebalancing, classify each partition as retained with a local position, newly acquired and restored from committed/0, or revoked and discarded.

Level 3

Use the example to have a submit a larger offset with its old generation; check the error code and verify every related `OffsetKey` remains unchanged.

Check your understanding

1. What can generation fencing reject, and why can it not guarantee exactly-once external side effects?
2. If validation unlocks before commit, what might happen between the two operations?

Self-check explanation. The server can reject a stale token's fetch/commit, but cannot recall a delivered record or undo an action in an external system. If a rebalance occurs between validation and append, a zombie could write progress after losing ownership.

Common pitfalls and differences from real Kafka

Validating, releasing the lock, and committing later leaves a rebalance race. Turning both `ILLEGAL_GENERATION` and `UNKNOWN_MEMBER` into automatic rejoin hides different states. Real Kafka group protocols and transactional commits are more complex; this course's in-process coordinator and local offset store are not a highly available `__consumer_offsets`.

Commands and progression. Use `./gradlew :stepTest -Pstep=20` to diagnose this step; `./gradlew :test -Pstep=20` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Chapter 7

Replication, ISR, and High Watermarks

This chapter moves from a single-machine log to replication. **First, read the boundaries of the course’s distributed model carefully:** three independent brokers each have a TCP server, threads, and a disk directory; a follower must fetch records over TCP, and cannot replicate by sharing memory. However, they run in the same JVM and are managed by one trusted ClusterAuthority. The authority linearizes metadata, epoch, and HW state, making teaching failovers controllable. It is not KRaft, has no controller consensus or authority HA, and makes no guarantee about cluster recovery after loss of the entire JVM/authority.

Replication is meaningful only when append, reporting, and visibility boundaries are consistent with one another. LEO is the next offset in each individual log. ISR is the set of replicas currently allowed to participate in acknowledgments. HW is the committed boundary readable by ordinary consumers; consumers can read only offsets $< \text{HW}$. Replicas fetch over a real network; the authority provides consistent state and role decisions, but does not move data.

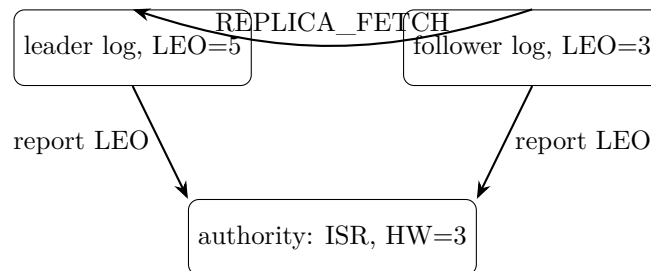


Figure 7.1: Replica logs are fetched over TCP, while progress is serialized by a trusted authority. In this example, $\text{HW} = \min(5, 3) = 3$.

7.1 Step 21: Fetch-Based Replication

Goals and prerequisites

Goal: Make a second log a verifiable disk replica: a follower fetches leader records over the network and appends them to its own log.

Before you begin: Understand why replicas have separate logs, why replication reads may pass the ordinary-consumer visibility boundary, and why one poll does not prove synchronization is complete.

Background and reasoning

Prerequisite route: the course sequence requires [Step 20 \(Consumer-Group Fencing\)](#); implementation also uses the `PartitionLog` from [Step 6 \(Segmented Logs\)](#) and request/reply handling from [Step 12 \(Broker Requests\)](#). A leader is the broker currently accepting writes for a partition; a follower stores another copy of that partition's log and actively fetches records from the leader. Each broker has its own durable log; replication is not shared objects or copied memory. LEO is the next offset in a local log; HW is the exclusive boundary ordinary consumers may read. A follower needs the leader's uncommitted tail to catch up, so `REPLICA_FETCH` can read through leader LEO, whereas ordinary consumer `FETCH` reads only below HW. One poll completes only "fetch and append locally" ; Step 22 reports the successfully persisted LEO to the authority.

Starting point and edit location.

Already provided: The three-broker TCP harness, `Messages.ReplicaFetchRequest` / `ReplicaFetchBody`, `ReplicaState`, `PartitionLog` lifecycle, and framed RPC are provided. The initial authority reported LEO is 0; do not mistake a direct write to the leader disk for a reported replica position.

Implement in: `exercises/src/main/java/io/simplekafka/replication/FollowerReplicator.java`: `pollOnce(intmaxRecords,intmaxBytes)` returns the number appended in this poll; also connect the `REPLICA_FETCH` branch in `handle(Messages.Request)` in `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`.

Inputs, outputs, and state. `maxRecords` is a positive record-count limit and `maxBytes` is a positive encoded-byte budget; records are indivisible, so a budget that cannot fit the next whole record may make no progress. The constructor requires a nonnegative broker id matching `ReplicaState`; the topic-partition, log, borrowed `RpcClient`, and state are non-null. The poll returns the number actually appended; an empty reply returns 0. The caller owns the RPC client, which the poll does not close. Success changes follower disk and local LEO; it does not change authority reported LEO, ISR, or HW.

Behavioral contract and invariants. First verify under the state monitor that the local broker is online and a follower, and capture its epoch; issue the request outside the monitor; validate the response; then recheck epoch/role and append under the same monitor. The request carries the current partition, local LEO, broker id, captured epoch, both positive budgets, and `recoveryRead=false`.

- The response must be a replica-fetch body with the captured epoch; its leader LEO cannot be below local LEO. Returned records must start at local LEO, have contiguous offsets, and be below the body's leader LEO. Receiving bytes is not success: only after `PartitionLog.append` succeeds can the method count those records.
- A nonpositive limit or unexpected body is `INVALID_REQUEST`; offline/already-leader local role is `NOT_LEADER`; a local epoch change during RPC or a wrong response epoch is `FENCED_EPOCH`; a regressing leader LEO, missing offset, wrong first offset, or misaligned append range is `CORRUPT_RECORD`. A remote leader error (for example `OFFSET_OUT_OF_RANGE` below its retained start) and its message are propagated; a local storage error retains its error.
- None of these validation failures may change the follower log; if its role changes during the network request, it must not append. An empty reply at leader LEO is valid; repeated empty polls do not change file bytes. The broker caps an oversized positive byte request to one reply-frame budget; later polls copy the remaining contiguous suffix. If local writing returns `STORAGE_ERROR`, do not count those records or report progress; some disk side effect may already have happened, so do not assume rollback and follow the log's failed/reopen recovery path. This step does not call report or advance HW.

Tiered hints (not the answer)**Level 1**

Track “network reply received” and “follower disk append succeeded” separately.

Level 2

Compare response epoch, first offset, contiguous offsets, and leader LEO; do not substitute HW for LEO.

Level 3

Poll the same follower again, then supply a wrong epoch, wrong first offset, and an offset beyond leader LEO; verify that its LEO and file bytes remain unchanged.

Observable examples and test points. Worked example assumptions: partition start 0, epoch 0, broker 1 is leader and broker 2 follower; the leader disk has three ordinary records with null keys, one-byte ASCII values `a`, `b`, `c`, and timestamps 11, 12, 13, so each occupies 33 bytes. Leader LEO=3, HW=0, and initial authority reportedLEO is still 0 for all three brokers. Broker 2 calls `pollOnce(10, 16384)`, receives offsets 0–2, appends all three, returns 3, and has local LEO=3; authority is unchanged. A second poll receives an empty reply, returns 0, and leaves file bytes unchanged. Ordinary consumers cannot yet see these records at HW=0, while replica requests can read them.

Tests and completion. The real `Step21Test` covers `replicaFetchIncludesUncommittedLeaderTailAndReturnsEmptyAtLeo`, `followerCopySurvivesEmptyPollsAndCatalogReopen`, `boundsLargeReplicaFetchesAndCopiesTheCompleteSuffix`, `rejectedReplicaFetchRequestPreserveDiskAndAuthority`, `followerAheadOfLeaderIsRejectedWithoutTruncatingItsLog`, and `malformedRealTcpReplicaFetchRepliesAreRejectedBeforeAppend`: real TCP, disk bytes, empty polls, frame-bounded paging, no truncation when a follower is ahead, side-effect-free epoch/offset errors, and reopened contents. The single-step command diagnoses Step21; the cumulative command runs Steps1–21. An unfinished skeleton is expected to fail, and a green single-step test does not complete Chapter 6.

Common pitfalls and differences from real Kafka

Sharing the leader’s `PartitionLog` bypasses the network and follower persistence; treating HW as leader LEO prevents copying the uncommitted tail. Real Kafka’s replica protocol is more involved; `REPLICA_FETCH` is the course protocol, not evidence about Kafka’s internal frames or control plane.

Commands and progression. Use `./gradlew :stepTest -Pstep=21` to diagnose this step; `./gradlew :test -Pstep=21` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can a replica fetch obtain records above HW that ordinary consumers cannot yet see?
2. Why must the follower not report its new LEO to the authority before its local append succeeds?

Self-check answers. Replication must transfer the leader’s uncommitted tail so the follower can catch up to the current LEO; ordinary consumers use a different HW boundary. Persisting before reporting prevents the authority from counting data that is not yet on the follower’s disk.

7.2 Step 22: ISR and High Watermark

Goals and prerequisites

Goal: Calculate the consumer-visible boundary only from validated progress reports, and remove long-lagging followers under a controlled clock.

Before you begin: Understand that physical LEO, authority reports, ISR, minISR, and HW are different state.

Background and reasoning

Prerequisite: [Step 21 \(Fetch-Based Replication\)](#). A follower may have written data in its own log without reporting it yet. The authority keeps each broker's most recently successful reported LEO for a partition; this can temporarily differ from the physical LEO just read from that log. ISR is the set of replicas currently participating in acknowledgment; minISR is the minimum ISR size for ALL requests; HW is the exclusive end of the prefix they have confirmed together. Only a successful report changes authority progress.

Starting point and edit location.

Already provided: Per-partition locks and Conditions, assigned-replica membership, online status, authority progress containers, and an injectable millisecond `TimeSource` are provided.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicationTracker.java`: `report(intbrokerId,intepoch,longleo)` returns void; `expireLagging()` returns the set of removed broker ids; `advanceHighWatermark()` returns the current HW.

Inputs, outputs, and state. `report` requires a nonnegative broker id belonging to the assignment and online, plus nonnegative epoch/LEO and the current leader epoch; invalid input leaves the snapshot unchanged. A leader report cannot move backward; a follower report cannot exceed the leader's reported LEO or move backward in the same epoch. A successful report updates that broker's reportedLEO; a follower refreshes its last-caught-up time only when its LEO reaches the leader LEO observed for that report. A report does not readmit a member already removed from ISR.

Behavioral contract and invariants. If ISR contains the leader, all member progress exists, and ISR size is at least minISR, the candidate HW is the minimum latest successful reported LEO in ISR, applied as $HW = \max(\text{oldHW}, \text{candidate})$; otherwise HW freezes. HW never decreases.

- Negative broker id, epoch, or LEO; unknown/unassigned/offline broker; a backward leader LEO; or a follower ahead of or behind its permitted progress is `INVALID_REQUEST`. A wrong epoch is `FENCED_EPOCH`. A rejected report changes neither reportedLEO, ISR, nor HW.
- `expireLagging()` checks non-leaders in ISR. A member with no catch-up time or with `now-lastCaughtUp >= lagTimeoutMillis` is removed and included in the immutable returned set. Timeout 0 means immediate expiration; if now precedes the catch-up time, elapsed time does not expire it. Expiration removes neither assignment nor log, and never removes the leader.
- After removals, reconsider HW; if the remaining ISR is below minISR, it freezes. Successful reports, actual expiration, or HW advancement notify waiters; `advanceHighWatermark()` recomputes HW from the current ISR and returns it. If HW actually advances, it signals waiters; the method itself does not report replica progress or expire lagging replicas.

Tiered hints (not the answer)**Level 1**

Validate follower reports against “not beyond leader” and “not backward” before calculating HW.

Level 2

If minISR is insufficient, do not advance from the remaining LEOs; refresh catch-up time only on catch-up.

Level 3

Use a fixed clock to check one millisecond before a deadline, the exact deadline, and one invalid report; compare snapshots without sleeps.

Observable examples and test points. Worked progress example: replicas/ISR=1,2,3, minISR=2, previous HW=0, and reported LEOs for leader1 and followers2,3 are 5,3,4. The minimum candidate is3, so HW=3. Follower2 reports5: candidate and HW become4. Follower3 reports5: HW becomes5. Separately, follower3 last caught up at clock=1000ms with lag timeout=1000ms; now=1999ms retains it, while now=2000ms expires it at the exact boundary. Its log is not rewritten.

Tests and completion. `Step22Test.rejectsInvalidProgressAndExpiresOnlyReplicasThatMissTheExactCatchUpDeadline` starts the clock at 1000 and verifies that negative progress, a future epoch, unassigned/offline replicas, and follower overrun/rollback leave the snapshot unchanged; no member expires at 1999, while broker 3 expires at 2000. It also checks HW freeze as ISR shrinks, progress to 4 after a new report, and that HW stays at 4 when only the leader reports 5. The single-step command diagnoses Step 22; the cumulative command runs Steps 1–22. A green single-step test does not complete Chapter 6.

Common pitfalls and differences from real Kafka

Online does not mean ISR, and physical log LEO does not mean authority-known LEO; using the former directly for HW could expose unconfirmed data. The course serializes state in a trusted single-JVM authority; real Kafka does not guarantee the global HW monotonicity across epochs used here.

Commands and progression. Use `./gradlew :stepTest -Pstep=22` to diagnose this step; `./gradlew :test -Pstep=22` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can insufficient minISR prevent HW from advancing even when some replicas have written the data?
2. Why does an online follower that has not caught up to leader LEO fail to refresh its catch-up time?

Self-check answers. HW marks a readable common prefix, not one replica’s write position; an undersized ISR cannot meet this course’s acknowledgment condition. Only a report that has caught up proves the follower reached the current leader tail; a live connection alone proves nothing about its data.

7.3 Step 23: Produce Acknowledgments

Goals and prerequisites

Goal: Explain the difference among a write, a replication commit, and an acknowledgment delivered to the caller, then implement both wait rules.

Before you begin: Understand half-open offset ranges, Conditions, monotonic-clock deadlines, and why a timeout does not undo an append.

Background and reasoning

Prerequisite: [Step 22 \(ISR and High Watermark\)](#). An ACK setting tells the producer how far the broker must wait before replying: **LEADER** returns when the leader's local append completes; **ALL** requires sufficient **minISR** before append and waits for **HW** to reach the batch's **nextOffset**. For a returned range $[a,b)$, $HW \geq b$ means the entire batch is in the committed prefix; readers can fetch only offsets below **HW**. A deadline is an absolute instant in the `System.nanoTime()` time domain, while `timeoutMillis` is a request duration; they are not interchangeable.

Starting point and edit location.

Already provided: The `ReplicationTracker` partition lock/Condition, `AppendResult`, `Acks`, `RecordData`, and `TimeSource` are provided; `deadlineAfterMillis(long)` is provided too.

Implement in: `exercises/src/main/java/io/simplekafka/replication/AckPolicy.java`
 a: `validateBeforeAppend(Acksacks)` returns void; `await(AppendResult result, Acksacks, int epoch, long deadlineNanos)` returns void.

Inputs, outputs, and side effects. `validateBeforeAppend` accepts non-null **LEADER/ALL**; **LEADER** needs no **ISR** check, while **ALL** checks **ISR** size before append. `await` accepts a non-null result and ACK mode, the epoch captured for that append, and an absolute monotonic deadline; **LEADER** returns immediately, while **ALL** waits for the result's **nextOffset**. The caller's `ReplicatedPartition.produce` rejects negative timeout or epoch. Null arguments throw `NullPointerException` under the interface contract. Any wait error after append does not retract the disk record.

Behavioral contract and invariants. Before append, **ALL** with $ISR < minISR$ reports `NOT_ENOUGH_REPLICAS`, so no record has yet been appended. After append it rechecks in this order: `epoch`→`ISR`→`HW`→`deadline`. An epoch change reports `FENCED_EPOCH`; **ISR** below **minISR** reports `NOT_ENOUGH_REPLICAS`; if **HW** has reached result `nextOffset`, return success; otherwise an expired deadline reports `REQUEST_TIMEOUT`. Satisfied **HW** takes precedence over an expired deadline. Condition waiting releases the partition lock; after waking, recheck all conditions. Interruption restores the thread's interrupt flag and reports `REQUEST_TIMEOUT`, retaining the interrupt as cause. **LEADER**'s `await` completes immediately.

Tiered hints (not the answer)

Level 1

Separate the pre-append **minISR** check from the post-append **HW** wait; their failures have different side effects.

Level 2

A Condition signal only says state may have changed; reacquire the lock and check every condition in order.

Level 3

Use latches to control report order and verify that a report wakes **ALL**; then test zero timeout, epoch change, **ISR** reduction, and interruption against **LEO/HW**.

Observable examples and test points. Assume leader epoch=0, ISR=1,2,3, minISR=2. Append one ordinary record with null key and empty value (32 encoded bytes), returning [0,1); leader LEO=1, follower LEO=0, HW=0. ALL with timeout 0 reports `REQUEST_TIMEOUT` after the append: LEO stays 1, HW stays 0, the caller has no receipt, and a consumer cannot see offset 0. Later both followers copy and report LEO=1, so HW may reach 1 and the record becomes visible; the old request does not retroactively receive a successful receipt. If ISR was already too small before append, ALL instead reports `NOT_ENOUGH_REPLICAS` with LEO still 0.

Tests and completion. `Step23Test` contains `insufficientIsrRejectsAllBeforeAppendWhileLeaderAckStillAppends`, `exactHighWatermarkBoundarySucceedsButOneOffsetBelowWaitsForProgress`, `zeroTimeoutRetainsTheAppendWithoutAdvancingTheHighWatermark`, `epochChangeWhileWaitingFencesTheAcknowledgement`, and `isrReductionWhileWaitingFailsAndInterruptionPreservesTheInterruptFlag`. They verify side-effect-free prechecks, HW boundary, retained timeout append, epoch/ISR wake-ups, and interrupt preservation. The single-step command diagnoses Step 23; the cumulative command runs Steps 1–23. Chapter completion requires cumulative Step 24.

Common pitfalls and differences from real Kafka

Do not wait on replication while holding the partition lock or the only progress-reporting thread may be blocked; wall-clock adjustments do not belong in monotonic deadlines. A timeout means the caller did not receive confirmation by the deadline, not that no append occurred. The course supports only two ACK modes and does not retry for the caller.

Commands and progression. Use `./gradlew :stepTest -Pstep=23` to diagnose this step; `./gradlew :test -Pstep=23` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must epoch, ISR, HW, and deadline be checked again after a Condition releases the lock?
2. After an ALL timeout, what can be true about LEO, HW, and the caller's knowledge of the append?

Self-check answers. Any state may change while waiting, and a signal does not prove the condition is satisfied. LEO may have advanced while HW has not, so data can become visible later even though the caller never received a safe success receipt.

7.4 Step 24: Concurrent Replicated Broker

Goals and prerequisites

Goal: Combine leader validation, appends, replication acknowledgments, and ordinary reads in a broker serving concurrent requests.

Before you begin: Understand authority/local-log lock order, HW read clipping, and socket/Condition lifecycle.

Background and reasoning

Prerequisites are [Step 21 \(Fetch-Based Replication\)](#), [Step 22 \(ISR and High Watermark\)](#), and [Step 23 \(Produce Acknowledgments\)](#). An ordinary consumer can read only the complete committed prefix offset $< HW$; a valid offset within the retained range but $offset \geq HW$ returns an empty list. An offset beyond LEO is out of range. Replica fetch is a separate request that may obtain the uncommitted tail. Produce permission checks, append, and leader progress are linearized under the authority per-partition lock; the lock order is authority lock $\rightarrow$ local `PartitionLog` monitor. No network call may hold either lock; an ALL wait must run outside them so follower reports can proceed.

Starting point and edit location.

Already provided: Independent TCP endpoints, broker workers, `ReplicaState`, `ReplicationTracker`, `AckPolicy`, `PartitionLog`, and broker routing are provided.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java`: `produce(List<RecordData>records, AckSacks, int epoch, long timeoutMillis)` returns `AppendResult`; `fetch(long offset, int maxRecords, int maxBytes, int epoch)` returns `List<LogRecord>`.

Parameters, results, and state. `produce` requires non-null records/ACK mode, nonnegative epoch and timeoutMillis, and a currently online leader matching the request epoch; success returns a half-open append range and may append to the log/update replication progress. `fetch` requires nonnegative offset/epoch and positive record/byte limits; it only reads log and authority state, returning complete records bounded by count, encoded bytes, and HW. It never splits a record. The provided handler routes network workers.

Behavioral contract and invariants. After argument/null checks, `produce` first runs ALL's pre-append minISR check, before online/role/epoch checks. Under the authority lock it then synchronizes and validates online/leader/epoch, rechecks ALL minISR to cover state changes since the first check, appends locally, reports leader progress/possible HW advancement, unlocks, and waits for the ACK. Either failed minISR check or role/epoch failure must not append; the first check can make a minISR error take precedence over `NOT_LEADER/FENCED_EPOCH`. Negative parameters or an empty produce batch are `INVALID_REQUEST`, a non-leader is `NOT_LEADER`, a stale epoch is `FENCED_EPOCH`, and local storage failure is `STORAGE_ERROR`. Post-append `NOT_ENOUGH_REPLICAS`, `REQUEST_TIMEOUT`, or `FENCED_EPOCH` does not roll back the append.

- Fetch rejects a negative offset/epoch or nonpositive budget with `INVALID_REQUEST`; a stale epoch is `FENCED_EPOCH`; a requester that is not the online leader is `NOT_LEADER`; $offset < \logStart$ or $offset \geq LEO$ is `OFFSET_OUT_OF_RANGE`.
- Fetch's legal offset domain is the closed interval $[\logStart, LEO]$; a value in that domain with $offset \geq HW$ returns empty, while $offset < HW$ returns only whole records with $offset < HW$. LEO is the next offset, so $offset = LEO$ can be valid and empty; greater than LEO is invalid.
- Keep lock order authority $\rightarrow$ local log; do not reverse it or hold locks across a broker RPC. Broker close must let sockets and waiting ACK workers finish; a pending append may remain in the log.

Tiered hints (not the answer)

Level 1

Draw the lock intervals for produce, replica poll, and consumer fetch, placing every TCP call outside them.

Level 2

Check that HW clipping happens before an ordinary fetch reply, rather than asking consumers to discard uncommitted records.

Level 3

Pause replication after an ALL append, fetch from another connection, and issue a LEADER append concurrently; resume replication and verify all three can finish.

Observable examples and test points. Concurrency example assumptions: leader=1, followers=2/3, epoch=0, initially empty log. First a LEADER append writes offset 0 and is replicated; all three LEOs=1 and HW=1. Then ALL appends offset 1 and waits; leader LEO=2, HW=1. A separate fetch(0) sees only offset 0, and fetch(1) is empty. During the wait, LEADER can append offset 2 (range [2,3)), but fetch still sees only offset 0. The followers then copy offsets 1–2 and report LEO=3; HW advances to 3, ALL returns [1,2), and ordinary fetch can now see offsets 0, 1, and 2. For the byte-budget fixture, assume each ordinary record has null key and empty value, hence 32 encoded bytes: among four appended records with HW=2, fetch from 0 with maxBytes=31 returns none, 32 or 63 returns offset 0 only, and 64 returns offsets 0 and 1; maxRecords=1 still caps the result at one record.

Tests and completion. `Step24Test.pendingAllAckLeavesFetchAtOldHwAllowsLeaderProduceAndCompletesAfterTcpReplication` checks reads at old HW and progress of another produce while waiting; `fetchHonorsHwLeoAndWholeRecordByteBudgetsAndRejectsInvalidRequests` checks the 31/32/63/64-byte boundaries, HW/LEO, and out-of-range case; `allPrecheckDoesNotAppendAndZeroTimeoutRetainsAnInvisibleAppendUntilReplication` and `stoppingLeaderCompletesAnOutstandingAllAckRequestWithoutLeakingTheProducer` check failure side effects and shutdown wake-up. The single-step command diagnoses Step 24; the cumulative command runs Steps 1–24 and is the Chapter 6 completion boundary.

Common pitfalls and differences from real Kafka

Holding a lock across a socket can prevent the only replication-progress thread from running; HW is not a group committed offset. The three independent logs replicate over real TCP, but the authority remains a single point in one shared JVM, so this is not production HA.

Commands and progression. Use `./gradlew :stepTest -Pstep=24` to diagnose this step; `./gradlew :test -Pstep=24` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can another fetch safely read the old HW prefix while ALL is waiting?
2. Why might closing only the broker socket be insufficient to end every waiting thread?

Self-check answers. Fetch reads the already committed, immutable prefix and cannot leak the uncommitted offset. A Condition waiter also needs a wake-up and an exit check on state change/shutdown; closing a connection alone does not replace worker and waiter lifecycle management.

Chapter 8

Election, Log Repair, and Replica Readmission

This chapter connects replica state changes into a controlled failover. Keep the model’s limitations in mind: the course’s authority is a trusted metadata/HW control plane in a single JVM, and leader changes are triggered explicitly by the course; this is not KRaft controller consensus and does not cover dual-leader arbitration during network partitions. Followers read leader data over real TCP, and each broker still writes its own independent log. The course chooses a clean candidate rather than trading data loss for availability; if there is no eligible candidate, the service must remain unavailable.

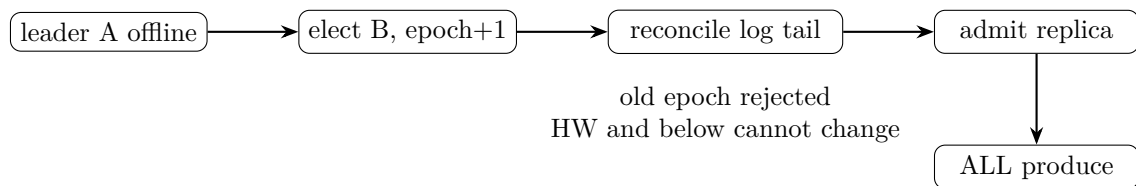


Figure 8.1: A role change must first fence the old role, then repair and prove the replica complete, and only then readmit it to ISR.

8.1 Step 25: Clean Election and Leader Epochs

Goals and prerequisites

Goal: When the old leader is offline, choose only an online replica with the committed prefix and make requests from the old role obsolete.

Before you begin: Understand eligibility, reported LEO, the HW boundary, and epoch fencing.

Background and reasoning

Prerequisite: [Step 24 \(Concurrent Replication\)](#) and [Step 22 \(ISR and HW\)](#). A leader is the broker currently allowed to accept writes; a follower has its own partition log. An election selects which known copy continues to serve after a leader failure—it is not an arbitrary choice of any online node. A leader epoch is a number for a role term. It increases when the role changes; requests carry an epoch so a broker can recognize an old term. Epoch fencing rejects later requests from an old role, but cannot undo earlier disk or business side effects. The course authority is a trusted,

single-JVM control plane and election is triggered explicitly. If no safe candidate exists, the model prefers unavailability to an unclear election.

Starting point and edit location.

Already provided: Provided `ClusterAuthority` (`provided/src/main/java/io/simplekafka/cluster/ClusterAuthority.java`) supplies `public boolean isOnline(int brokerId)`, `public PartitionMetadata metadata(TopicPartition tp)`, and `public ReplicationSnapshot hotSnapshot(TopicPartition tp)`; `ReplicationSnapshot` is provided `provided/src/main/java/io/simplekafka/cluster/ReplicationSnapshot.java`. The three broker networks and local logs are provided.

Implement in: `exercises/src/main/java/io/simplekafka/replication/LeaderElection.java`: `public PartitionMetadata elect(TopicPartition tp)` and `public void checkLeader(int brokerId, TopicPartition tp, int epoch)`.

Inputs, results, and state. `elect` reads the current leader, epoch, old ISR, online state, reported LEO, and HW for a known topic-partition; it has no separate transport input. If the leader remains online, it returns unchanged metadata without mutating state. A successful change updates authority roles/progress, but does not directly change any broker's disk. `checkLeader` checks broker id, partition, and request epoch.

Behavioral contract and invariants. A candidate must belong to the ISR at the start of election, be online now, and have authority-reported $LEO \geq HW$; a longer-looking physical log cannot replace a report. Choose the smallest broker id among eligible candidates, not the highest LEO. On success, increment epoch exactly once, keep HW, reset ISR to only the new leader, and reset other replicas' authority reported LEO to 0 so they must recover and be admitted before acknowledgments count them. An old-epoch request returns `FENCED_EPOCH`; the correct epoch from a broker that is not the online leader returns `NOT_LEADER`. Wake waiters still awaiting the former leader's acknowledgment.

- If the known leader is online, return current metadata and leave epoch/ISR/HW/progress unchanged. With no candidate, return `NO_ELIGIBLE_LEADER`; metadata, epoch, HW, and ISR remain unchanged. An online broker outside the old ISR cannot replace it.
- A negative broker id or epoch is `INVALID_REQUEST`; an unknown partition is `UNKNOWN_TOPIC_OR_PARTITION`. Election and validation do not make TCP RPCs.

Tiered hints (not the answer)

Level 1

Apply old ISR, online status, and reported $LEO \geq HW$ in that order before selecting a candidate.

Level 2

Distinguish an old epoch (`FENCED_EPOCH`) from the correct epoch at a broker with the wrong role (`NOT_LEADER`).

Level 3

Try a still-online leader, an ISR replica below HW, an online broker outside ISR, and a state with no eligible candidate; compare authority snapshots before and after.

Observable examples and test points. Assume leader 1 is offline, epoch=0, old ISR=1,2,3, and HW=3. The no-retention log starts at offset 0; the common prefix contains three records with null key, one ASCII value byte each, and fixed timestamps, so each occupies 33 bytes. Broker 2 reports LEO=3; broker 3 reports LEO=5; both are online ISR members. Both meet the HW bound, so the smaller id, broker 2, wins even though broker 3 has a longer log. The result is epoch=1, leader=2, ISR=2, HW=3. Authority-reported LEO for other replicas resets to 0, but election does not delete their physical logs. Broker 3 then gets `NOT_LEADER` with epoch 1; an old epoch 0 request gets `FENCED_EPOCH`. If all candidates are offline or below HW, even online broker 4 cannot be added, and election leaves state unchanged.

Tests and completion. `Step25Test.electsTheLowestEligibleIsrAndFencesOldEpochWithoutLosingCommittedPrefix` checks no-op elections while the leader is online, failover, retained HW, and old-epoch fencing; `electionRequiresTheCommittedPrefixBeforeChoosingTheLowestIsr` checks that a smaller id below HW is skipped; `electionFencesAnOutstandingAllAckWithoutRollingBackAppend` checks that a pending append is not rolled back. The single-step command isolates Step25; the cumulative command runs Steps1–25. A green single-step run is not chapter completion.

Common pitfalls and differences from real Kafka

Choosing the online replica with the largest LEO can select a non-ISR or divergent copy; silently choosing a lagging replica when no candidate qualifies can lose the known committed prefix. Real Kafka’s controller, epoch history, and consensus are outside this course model; the course does not promise production HA if the authority is lost or the network is partitioned.

Commands and progression. Use `./gradlew :stepTest -Pstep=25` to diagnose this step; `./gradlew :test -Pstep=25` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. What minimum safety condition does candidate `reportedLEO ≥ HW` provide?
2. Why can a broker with the correct epoch but not the leader still reject produce?

Self-check answers. The condition says that the candidate has at least the prefix the authority marked committed; it does not prove that an uncommitted tail is the same. Epoch identifies the term, while leader identity determines current write permission; both must hold.

8.2 Step 26: Divergent-Log Reconciliation

Goals and prerequisites

Goal: Compare old and new leader contents at matching offsets, protect their committed prefix, and repair only a suffix allowed to change.

Before you begin: Understand full-prefix scans, recovery reads, the strict HW boundary, and how long a proof remains valid.

Background and reasoning

Prerequisites: [Step 25 \(Clean Election\)](#) and the log/truncation and follower-fetch ideas from [Step 8 \(Tail Truncation\)](#) and [Step 21 \(Follower Pull Replication\)](#). Equal LEOs mean equal record counts, not equal contents; repair compares each `RecordData` at the same offset. The course uses a simple but slower full-prefix scan from offset 0 to the captured leader LEO. Recovery reads can inspect the leader’s tail above HW but do not report replication progress. HW is an exclusive boundary: a conflict at offset $<$ captured HW is committed and cannot be overwritten; the offset exactly equal to HW is in the uncommitted tail and may be repaired. A possible truncation point must not be below either captured HW or the current HW rechecked at completion.

Starting point and edit location.

Already provided: `Provided RpcClient.call(Messages.Request):Messages.Reply` (provided/src/main/java/io/simplekafka/transport/RpcClient.java), `Messages.Replica`

FetchRequest/ReplicaFetchBody (provided/src/main/java/io/simplekafka/protocol/Messages.java), RecoveryProof (provided/src/main/java/io/simplekafka/cluster/RecoveryProof.java), and ClusterAuthority epoch/HW snapshots and partition locks are available; repair does not automatically add the replica to ISR.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicaReconciler.java`: `public long reconcile(int expectedEpoch)`; supplied `public Optional<RecoveryProof> recoveryProof()`. Recovery uses `REPLICA_FETCH(recoveryRead=true)`, at most 4 records and 4,200,000 bytes per page.

Inputs, outputs, and state. `expectedEpoch` must match the current epoch. Broker id, topic-partition, local log, borrowed `RpcClient`, and authority are fixed at construction. Reconciliation captures the leader's reported LEO/HW and the local `mutationVersion` at offset 0, then makes RPCs outside the lock. On success it returns the local end offset and publishes a proof; recovery reads themselves do not change ISR, reportedLEO, or HW. Entry clears any old proof; a failed validation publishes no new proof.

Behavioral contract and invariants. Scan the leader's captured prefix and the local complete prefix to find the first actual content difference or the point where one ends.

- A stale expected epoch is `FENCED_EPOCH`; an unknown partition is `UNKNOWN_TOPIC_OR_PARTITION`; the local broker being the leader or unassigned is `INVALID_REQUEST`. A nonzero local retained start, unreadable/noncontiguous leader prefix, committed conflict, or truncation below the final HW is `CORRUPT_RECORD`.
- A differing local record below captured HW is `CORRUPT_RECORD` and the file remains unchanged. A missing local record, including a missing committed prefix, can be copied from the leader. A first mismatch exactly at captured HW can be rewritten. A local uncommitted suffix beyond the leader LEO can be truncated.
- RPC timeout or a local log API mutation during the scan (even a truncate-plus-append returning LEO to the same value) is `REQUEST_TIMEOUT`; the log is not repaired and no proof is published. Epoch or leader changes during the scan are `FENCED_EPOCH`. Remote error codes propagate; an unexpected body is `INVALID_REQUEST`; local disk failure remains `STORAGE_ERROR`. Multi-file truncate/append is not guaranteed to roll back atomically on I/O failure.
- Under authority-lock then local-log-monitor order, recheck epoch, leader, HW, start, LEO, and `mutationVersion` before repair and proof publication; hold the log monitor through both. The proof binds captured epoch, leader LEO and HW, local log object identity, and final `mutationVersion`. It is not persistent and does not detect external disk changes that bypass the API.

Tiered hints (not the answer)

Level 1

Separate “the local record is missing” from “a local record exists with different content”; they are different repair situations.

Level 2

Treat HW as an exclusive boundary: offsets below it are protected, while the offset equal to it can be the first repairable one.

Level 3

Construct a common committed prefix plus a divergent uncommitted tail, then a short-/empty replica; check both disk and proof state.

Observable examples and test points. Assume offsets 0–2 are common committed records, key=null and one ASCII value byte per record, 33 bytes each; HW=3 and captured leader LEO=4. The old replica has X at offset 3 and the new leader has Y at offset 3,

with the same timestamp. Since the first difference is exactly at $3=HW$, repair leaves offsets 0–2 unchanged, writes Y at 3, returns $LEO=4$, and creates a proof; authority ISR/HW do not change. In an independent state where offset 2 differs ($2 < HW$), reconciliation returns `CORRUPT_RECORD`, leaves local file bytes and authority unchanged, and has cleared any old proof. An empty local log can instead be filled with offsets 0–3.

Tests and completion. `Step26Test.repairsOnlyDivergentUncommittedTailAtHighWatermarkAndReturnsLeaderPrefixProof` checks repair at $offset=HW$ and proof publication; `refusesMismatchAtHighWatermarkMinusOneWithoutChangingTheLogOrAuthorityAndClearsOldProof` checks rejection at $HW-1$; `fillsShortAndEmptyReplicasWithTheCommittedPrefix`, `trimsOnlyAnUncommittedTailWhenTheLocalReplicaHasTheCompleteLeaderPrefix`, and `repairsTenRecordsInFourRecordFetchRoundsWithoutReportingRecoveryProgress` check missing/excess tails and four-record pages. Other tests exercise RPC timeout, malformed replies, same-LEO mutations, and epoch/HW races. The single-step command isolates Step26; the cumulative command runs Steps1–26.

Common pitfalls and differences from real Kafka

A local $HW=0$ does not make an existing valid local prefix safe to delete; a missing record is not a content conflict. Real Kafka uses leader-epoch history to locate divergence more efficiently; this course’s $O(n)$ scan is a teaching model, not a production repair algorithm.

Commands and progression. Use `./gradlew :stepTest -Pstep=26` to diagnose this step; `./gradlew :test -Pstep=26` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why handle a conflicting committed record differently from a missing committed record?
2. If the leader appends after the scan snapshot, why is the old proof no longer safe for admission?

Self-check answers. A conflict means the replicas disagree about a protected committed value, so silently overwriting it is unsafe; a missing record means the local replica can catch up from the available leader prefix. A leader LEO change means the proof did not examine the current tail, so admission must require a new scan.

8.3 Step 27: Metadata Routing Refresh

Goals and prerequisites

Goal: Route a partition request through cached leader metadata without automatically replaying a business operation after an error.

Before you begin: Learn bootstrap discovery, cached routes, uncertain outcomes, and why refreshing the control plane is not a data-request retry.

Background and reasoning

Prerequisites: [Step 26 \(Divergent-Log Reconciliation\)](#), and the producer and acknowledgment behavior in [Step 13 \(Batched Producer\)](#) and [Step 23 \(Produce Acknowledgments\)](#). A bootstrap

endpoint is an entry point for discovering the cluster. Metadata gives the current leader endpoint and epoch for each topic-partition. The cached leader is the data-request destination; authority/metadata requests are control-plane traffic, while produce/fetch are data-plane traffic. An uncertain outcome means the broker may have finished appending, but the reply was lost or acknowledgment failed, so the client cannot tell whether a write happened. Automatically resending ordinary produce can create a second record.

Starting point and edit location.

Already provided: `Provided RpcClientFactory.connect(Endpoint):RpcClient` (provided/src/main/java/io/simplekafka/transport/RpcClientFactory.java), `RpcClient.call(Messages.Request):Messages.Reply` (provided/src/main/java/io/simplekafka/transport/RpcClient.java), `Endpoint` (provided/src/main/java/io/simplekafka/model/Endpoint.java), and `MessageCodec.encodeRequest/decodeRequest` (provided/src/main/java/io/simplekafka/protocol/MessageCodec.java) handle connections/frames; broker metadata is supplied by course servers. The existing single-broker `RpcClient` constructor path remains available.

Implement in: `exercises/src/main/java/io/simplekafka/client/MetadataRouter.java`: constructor `MetadataRouter(List<Endpoint>bootstrap,RpcClientFactoryfactory)` requires a nonempty bootstrap list with no null entries and a non-null factory; `publicvoid refresh(Stringtopic)` and `publicMessages.Replycall(TopicPartitiontp,Messages.Requestrequest)`. The router caches and owns `RpcClients` created by its factory; `close()` closes those clients. Router client constructor paths are also in `exercises`: `SimpleProducer(MetadataRouter,Partitioner,int)`, `SimpleConsumer(MetadataRouter,String)`, and `GroupConsumer(MetadataRouter,String,String)`; do not remove the single-broker `RpcClient` overloads.

Inputs, outputs, and state. `refresh` requests metadata for one topic using bootstrap endpoints in construction order. A connection/factory failure or `REQUEST_TIMEOUT` can fall through to the next endpoint. If every endpoint fails, propagate the last failure instead of fabricating a route. Invalid topics or a reply for a different topic return `INVALID_REQUEST`. A failed refresh preserves the last successful cache. In Step27, `call` accepts matching `ProduceRequest` or `FetchRequest`; Step30 later adds `IDEMPOTENT_PRODUCE`.

Behavioral contract and invariants. `call` sends one business request to the cached leader endpoint with the cached epoch. An unsupported request or mismatch between request partition and `tp` is `INVALID_REQUEST`; absent topic/partition metadata is `UNKNOWN_TOPIC_OR_PARTITION`.

- On a received `NOT_LEADER` or `FENCED_EPOCH` response, the router may make a best-effort metadata refresh so a later independent call can use the new route; it must return the original Reply. A refresh failure does not replace that Reply.
- Other business errors, RPC timeouts, I/O errors, and storage errors do not automatically refresh or replay produce or fetch. In particular, a timeout may happen after the broker appended; an unknown outcome does not mean “nothing happened.”
- Refresh only reads metadata; the router does not retry the current business operation. A later request is an explicit caller decision. Using a closed router throws `IllegalStateException`.

Tiered hints (not the answer)

Level 1

Draw only one business request for a `call`; refreshing metadata is another control-plane request, not a second produce.

Level 2

For bootstrap fallback, distinguish connection failures from error replies returned by a broker.

Level 3

Use an old route to elicit `FENCED_EPOCH` or `NOT_LEADER`, then compare the returned reply, refreshed cache, and next explicit request.

Observable examples and test points. Assume bootstrap order [broker1,broker2,broker3], a cache pointing to broker1/epoch0, and an empty partition. Each example record has null key, a one-byte ASCII value, and a fixed nonnegative timestamp (33 bytes). Broker1 is offline and broker2 has been elected at epoch1. The stale-route request receives an original `NOT_LEADER` or `FENCED_EPOCH` reply; that same reply returns to the caller while refresh changes the cache to broker2/epoch1. No produce is sent again to broker2 by that call, so it creates no receipt. A later explicit request can receive [0,1), leaving one new log record. If a request instead times out after append, the router still does not replay it; Step29 observes this uncertain result.

Tests and completion. `Step27Test.refreshSkipsUnavailableAndFactoryThrowing` `BootstrapsBeforeCachingMetadata` checks bootstrap order; `failedRefreshTimesOutWithoutReplacingCacheAndLaterSuccessfulRefreshReplacesIt` checks cache preservation; `returnsLeaderErrorsUnchangedAndRefreshesOnlyForTheNextRequest` checks original reply and next-call routing; `routerDoesNotReplayAProduceAfterItsAppendResponseTimesOut`, `makesRoutedProducerOutcomeUnknownAfterEveryAppendError`, and `routerDoesNotReplayStorageOrTransportFailures` check no replay. Other tests cover borrowed-router lifecycle and independent per-topic routes. The single-step command isolates Step27; the cumulative command runs Steps1–27.

Common pitfalls and differences from real Kafka

Treating `NOT_LEADER` as proof that no record was written can duplicate a message; inventing a success receipt after cache refresh is also wrong. Ordinary `SimpleProducer` does not retry automatically. Real Kafka clients have more elaborate retry semantics; this course router refreshes routes but never replays the current business request.

Commands and progression. Use `./gradlew :stepTest -Pstep=27` to diagnose this step; `./gradlew :test -Pstep=27` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why are refreshing metadata and replaying a business request separate actions?
2. Why can a client not infer that the server had no side effect from an error code alone?

Self-check answers. Metadata says where future requests should go; it cannot provide a result for an earlier request. A timeout can occur after append, and `FENCED` can also arise while waiting for acknowledgment. Without a success receipt for that specific request, the caller must treat the result as unknown instead of concluding that nothing was written.

8.4 Step 28: Replica Readmission to ISR**Goals and prerequisites**

Goal: Add a repaired replica to ISR only with fresh prefix proof, and connect repair, readmission, and ALL availability.

Before you begin: Understand assignment, online/follower role, epoch, log identity, and proof version.

Background and reasoning

Prerequisites: [Step 26 \(Divergent-Log Reconciliation\)](#) and [Step 25 \(Clean Election\)](#). Repair makes a log a candidate copy of the current leader prefix; ISR readmission is a separate control-plane decision. Only after admission does the replica count toward ALL/minISR and HW. Equal LEOs mean equal record counts, not equal contents. The reconciliation proof binds a prefix comparison to broker, partition, epoch, leader LEO, local log identity, and mutationVersion; a later leader append, target-log change, truncation, or retention makes the evidence stale.

Starting point and edit location.

Already provided: Provided `ClusterAuthority` (`provided/src/main/java/io/simplekafka/cluster/ClusterAuthority.java`) manages roles/ISR/progress; `RecoveryProof` (`provided/src/main/java/io/simplekafka/cluster/RecoveryProof.java`) records `brokerId`, `tp`, `epoch`, `leaderLEO`, `localLEO`, `highWatermark`, `localLog`, `localMutationVersion`. Replica reopen and authority state containers are provided.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicaAdmission.java`: `public boolean tryAdd(int brokerId, TopicPartition tp, int epoch)`.

Inputs, outputs, and state. `tryAdd` makes no RPC. Its target broker, partition, and expected current epoch are arguments; the local `PartitionLog` is bound at construction. `true` means already in ISR or newly admitted; `false` means current state/proof does not permit admission. A successful transition adds the target to ISR, initializes its reported LEO and caught-up clock from the leader LEO, recomputes HW, signals waiters, and consumes/clears the proof.

Behavioral contract and invariants. For new admission, verify the target is assigned, online, and a follower for the current epoch; the proof's broker/partition/epoch are correct and its leaderLEO equals the current authority-reported leader LEO; the local log is the same bound object, has the same mutationVersion, start=0, and localLEO=leaderLEO, and proof HW is not above leaderLEO. Under authority lock then local-log monitor, do not allow a local-log write between validation and publication of ISR.

- A negative broker id/epoch or unassigned broker returns `INVALID_REQUEST`; an unknown partition returns `UNKNOWN_TOPIC_OR_PARTITION`; local-log read failure returns `STORAGE_ERROR`.
- A stale epoch, offline target, target that is the current leader, or missing/stale proof returns `false` without ISR admission. If proof contents or local identity/version/start/LEO disagree, clear the proof; reconcile again before retry. Any target-log append/delete/truncate/close or a change in leader LEO invalidates old proof, even if truncate-plus-rewrite restores the same LEO.
- An already-in-ISR target returns `true` idempotently only when it is online, is a follower, and the supplied epoch is current; this repeat call leaves HW unchanged. A stale epoch, offline target, or leader target follows the preceding false-return rule. On first admission, set reportedLEO to leader LEO and caught-up time to the current clock. Admission may succeed even if minISR is not met; ALL preflight can still fail.

Tiered hints (not the answer)

Level 1

Treat proof as short-lived evidence for a particular log object and version, not as a permanent Boolean.

Level 2

List the current leader checks and the local identity/version/start/LEO checks that admission repeats.

Level 3

Verify a no-change admission, then append at the leader or truncate the target and try the old proof; reconcile before the final admission.

Observable examples and test points. Assume offset 0 contains one ordinary record with null key, one ASCII value byte, and timestamp 30 (33 bytes); leader1 and replica2 have copied and reported to LEO=1, so HW=1. Leader1 stops; broker2 becomes epoch1 leader with ISR=2. Restarted broker1 still has physical LEO=1 but is outside ISR. Reconciliation creates proof for broker1's log and leader LEO1. If leader2 first appends another same-sized record at offset1 with timestamp31, current leader LEO becomes2; calling `tryAdd(1, tp, 1)` using old proof returns false, ISR/HW remain unchanged, and broker1 remains at LEO1. Reconcile again to copy offset1, then admission returns true, ISR=1,2, reportedLEO=2, and HW can advance to2.

Tests and completion. `Step28Test.proofBecomesStaleWhenLeaderAppendsAfterReconciliation` verifies that leader append invalidates proof; `admissionRequiresCurrentProofAndOnlineAssignedFollowerAndIsIdempotent` covers missing proof, old epoch, offline/unassigned states, and repeated admission; `recoveryProofCannotBeUsedWithAnotherLogObjectWithTheSameRecordsAndVersion` and `admissionRejectsProofAfterTruncateAndAfterDifferentDataRestoresSameLeo` cover identity and same-LEO rewrite. `recoveredProofRestoresIsrForAllLacksAndReplicaLogsConverge` and minISR integration tests check HW wake-up, ALL, and log convergence. The single-step command isolates Step28; the cumulative command runs Steps1–28, the Chapter 7 completion boundary.

Common pitfalls and differences from real Kafka

Equal LEO does not prove equal prefixes; checking only the proof's original epoch without current leader LEO misses a newer tail. Real Kafka ISR admission and replication are more complex; course authority/proof behavior is a teaching model, not a production controller design.

Commands and progression. Use `./gradlew :stepTest -Pstep=28` to diagnose this step; `./gradlew :test -Pstep=28` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why does equal LEO alone not prove that two logs are safe replicas?
2. Why must successful admission update progress, recompute HW, and wake ALL waiters?

Self-check answers. Equal lengths can hold different records; a proof compares contents and binds to a particular log version. Adding a replica to ISR can increase the minimum LEO among enough replicas, so HW must be recomputed and waiters notified to recheck acknowledgment conditions.

Chapter 9

Consistency Across Producers, Brokers, and Consumers

This chapter follows a message across three independent boundaries: producer confirmation, broker replication, and consumer processing. An **unknown outcome** means the server may have completed an action but its reply did not arrive; retrying the same intention can duplicate it. **Idempotence** means repeating a request with the same stable identity and content does not add another durable effect; it is not a transaction. At-least-once processing replays uncertain work and therefore permits duplicate effects. A **side effect** changes state outside the log, such as updating a local ledger. A **transaction** would commit changes across participants as one indivisible unit; this course has no shared transaction spanning producer, replicated log, consumer offset, and ledger. If exactly-once means one business effect across all those boundaries, they must be coordinated; an ACK, HW, commit, or producer idempotence alone does not provide that guarantee. Experiments use real TCP and independent broker logs, recording physical LEO, authority-reported progress, HW, consumer position, durable committed offset, visible records, and callbacks separately.

9.1 Step 29: Cross-Layer Consistency Experiments

Goals and prerequisites

Goal: Reproduce producer, broker, and consumer failures, then record the real state reached at each layer.

Before you begin: Learn from first principles to distinguish a successful append, lost reply, HW visibility, in-memory position, committed group offset, and external side effect.

Background and reasoning

Prerequisites: [Step 27 \(Metadata Routing Refresh\)](#) and [Step 28 \(Replica Readmission\)](#). LEO is the next writable partition offset; physical LEO is a broker's disk end, while reported LEO is the replication progress known to the authority. HW is the replication commit boundary, and consumers can read only offsets < HW. Poll can advance in-memory position; commit stores the next offset used by resume. Neither deletes log records. A callback can change external state before commit. A producer receipt reports the append request's acknowledgment result, not consumer visibility.

Starting point and edit location.

Already provided: Provided real-state tools are in `provided/src/main/java/io/simpleka`

fka/lab/: FaultProxy.java, EffectRecorder.java, ConsistencyObservation.java, ConsistencyTrace.java, and LabSupport.java; ClusterHarness is in provided/src/main/java/io/simplekafka/cluster/ClusterHarness.java.

Implement in: exercises/src/main/java/io/simplekafka/lab/ConsistencyExperiments.java: all five public static methods return ConsistencyTrace: producerOutcomeUnknown(ClusterHarness, TopicPartition, RecordData, FaultProxy), leaderAckLoss(ClusterHarness, TopicPartition, RecordData), allAckTimeout(ClusterHarness, TopicPartition, RecordData), consumerReplay(ClusterHarness, TopicPartition, String, RecordData, EffectRecorder, FaultProxy) (throwsException), and staleGroupSideEffect(ClusterHarness, TopicPartition, String, RecordData, EffectRecorder).

Understand the observation tools. ConsistencyObservation has the exact record type recordConsistencyObservation(Stringphase, ErrorCodeerror, ReplicationSnapshot replication, Map<Integer, Long>physicalLeo, OptionalLongposition, OptionalLong committedOffset, List<LogRecord>visible, List<LogRecord>effects, Optional<AppendResult>receipt); ConsistencyTrace is recordConsistencyTrace(TopicPartitiontp, List<ConsistencyObservation>observations). Effects retain every callback and do not deduplicate. The provided FaultProxy(Endpoint) exposes upstream():Endpoint, start():Endpoint, endpoint():Endpoint, armed():boolean, dropNextReply(short), rejectNextRequest(short), forwarded/dropped/rejected(short):int, and close(); only one API fault can be armed at a time. Drop forwards to the real broker and reads its reply, then discards it; reject returns REQUEST_TIMEOUT and ErrorBody("injectedpre-forwardfailure") before forwarding. An API that does not match the armed fault passes through unchanged. LabSupport.replicateAndReport(ClusterHarness, TopicPartition):void replicates over TCP and reports only durable LEO; observe(String, ErrorCode, ClusterHarness, TopicPartition, OptionalLong, String, EffectRecorder, Optional<AppendResult>):ConsistencyObservation independently reads visible records and committed offset; awaitLeaderLeo(ClusterHarness, TopicPartition, long):void waits for authority progress. Provided EffectRecorder.process(TopicPartition, LogRecord):void and snapshot():List<LogRecord> retain callback order.

Behavioral contract and invariants. Each method requires a caller-owned, already-started cluster, one partition tp, and a non-null unstamped input record. Initial state must be epoch0, leader1, replicas/ISR=1,2,3, all online, each log start=end=0, HW=0, and minISR=2. Consumer scenarios also require no committed group offset and an initially empty recorder; group and recorder must either both be supplied or both absent. The two proxy scenarios require a caller-owned started, unarmed proxy whose upstream is broker1. Invalid initial state, null arguments, or stamped input returns INVALID_REQUEST without appending, changing offsets, recording effects, or altering proxy fault counters. Close only clients created by the scenario; catch only expected failures and propagate unrelated exceptions.

Experiment 1: lost producer reply and explicit resend. The test record is key="customer-A" (10 ASCII bytes), value="request-unknown-73" (18 bytes), timestamp=1723; an ordinary record occupies 32+10+18 = 60 bytes. The reader starts at position0 and the producer uses batchSizeRecords=1. Physical LEO order below is broker 1/2/3; replication is leader/epoch/ISR/reported LEO/HW.

- UNKNOWN: broker appends offset0; the LEADER reply is dropped, so the producer reports REQUEST_TIMEOUT.

State: replication=(1/0/1,2,3/1,0,0/0), physical LEO=(1,0,0);

position=0, committed=none, visible=[], effects=[], receipt=none. The reply is missing.

- POISONED: the same ordinary producer locally rejects a later send, error=REQUEST_TIMEOUT; replication and LEO are unchanged, position=0, committed=none, visible=[], effects=[],

receipt=none. At this point forwarded(PRODUCE)=1, dropped(PRODUCE)=1; the second call did not reach the network.

- **EXPLICIT_RESEND:** a new producer explicitly sends the same data and receives receipt=[1,2). After replicating and reporting all three logs:
replication=(1/0/1,2,3/2,2,2/2), physical LEO=(2,2,2);
reader position=2, committed=none, visible=[offset0,offset1] (same contents), effects=[], error=NONE. Final counters: forwarded(PRODUCE)=2 and dropped(PRODUCE)=1.

Experiment 2: a LEADER receipt does not protect an uncommitted tail. The test record is key="leader-tail" (11 bytes), value="acked-but-uncommitted" (21 bytes), timestamp=2419, for 64 bytes. A reader without a committed offset starts at position0.

- **ACKED_NOT_VISIBLE:** broker1 appends offset0 and returns LEADER receipt=[0,1), error=NONE; replication=(1/0/1,2,3/1,0,0/0), physical=(1,0,0), position=0, committed=none, visible=[], effects=[].
- **ELECTED_WITHOUT_TAIL:** stop broker1 and elect broker2/epoch1; replication=(2/1/2/0,0,0/0), physical=(1,0,0), reader position=0, committed=none, visible=[], effects=[], receipt=none, error=NONE. The old log is still on broker1's disk, but is not part of the new leader's visible HW prefix.
- **OLD_TAIL_REMOVED:** restart broker1, then call **reconcile** and **admit**; offset0 is truncated. New state: replication=(2/1/1,2/0,0,0/0), physical LEO=(0,0,0); position=0, committed=none, visible=[], effects=[], receipt=none, error=NONE. The earlier LEADER receipt is not retroactively revoked.

Experiment 3: ALL timeout followed by later visibility. The test record is key="all-key" (7 bytes), value="late-visible-value" (18 bytes), timestamp=3101, for 57 bytes; consumer position starts at0.

- **TIMEOUT_NOT_VISIBLE:** append with **Acks.ALL**, **timeoutMillis=0**, error=REQUEST_TIMEOUT; replication=(1/0/1,2,3/1,0,0/0), physical=(1,0,0), position=0, committed=none, visible=[], effects=[], receipt=none. The append is not rolled back.
- **LATE_VISIBLE:** later perform real replication and report all three replicas. New state: replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1); position=1 after poll, committed=none, visible=[offset0], effects=[], receipt=none, error=NONE. A later HW advance does not create or resend the original receipt.

Experiment 4: a failed commit replays an effect. The test record is key="order-87" (8 bytes), value="effect-after-poll" (17 bytes), timestamp=4003, for 57 bytes. The Kafka record is already replicated at offset0 with HW=1; the consumer group has no committed offset. The proxy rejects the first **COMMIT_OFFSET** before forwarding it.

- **COMMIT_FAILED:** first consumer resumes at0, polls and invokes the callback, then has position=1; error=REQUEST_TIMEOUT; replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1), committed=none, visible=[offset0], effects=[0], receipt=none. Proxy counts: rejected(**COMMIT_OFFSET**)=1 and forwarded(**COMMIT_OFFSET**)=0.
- **REPLAY_COMMITTED:** a new consumer resumes at0 because no stored offset exists, calls the callback again, and commits nextOffset=1; error=NONE, replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1), position=1, committed=1, visible=[offset0], effects=[0,0], receipt=none. The external work repeated; Kafka did not append another record.

Experiment 5: stale group side effect. The test record is key="inventory-13" (12 bytes), value="old-owner-effect" (16 bytes), timestamp=5007, for 60 bytes; it has already replicated and is visible at offset0.

- Member z owns the partition at generation1, fetches offset0, and performs its callback. Member a joins; generation2 assigns the partition to a. z's old-token commit and fetch both return `ILLEGAL_GENERATION`. Observation phase=`STALE_COMMIT_REJECTED`, error=`ILLEGAL_GENERATION`, replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1), position=1, committed=none, visible=[offset0], effects=[0], receipt=none.
- a fetches the same offset0 because no offset was committed, processes it, and commits nextOffset1. Phase=`NEW_OWNER_COMMITTED`, error=`NONE`; replication/LEO/HW unchanged, position=1, committed=1, visible=[offset0], effects=[0,0], receipt=none. Generation fencing rejects the old member's broker requests but cannot undo z's callback.

Tiered hints (not the answer)

Level 1

Distinguish physical LEO, authority-reported LEO, HW, client position, and durable committed nextOffset; they are separate states.

Level 2

For each failure, order append, replication, callback, commit, and error instead of treating success in one layer as success in another.

Level 3

Compare each real snapshot/trace, then independently check the caller-owned FaultProxy counters and EffectRecorder list; do not manufacture a race with sleep.

Observable examples and test points. Use `UNKNOWN-POISONED-EXPLICIT_RESEND` to identify one real append, one local rejection, and one explicit resend. Then trace leader election, ALL timeout, commit rejection, and generation change. A `LEADER receipt=[0,1]` can coexist with HW0 and no visibility; a rejected commit can coexist with a completed callback; and the new group owner can invoke that callback again. The task is to explain observed state, not to assume every phase succeeded.

Tests and completion. `Step29Test.producerOutcomeUnknownAndExplicitResendAreObserved` checks the dropped reply, poisoned producer, and duplicate offsets; `leaderAcknowledgmentDoesNotProtectAnUncommittedTail` checks clean election, tail removal, and admission; `allAcknowledgmentTimeoutCanBecomeVisibleLater` checks visibility without a receipt; `consumerCommitFailureReplaysEffectAndCommitsOnFreshConnection` checks commit counters, position, offset, and callback replay; `staleMemberSideEffectSurvivesGenerationFencing` checks both stale-token errors and the surviving effect. Other tests cover single-fault proxy behavior, stamped input, nonempty logs, and invalid initial state without mutation. The single-step command isolates Step29; the cumulative command runs Steps1-29.

Common pitfalls and differences from real Kafka

A receipt reports one ACK result; a `LEADER`-acknowledged tail may still be lost in election. A timeout does not prove there was no append. Position advances before commit; generation fencing constrains broker operations, not external callbacks. These experiments do not retry ordinary `SimpleProducer` and do not claim end-to-end transactions or exactly-once.

Commands and progression. Use `./gradlew :stepTest -Pstep=29` to diagnose this step; `./gradlew :test -Pstep=29` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Which observations distinguish a LEADER acknowledgment from visibility after replication commits?
2. Why can a stale member's commit be rejected even though its side effect already happened?

Self-check answers. Check receipt, physical/reported LEO, HW, and visible records: a LEADER receipt can precede replication and visibility. Callback and commit are separate actions; commit failure says progress was not persisted and does not undo a completed effect.

9.2 Step 30: Idempotent Producer**Goals and prerequisites**

Goal: Persist producer batch identity so an explicit retry of the same batch returns its original offset range without another append.

Before you begin: Start with basic identity concepts and connect stamped records, log recovery, replication acknowledgment, routing, and explicit client retry.

Background and reasoning

Prerequisites: [Step 29 \(Cross-Layer Experiments\)](#), plus [Step 1 \(Record Format\)](#), [Step 6 \(Segmented Log\)](#), and [Step 24 \(Replicated Writes\)](#). A network error can leave a producer unsure whether the broker appended; without a durable identity, sending again creates a new record. Idempotent append assigns a stable identity to a logical batch so the broker can map an exact retry to its first offsets. Do not confuse four numbers: leader epoch is the broker role term; producer epoch is a caller-managed producer session term; firstSequence is that producer's next batch sequence for this partition; Kafka offset is the record's location in the partition log. eventId identifies a business event and is used in Step31 to deduplicate across offsets.

Starting point and edit location.

Already provided: The provided layer supplies `ProducerStamp` (`provided/src/main/java/io/simplekafka/model/ProducerStamp.java`) and `RecordData` (`provided/src/main/java/io/simplekafka/model/RecordData.java`), `ProducerBatchFingerprint.fingerprint(List<RecordData>):byte[]` (`provided/src/main/java/io/simplekafka/protocol/ProducerBatchFingerprint.java`), `ProducerBatchSupport.validateAppend(ProducerStamp,List<RecordData>):void` (`provided/src/main/java/io/simplekafka/storage/ProducerBatchSupport.java`), `Messages.IdempotentProduceRequest`, `MessageCodec.encodeRequest(Messages.Request):byte[]`, and `decodeRequest(short,byte[]):Messages.Request` (`provided/src/main/java/io/simplekafka/protocol/Messages.java` and `provided/src/main/java/io/simplekafka/protocol/MessageCodec.java`), plus `IdempotentProduceBackend.produceIdempotent(List<RecordData>,Acks,int,long,long,int,long):AppendResult` (`provided/src/main/java/io/simplekafka/broker/IdempotentProduceBackend.java`). The `IdempotentProducer(MetadataRouter,long,int)` constructor/`close()` and `IdempotentAppender(PartitionLog)` constructor are already in their exercise classes; appender construction does not automatically recover.

Implement in: Complete these six dependent tasks in order; editable paths and method signatures are listed below.

1. **Identity and formats.** Provided `ProducerStamp(longproducerId,intproducerEpoch,longfirstSequence,intbatchSize,intbatchIndex,byte[]batchHash)` stores producer id, producer epoch, first batch sequence, batch size, zero-based record index, and a 32-byte SHA-256 hash. IDs/epoch/sequence are nonnegative; batchSize>0; index is in [0,batchSize);

- sequence+size must not overflow. Hash input is big-endian count:int32, then per record timestamp:int64|keyLength:int32(-1=null,0=empty)|key|valueLength:int32|value; offsets and leader epochs are excluded. Ordinary record bytes keep their old format; a stamped record adds 64 bytes. Disk layout is length|crc32c|offset|timestamp|marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32|keyLength:int32|valueLength:int32|key|value; CRC covers offset through value. Ordinary minimum total is 32 bytes, stamped minimum 96; the length field limit is 1,048,576, and total encoding is at most 1,048,580 bytes (including the separate 4-byte prefix). With null key, stamped value maximum is 1,048,484 bytes; key and value both count. The teaching wire stamp layout differs from disk: marker, identity, hash, keyLength/key, valueLength/value, timestamp.
2. **Record codec and segment log.** Extend `exercises/src/main/java/io/simplekafka/storage/RecordCodec.java`: `public static ByteBuffer encode(LogRecord)` and `public static LogRecord decode(ByteBuffer)`. Preserve ordinary bytes, CRC behavior, and the existing buffer contract; use the marker to distinguish stamped records. Extend `exercises/src/main/java/io/simplekafka/storage/SegmentLog.java`: `public AppendResult append(List<RecordData>)` and `public long recover()` to maintain/rebuild the tail `ProducerStamp` and validate record structure/CRC on write and recovery. The supplied `void truncateToOffset(long)` is a `SegmentLog` tail-truncation operation, not a new `PartitionLog` API.
 3. **Partition log and prefix versions.** Extend `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`: `public synchronized AppendResult append(List<RecordData>)`, `public synchronized long deleteBefore(long offset)`, and `public synchronized void truncateTo(long nextOffset)`; provided accessors are `long mutationVersion()`, `ProducerStamp tailProducerStamp()`, and `long prefixVersion()`. `append` requires a nonempty valid batch, assigns consecutive offsets, validates the batch before writing, and updates the tail stamp; `tailProducerStamp()` is null when the tail has no stamp. `deleteBefore` removes only closed nonactive segments whose `endOffset <= offset`, keeps at least one segment, rejects negative offset with `INVALID_REQUEST`, and returns actual `logStart`. `truncateTo` accepts `[logStart, LEO]`, returns `OFFSET_OUT_OF_RANGE` outside it, and is a no-op at LEO. Mark an effective mutation once before its first I/O; an actual prefix rewrite or retention deletion increments `prefixVersion`, while a no-op does not invalidate caches. I/O failure may already have changed disk; follow the log's failed/reopen behavior. Do not describe `SegmentLog`'s `truncateToOffset(long)` as a `PartitionLog` method.
 4. **Log-derived batch deduplication.** Implement `exercises/src/main/java/io/simplekafka/replication/IdempotentAppender.java`: `public AppendResult append(long producerId, int producerEpoch, long firstSequence, List<RecordData> records)` and `public void recover()`. Its supplied constructor is `IdempotentAppender(PartitionLog)`. `append` accepts ordinary unstamped input, validates the complete batch before writing, and `recover` derives producer epoch, next sequence, completed ranges, and hashes from retained records rather than treating memory cache as durable truth. A log start above 0 returns `PRODUCER_STATE_EXPIRED`; malformed complete history, sequence, or hash returns `CORRUPT_RECORD`.
 5. **Replicated write and API 11.** Extend `exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java`: `public AppendResult produceIdempotent(List<RecordData> records, AckSacks, int leaderEpoch, long timeoutMillis, long producerId, int producerEpoch, long firstSequence)`. Extend `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`: `public Messages.Reply handle(Messages.Request request)` to dispatch API `IDEMPOTENT_PRODUCE=11`; success uses `ProduceBody(AppendResult)`. Extend `exercises/src/main/java/io/simplekafka/client/MetadataRouter.java`: `public Messages.Reply call(TopicPartition tp, Messages.Request request)` to route API 11. The provided request record signature is `IdempotentProduceRequest(Topi`

`cPartitiontp, intepoch, Acksacks, longtimeoutMillis, longproducerId, intproduce
rEpoch, longfirstSequence, List<RecordData>records`); records are ordered unstamped
input. A single-broker backend lacks this capability and must return `INVALID_REQUEST`
without appending.

6. **Client and explicit retry.** Implement `exercises/src/main/java/io/simplekafka/c
lient/IdempotentProducer.java`: `publicProduceReceiptsend(TopicPartitiontp, Lis
t<RecordData>records, Acksacks, longtimeoutMillis)` and `publicProduceReceiptre
try(TopicPartitiontp)`. `IdempotentProducer(MetadataRouterrouter, longproduce
rId, intproducerEpoch)` and `close()` are supplied. Maintain `nextSequence` and at most
one pending batch per partition; failure retains its frozen identity/data, and an explicit retry
sends only one RPC.

Errors, state, and side effects. Negative producer id/epoch/sequence, null topic-partition/acks, null/empty batch, null record, stamped input, sequence addition overflow, a record over its limit, negative timeout, or an oversized frame is `INVALID_REQUEST`; reject before append without consuming sequence. A new producer or higher producer epoch starts at sequence0; a higher epoch becomes durable only with a persisted stamped record. Current epoch accepts only exact `nextSequence`; an older epoch is `FENCED_PRODUCER_EPOCH`, a gap or unrecorded old sequence is `OUT_OF_ORDER_SEQUENCE`. For the same completed producerId/producerEpoch/firstSequence, a changed batchSize is `OUT_OF_ORDER_SEQUENCE`; with the same batchSize, a different batch-Hash is `DUPLICATE_SEQUENCE_CONFLICT`. An exact full duplicate with the same identity, boundary, and hash returns its original half-open `[firstOffset, nextOffset)` range with no append or physical-LEO change. The replicated backend still checks current leader epoch/min-ISR and performs the full ALL HW wait. If only a matching tail-batch prefix is durable, validate it and append only the missing suffix, returning the original full range; another producer/batch receives `INCOMPLETE_BATCH`, while the same identity with changed payload receives `DUPLICATE_SEQUENCE_CONFLICT`. A retained start above0 returns `PRODUCER_STATE_EXPIRED`.

Teaching frame bound. API11 uses the course TCP frame from Step11: the length field is in `[10, 8, 388, 608]` bytes and excludes its 4-byte prefix; payload is at most 8,388,598 bytes, total frame at most 8,388,612 bytes. Reject a batch request over that bound before sending; the stamped-record bound is independently 1,048,580 bytes total.

Derived state and call boundaries. Producer-state cache is derived from stamped log records. Scan new suffixes incrementally; when `prefixVersion` changes or LEO moves backward, discard the cache and rebuild from `offset0`. Recovery reads one complete record at a time with `read(offset, 1, 1_048_580)`, verifying each producer sequence, batch index, and batch hash. Only a final batch may be incomplete; recovery can retain it for matching continuation, while complete invalid history is rejected without rewriting the log. If an uncertain I/O error occurs after storage mutation, do not treat memory state as durable; reopen the failed log and recover. Leader epoch differs from producer epoch: after leadership changes, the same producer identity can retry with its original producer epoch.

Client boundaries. The `IdempotentProducer(router, producerId, producerEpoch)` constructor requires nonnegative identity; negative id/epoch is `INVALID_REQUEST`. On an open producer, `send` also rejects null tp/records/acks, empty batches, null records, negative timeout, stamped input, record/frame overflow, or sequence overflow. Metadata failure before the data RPC is not an unknown append. Once the request could reach the broker, failure leaves a per-partition pending batch; another send to that partition is `INVALID_REQUEST`, while other partitions are independent. Only a successful receipt whose range length equals the batch count

advances nextSequence and clears pending state. `retry(tp)` without pending state is `INVALID_REQUEST`; otherwise it explicitly refreshes metadata and sends once to the new route with the same identity, firstSequence, frozen records, ACK mode, and timeout. Refresh/RPC failure or invalid receipt preserves pending state. `close` clears local pending state but not its borrowed router; `send/retry` on a closed client throws `IllegalStateException`.

Behavioral contract and invariants. Only a repeated batch with the same producer identity, batch boundary, and content returns the original durable range; never fabricate a receipt for a failed path. Replication errors do not roll back a stamped record already appended.

Tiered hints (not the answer)

Level 1

First distinguish leader epoch, producer epoch, batch sequence, Kafka offset, and business eventId; they cannot substitute for one another.

Level 2

Treat the durable log as truth and producer-state cache as a derived result tagged by prefixVersion; explain how ordinary append, retention, and truncation affect it.

Level 3

Implement and inspect disk in the six-task order: verify an exact duplicate leaves bytes unchanged, then a partial batch adds only its suffix, then drop an API11 reply, change leader, and verify one durable batch after explicit retry.

Observable examples and test points. Assume an unretained partition, leader broker1/epoch0, producerId=7, producerEpoch=0, firstSequence=0. The batch is three ASCII records ("retry-0", "value-zero", 1001), ("retry-1", "value-one", 1002), and ("retry-2", "value-two", 1003). Their key/value lengths are (7,10), (7,9), and (7,9) bytes; ordinary records occupy 49,48,48 bytes, and stamped disk records occupy 113,112,112 bytes. batchIndex is 0/1/2, batchSize=3, and all share one hash. Call `send(tp, batch, Acks.LEADER, 5_000)`; if the API11 reply is lost, the client retains pending state and leader physical LEO=3. After replicating and reporting all replicas, HW=3. Broker1 stops and broker2 becomes leader/epoch1; explicit `retry(tp)` retains the original LEADER ACK and producerEpoch0/sequence0, returning the original [0,3) with three records in each log. Reopen broker2's disk: recovery still recognizes the request and returns [0,3) without changing bytes or LEO. If the duplicate instead uses `Acks.ALL`, broker2's `ISR=2 < minISR=2`, so it returns `NOT_ENOUGH_REPLICAS` before append; `allAckDuplicateWaitsForHwAndStillRequiresMinIsrAndCurrentEpoch` covers this boundary. Contrast a three-record batch whose offsets 0–1 alone were persisted: an ordinary new request gets `INCOMPLETE_BATCH`; resubmitting the exact original identity/size/hash/payload writes only offset 2 and returns [0,3).

Tests and completion. `Step30Test.producerRejectsInvalidBatchesBeforeChangingSequenceOrLogs` and `sequenceErrorsDoNotAppendOrInstallAnUnpersistedEpoch` cover validation and side-effect-free epoch/sequence errors; `stampedRecordDiskBytesAreIndependentAndStructurallyChecked` checks disk layout, CRC, and bounds; `incompleteBatchSurvivesRecoveryAndCanOnlyBeCompletedByItsOriginalRequest`, `sameLeoRewriteCannotReuseOldProducerReceipt`, `recoveryRejectsCompleteBatchWithIncorrectFingerprint`, and `retainPrefixCannotServeCachedOrRecoveredProducerReceipts` cover recovery, partial batches, and prefix invalidation; `singleBrokerRejectsIdempotentProduceWithoutAppending` and `idempotentApiAndStampedFetchWireFormatsRoundTrip` cover API behavior; `allAckDuplicateWaitsForHwAndStillRequiresMinIsrAndCurrentEpoch`, `lostResponseRetryAndDiskReopenPreserveOneDurableBatch`, `boundedReplicaFetchAndDiskReopenPreserveStampedSegments`, and `reconciliationComparesProducerStampsAtCommittedAndUncommittedOffsets` cover ACK, real TCP, disk reopen, and repair. The single-step command isolates Step30; cumulative runs Steps1–30, but Chapter 8 completion still requires Step31.

Common pitfalls and differences from real Kafka

Producer retry deduplicates a partition batch by sequence, not business events at different offsets. This does not allocate producer IDs, implement Kafka PID/session semantics, make a transactional producer, or guarantee exactly-once business effects. Ordinary `SimpleProducer` still does not retry; only a caller explicitly invokes `retry` on the new client.

Commands and progression. Use `./gradlew :stepTest -Pstep=30` to diagnose this step; `./gradlew :test -Pstep=30` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must a completed duplicate still check current epoch, minISR, and HW?
2. Why can producer sequence or an old receipt not be guessed after retention deletes the log prefix?

Self-check answers. A duplicate needs no new append, but ALL still promises that the current leader and replicas satisfy the condition; replication state can change after election or ISR changes. A removed prefix may have contained the producer's first epoch, sequence, and original offset range, so without that durable evidence the broker cannot safely distinguish a new write from a duplicate.

9.3 Step 31: Idempotent Consumer Effects**Goals and prerequisites**

Goal: Apply each stable business event to a durable ledger once even when its Kafka record is delivered repeatedly.

Before you begin: Explain why offset deduplication and `eventId` deduplication differ, then implement a recoverable local ledger.

Background and reasoning

Prerequisites: [Step 30 \(Idempotent Producer\)](#), plus [Step 16 \(Process Then Commit\)](#) and [Step 29 \(Replayed Side Effects\)](#). A Kafka offset identifies one log record's position, not a business event: the same event can be sent to another offset, and a callback can repeat after commit failure. A stable `eventId` comes from business data; if the same event keeps the same ID through replay and restart, the ledger can use it as an application-level deduplication key. This task coordinates one local forced log and its in-memory balances; it does not share a transaction with Kafka's offset store.

Starting point and edit location.

Already provided: The provided layer supplies `BusinessEvent(String eventId, String account, long delta)` (`provided/src/main/java/io/simplekafka/model/BusinessEvent.java`) and `MessageCodec.encodeBusinessEvent(BusinessEvent): byte[]` / `decodeBusinessEvent(byte[]): BusinessEvent` (`provided/src/main/java/io/simplekafka/protocol/MessageCodec.java`).

Implement in: `exercises/src/main/java/io/simplekafka/client/DurableEffectStore.java`: the existing skeleton's `publicDurableEffectStore(Path directory)` opens a `PartitionLog` and invokes `recovery` first; implement `public synchronized boolean apply(Bu`

usinessEventevent) and public synchronized void recover(). Existing API methods are public synchronized long balance(String account) and public synchronized void close().

Business event and durable format. `BusinessEvent` requires `eventId` and `account` to be non-empty strict UTF-8 strings of at most 255 bytes each; `delta` is any signed long, including positive, negative, or zero. Its value format is `version:uint8=1|eventId:length-prefixedidentifier|account:length-prefixedidentifier|delta:int64`; unknown version, truncated input, invalid UTF-8/identifier, or trailing bytes returns `INVALID_REQUEST` at the codec boundary. The provided `BusinessEvent` constructor rejects an invalid identifier with `IllegalArgumentException`. The store writes one `RecordData(null, encodedEvent, 0)` per event: null key, timestamp 0, no producer stamp. Ordinary record overhead is 32 fixed bytes.

Behavioral contract and invariants. `apply` on a new valid `eventId` first computes the account balance with checked addition, then force-appends one record, then publishes the in-memory seen/balance state; it returns true. Replaying the exact same `eventId/account/delta` returns false and changes neither log nor balance. Reusing the ID with another `account/delta` is `INVALID_REQUEST`; balance overflow is also `INVALID_REQUEST`; neither appends.

- On a usable store with a complete log, `apply(null)` returns `INVALID_REQUEST`; an invalid `eventId/account` cannot form a valid `BusinessEvent` (the provided constructor throws `IllegalArgumentException`). `balance(account)` returns 0 for an unseen account; null, empty, invalid-Unicode, or over-255-byte account input returns `INVALID_REQUEST`.
- Calls to `apply/balance/recover/close` on one store instance are synchronized. Construction and explicit `recover()` rebuild identity and balances from `offset0` in order, reading one complete record per page (`maxRecords=1, maxBytes=1_048_580`); require `logStart=0`, null key, timestamp 0, and no stamp. Complete invalid payload, invalid record fields, conflicting historical ID, balance overflow during replay, or scan without progress returns `CORRUPT_RECORD`; complete corrupt history is not rewritten.
- If the physical tail contains only an incomplete record, `PartitionLog/SegmentLog` recovery truncates it to the last complete record before replay. A complete record with bad CRC/structure/business encoding is not treated as a partial tail. If identical events already repeat in history, recovery counts their balance once and leaves the existing log records intact.
- If append I/O or its returned range is uncertain, do not publish a new in-memory balance and mark the instance failed. A truncated ledger (`logStart>0`) returns `CORRUPT_RECORD` and fails the store. After failure or close, `apply/balance/recover` return `STORAGE_ERROR`; close and construct a new instance to recover from disk. close is idempotent.

Tiered hints (not the answer)

Level 1

Deduplicate with a business `eventId`, not a Kafka offset; one replayed event may have a different offset but the same `eventId`.

Level 2

Check event-ID conflict and checked balance before writing; update in-memory seen/balance only after the forced append succeeds.

Level 3

Apply an identical event twice and verify the second call does not grow the log; close and reopen, replay history, and compare balances, deduplication result, and disk bytes.

Observable examples and test points. Assume one partition's input log carries A, A, B at offsets 0, 1, 2 in that order, each as a null-key, timestamp 0 record whose value is the encoded

business event. `A=BusinessEvent("event-A", "acct", +10)`, the duplicate A has exactly the same `eventId/account/delta`, and `B=BusinessEvent("event-B", "acct", -3)`. ASCII event IDs are 7 bytes and account is 4; each encoded value is $1 + 4 + 7 + 4 + 4 + 8 = 28$ bytes, so each ordinary ledger record occupies $32 + 28 = 60$ bytes. `apply` returns `true, false, true`; the ledger has only `[A,B]` (120 bytes) and balance is 7. If the first callback processes all three records but commit fails, callback history is `[0,1,2]` while committed remains empty. A new consumer replays from 0 and commits 3; history becomes `[0,1,2,0,1,2]`, ledger remains `[A,B]`, balance remains 7, and committed `nextOffset=3`. Kafka offset did not become the `eventId`, and the two systems had no shared transaction.

Tests and completion. `Step31Test.appliesAndRecoversDurableBusinessEffects` checks first/duplicate apply, balance, and reopen; `rejectsConflictsInvalidIdentifiersAndOverflowWithoutAppending` checks conflicts, UTF-8 boundaries, and overflow; `businessEventCodecRejectsMalformedPayloads` checks value encoding; `concurrentDuplicateApplicationsAppendOnceAndRecoverOnce` checks synchronization; `recoversRepeatedHistoryAndTruncatesIncompleteFinalRecord` and `rejectsCorruptCompleteHistoryWithoutChangingItsLog` distinguish a partial tail from complete corruption; `consumerCommitFailureReplaysCallbacksButNotDurableBusinessEffects` checks real TCP commit failure, six callbacks, two ledger entries, balance 7, and committed=3. The single-step command isolates Step 31; the cumulative command runs Steps 1–31, the Chapter 8 completion boundary.

Common pitfalls and differences from real Kafka

A forced ledger append coordinates only one local store's log and in-memory publication; it does not form a distributed transaction with the producer, Kafka log, or consumer offset. The same event can appear at different Kafka offsets; a stale group token can reject commit but cannot roll back the ledger. This is application-level local deduplication, not a global transaction or an end-to-end exactly-once guarantee.

Commands and progression. Use `./gradlew :stepTest -Pstep=31` to diagnose this step; `./gradlew :test -Pstep=31` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is stable `eventId` a better business deduplication key than Kafka offset or callback count?
2. After the ledger append succeeds but consumer offset commit fails, which states are still uncoordinated?

Self-check answers. Offset identifies a log position and callback count records attempts; neither identifies one business event across positions. The ledger effect is durable while the commit is not, so callbacks repeat, but `eventId` makes the ledger return false and prevents another balance change. The consumption position and ledger still were not committed atomically.

Appendix A

Formats, Error Codes, and Interface Index

This appendix gathers stable conventions used repeatedly across chapters for convenient reference. If anything here differs from a public signature in an exercise, follow the course manifest and Java types. Layouts and error codes defined by this course belong only to the simpleKafka teaching interface; unless an official source is explicitly cited, do not describe them as Kafka wire formats, storage formats, or configuration defaults.

A.1 Disk and Network Formats

A.1.1 Records and Indexes

Records use big-endian encoding:

Field	Width	Semantics
length	int32	Record length excluding itself and including CRC; 28 to 1,048,576.
crc32c	int32	Checks offset through the end of value, excluding the length/CRC fields themselves.
offset	int64	Nonnegative logical offset.
timestamp	int64	Nonnegative timestamp.
keyLength	int32	-1 means null, 0 means a present empty key, otherwise the number of bytes.
valueLength	int32	Nonnegative number of bytes; value is not null.
key/value	variable	Variable-length fields follow in the order described above.

The fixed-length part of each record (excluding the length prefix) contains the CRC, offset, timestamp, and two lengths, for a total of 28 bytes; the length prefix makes the minimum total size 32 bytes. The lower bound for the length field remains 28 because its count starts at the CRC. Each segment's sparse-index entry is `offset:int64,position:int64`, totaling 16 bytes; one index point is written for every intervalRecords records, and the first record always has a point. The index may be discarded and rebuilt from the log. Ordinary records retain this layout. A stamped record inserts the following 64-byte producer identity after timestamp and before keyLength: `marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32bytes`. Its full layout is `length|crc32c|offset|timestamp|marker|producerId|producerEpoch|firstSequence|b`

atchSize|batchIndex|batchHash|keyLength|valueLength|key|value. The minimum total size is 96 bytes, and the 1 MiB maximum includes the stamp. CRC32C covers offset through the final value byte, including the stamp. Ordinary record bytes are unchanged. The supplied `ProducerStamp.ENCODED_OVERHEAD` is 64 bytes and is included in the 1 MiB record bound for both disk and `MessageCodec` wire records.

The directory convention is `<root>/<topic>/<partition>/<baseOffset:020d>.log`, with the paired index suffix `.index`. The retained range is `[logStartOffset,LEO)`, and fetch is left-closed/right-open. LEO is the next writable offset; HW is the highest committed boundary visible to ordinary consumers (only offsets `< HW` are visible). A group committed offset is also a next offset, but is not the same as HW.

A.1.2 TCP Frames and Request Bodies

The teaching frame is `length:int32|version:int16(1)|api:int16|correlationId:int32|error:int16|payload;` length excludes itself and is in the range `[10,8,388,608]`. EOF before the first byte of a new frame returns null; EOF within a frame reports `INVALID_REQUEST`. The request error field must be 0. Unknown versions/APIs, oversized frames, and trailing body bytes are all rejected. The maximum size of an individual disk record remains 1 MiB and is not changed by the frame limit.

API ids are fixed as follows:

ID	API	ID	API
1	METADATA	6	JOIN_GROUP
2	PRODUCE	7	HEARTBEAT
3	FETCH	8	LEAVE_GROUP
4	COMMIT_OFFSET	9	GROUP_ASSIGNMENT
5	FETCH_OFFSET	10	REPLICA_FETCH
11	IDEMPOTENT_PRODUCE		

`MessageCodec` uses big-endian primitive fields; strings are an int32 UTF-8 byte length followed by bytes, collections are an int32 count followed by each item, Optional values use a boolean followed by the value, and `byte[]` uses an int32 length (only key may be -1). Request and successful-response fields are encoded in the declaration order of the provided Messages nested records; map results are sorted by key. When the reply header has `error=NONE`, the body is the corresponding success record; on error, the body is `ErrorBody(message)`, and the error code appears only in the frame header. Field schemas are fixed by `provided/.../protocol/Messages.java`; students do not need to redefine the eleven message formats. A stamped `RecordData` on `FETCH` or `REPLICA_FETCH` wire uses `marker=-2|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash[32]|keyLength|key|valueLength|value|timestamp`; an ordinary record's wire encoding is unchanged.

A.1.3 Message Body Field Order

The requests and responses below are encoded by `MessageCodec` in the order of the declared parameters in each record. Collection lengths and string/byte lengths are all big-endian int32; list items have no extra tag and are inlined in the declared order for each value type. `GroupToken` is nullable, encoded as a boolean present followed by group, member, and generation; `Acks` is int16, with `LEADER=1` and `ALL=-1`.

API	Request record and fields	Successful-response record and fields
METADATA	MetadataRequest: topic	MetadataBody: partitions

API	Request record and fields	Successful-response record and fields
PRODUCE	ProduceRequest: tp, epoch, acks, timeoutMillis, records	ProduceBody: result
IDEMPOTENT_PRODUCE	IdempotentProduceRequest: tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records	ProduceBody: result
FETCH	FetchRequest: tp, epoch, offset, maxRecords, maxBytes, token	FetchBody: records, logStartOffset, logEndOffset, highWatermark, epoch
COMMIT_OFFSET	CommitOffsetRequest: key, nextOffset, token	EmptyBody
FETCH_OFFSET	FetchOffsetRequest: key	OffsetBody: nextOffset (OptionalLong)
JOIN_GROUP	JoinGroupRequest: group, member, topic	GroupBody: assignment
HEARTBEAT	HeartbeatRequest: token	EmptyBody
LEAVE_GROUP	LeaveGroupRequest: group, member	GroupBody: assignment
GROUP_ASSIGNMENT	GroupAssignmentRequest: group, member	GroupBody: assignment
REPLICA_FETCH	ReplicaFetchRequest: tp, brokerId, epoch, fetchOffset, maxRecords, maxBytes, recoveryRead	ReplicaFetchBody: records, epoch, highWatermark, leaderLogEndOffset

Record	Fields in declaration order
TopicPartition	topic, partition
RecordData	key, value, timestamp, nullable producerStamp
LogRecord	offset, data
AppendResult	firstOffset, nextOffset
OffsetKey	group, tp
Endpoint	host, port
GroupAssignment	generation, assignments
PartitionMetadata	tp, leaderId, epoch, replicas, isr, leader
ProducerStamp	producerId, producerEpoch, firstSequence, batchSize, batchIndex, batchHash
BusinessEvent	eventId, account, delta

Each byte array is length-prefixed; only key may be null (length -1), while value is always present but may be empty.

OffsetStore entries use `MessageCodec.encodeOffsetEntry`, with null key and timestamp 0. The record's physical log offset is independent of committed nextOffset. Business-event values use `MessageCodec.encodeBusinessEvent: version:uint8=1|eventId:identifier|account:identifier|delta:int64`. Both identifiers are non-empty strict UTF-8 strings of at most 255 bytes; delta may be negative, zero, or positive.

A.1.4 Identifiers and Value Types

topic, group, and member are non-empty UTF-8 identifiers with a maximum length of 255 bytes; topic may contain only ASCII letters, digits, periods, underscores, and hyphens, and . and .. are rejected. TopicPartition's partition is nonnegative. RecordData's key may be null, value may not be null (but may be an empty array), and timestamp is nonnegative; LogRecord offset is nonnegative. Once array ownership is transferred to a value object, the receiver owns it, so it need not be copied repeatedly. All offsets that denote the next item use half-open boundaries; see the relevant chapters for the details.

A.2 Idempotent Append and Durable Effects

API id 11 is the separate `IDEMPOTENT_PRODUCE` request; ordinary `PRODUCE` remains non-idempotent and rejects stamped client input. The request carries `tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records`; the broker stamps each record with the request's batch identity and a streamed SHA-256 fingerprint of the record count and timestamp/key/value contents. The caller manages producer id and epoch. A new producer epoch starts at sequence zero and takes effect only when a record is persisted. An exact completed duplicate returns its original offset range without appending; same identity with different content is a conflict; sequence gaps and unknown old sequences are rejected. A matching partial batch can be completed by appending only its missing suffix. Derived state is recovered from retained log history; when `logStartOffset` is nonzero, idempotent requests fail with `PRODUCER_STATE_EXPIRED`. There is no producer-state snapshot or client session WAL.

`IdempotentProducer` sends once per call. A failed call retains its pending batch; only the caller's explicit `retry` refreshes metadata and sends that same identity, sequence, and payload once. Duplicate requests still pass the normal ACK precheck and wait, including ALL's ISR/H-W/deadline rules. This does not change `SimpleProducer`, which never retries automatically.

`DurableEffectStore` applies one `BusinessEvent` as one forced local log record. The first event id appends and updates its account balance; an identical id/content replay returns false without another append; an id reused with different content or a checked-balance overflow is rejected. Recovery rebuilds event identities and balances from the retained log and requires log start zero. This local ledger is not atomically coordinated with `ProcessingLoop`'s offset commit: the callback may run again after a commit failure, while a stable event id prevents a duplicate local accounting effect. Group fencing cannot undo an effect that already happened.

A.3 Fixed Error Codes

Wire ID	Name	Purpose
0	NONE	Success.
1	INVALID_REQUEST	Invalid parameter, length, version, or request format.
2	UNKNOWN_TOPIC_OR_PARTITION	Topic/partition does not exist.
3	OFFSET_OUT_OF_RANGE	Offset violates the current log start/end constraints.
4	CORRUPT_RECORD	Complete-record CRC/format error or conflict with committed history.
5	NOT_LEADER	The requester is not the current leader.
6	FENCED_EPOCH	The request uses an old leader epoch.
7	NOT_ENOUGH_REPLICAS	The current ISR has fewer than minISR members.
8	REQUEST_TIMEOUT	The request deadline expired; the outcome may be unknown.
9	ILLEGAL_GENERATION	The consumer-group generation has expired.
10	UNKNOWN_MEMBER	The consumer-group member is not registered.
11	NOT_ASSIGNED	The operation involves an unassigned partition.
12	NO_ELIGIBLE_LEADER	No candidate satisfies the clean-election conditions.
13	STORAGE_ERROR	I/O failure; the underlying cause is retained.
14	FENCED_PRODUCER_EPOCH	The request uses an older producer epoch.
15	OUT_OF_ORDER_SEQUENCE	The batch sequence or boundary is invalid.
16	DUPLICATE_SEQUENCE_CONFLICT	A repeated batch identity has different content.

Wire ID	Name	Purpose
17	PRODUCER_STATE_EXPIRED	Required producer history is outside the retained log prefix.
18	INCOMPLETE_BATCH	An unrelated append would cross an incomplete producer batch.

An unknown ID is not mapped to success; I/O errors do not swallow the cause; network-error bodies do not return the server stack trace.

A.4 The 31 Steps and Edit Entry Points

All paths are relative to the repository root. Students modify the specified methods under `exercises/src/main/java/io/simplekafka/`. The cumulative command for each step is `./gradlew :test -Pstep=N`; the single-step diagnostic is `./gradlew :stepTest -Pstep=N`, which does not prove progression.

Step	Course title	Edit methods
1	Record encoding and decoding	storage/RecordCodec: encode, decode
2	Log append	storage/SegmentLog: append
3	Offset-based reads	storage/SegmentLog: readFrom
4	Crash recovery	storage/SegmentLog: recover
5	Sparse index	storage/SparseIndex: add, floor, rebuild
6	Segmented log	storage/PartitionLog: append, read
7	Log retention	storage/PartitionLog: deleteBefore
8	Tail truncation	storage/PartitionLog: truncateTo
9	Partition routing	client/Partitioner: choose
10	Topic metadata	broker/PartitionCatalog: createTopic, metadata
11	TCP framing	protocol/FrameCodec: write, read
12	Single-broker requests	broker/BrokerHandler: handle
13	Batched producer	client/SimpleProducer: send, flush
14	Pull-based consumer	client/SimpleConsumer: assign, seek, poll
15	Consumer persistence	group/OffsetStore: commit, fetch; client/SimpleConsumer: resume, commitSync; broker/BrokerHandler: COMMIT_OFFSET, FETCH_OFFSET
16	At-least-once processing	client/ProcessingLoop: runOnce
17	Partition assignment	group/RoundRobinAssignor: assign
18	Membership and rebalancing	group/GroupCoordinator: join, leave, assignment; broker/BrokerHandler: JOIN_GROUP, LEAVE_GROUP, GROUP_ASSIGNMENT
19	Heartbeats and expiration	group/GroupCoordinator: heartbeat, expire; broker/BrokerHandler: HEARTBEAT
20	Consumer fencing	group/GroupCoordinator: validate; client/GroupConsumer: subscribe, poll, commitSync; broker/BrokerHandler: FETCH/COMMIT_OFFSET token fencing

Step	Course title	Edit methods
21	Fetch-based replication	broker/BrokerHandler: handle(REPLICA_FETCH); replication/FollowerReplicator: pollOnce
22	ISR and high watermark	replication/ReplicationTracker: report, expireLagging, advanceHighWatermark
23	Produce acknowledgments	replication/AckPolicy: validateBeforeAppend, await
24	Concurrent replicated broker	replication/ReplicatedPartition: produce, fetch
25	Clean election and leader epochs	replication/LeaderElection: elect, checkLeader
26	Divergent log reconciliation	replication/ReplicaReconciler: reconcile
27	Metadata routing	client/MetadataRouter: refresh, call refresh
28	Replica admission to ISR	replication/ReplicaAdmission: tryAdd
29	Cross-layer consistency experiments	lab/ConsistencyExperiments: producerOutcomeUnknown, leaderAckLoss, allAckTimeout, consumerReplay, staleGroupSideEffect
30	Idempotent producer	client/IdempotentProducer: send, retry; replication/IdempotentAppender: append, recover; replication/ReplicatedPartition: produceIdempotent; broker/BrokerHandler: handle(IDEMPOTENT_PRODUCE); client/MetadataRouter: call; storage/RecordCodec: encode, decode; storage/SegmentLog: append, recover; storage/PartitionLog: append, deleteBefore, truncateTo
31	Idempotent consumer effects	client/DurableEffectStore: apply, recover

The first seven chapters cover four steps each; Chapter 8 covers Steps 29–31. The cumulative pass points at chapter ends are 4, 8, 12, 16, 20, 24, 28, and 31. If an earlier step has not passed, a later test will usually fail as well, because the course develops one evolving system rather than unrelated demo programs.

A.5 Course Simplifications and Differences from Kafka

1. **Routing:** keys use CRC32C unsigned modulo; Kafka’s default key partitioning algorithm is different. null keys use per-instance round robin rather than Kafka’s sticky batching.
2. **Disk:** records are encoded individually and each batch explicitly calls `force(false)`; this is not Kafka record batches or its default page-cache path. Forcing a write does not guarantee media-level power-loss safety.
3. **Index:** write one sparse point per fixed number of records, not at byte intervals.
4. **Consumer groups:** a simplified model with one topic and immediate rebalancing, not the full classic barrier, cooperative rebalancing, or KIP-848. Token fencing protects offset commits inside the course broker only; it does not make external side effects atomic.
5. **Offsets:** the local OffsetStore is not the highly available, replicated/compacted `__consumer_offsets`.
6. **Replication and control plane:** each broker fetches over real TCP and writes to an independent directory, but the authority/coordinator is single and trusted in the same JVM.

This is not KRaft and does not guarantee cluster recovery after controller/authority failure or consensus under arbitrary partitions.

7. **Visibility assumption:** the course retains the committed prefix and requires a clean candidate's LEO to be no lower than the authority HW, so the known commit boundary does not move backward during a controlled failover. This is stronger than real Kafka's HW behavior across epochs and must not be generalized into a claim that Kafka is always monotonic.
8. **Retries, idempotence, and transactions:** the ordinary `SimpleProducer` never retries automatically and a timeout may follow an append. The separate `IdempotentProducer` supports caller-managed producer identity and explicit retry, using batch stamps persisted in the log; it does not implement Kafka PID allocation, a transaction coordinator, client session WAL, or producer-state snapshots, and retention can expire the required prefix. The local `DurableEffectStore` deduplicates by stable business event id but is not atomic with consumer-offset commits. There are no Kafka transactions or `read_committed` guarantees, compaction, fire-and-forget, or ELR. At-least-once allows duplicates; this teaching subset does not claim end-to-end exactly-once.
9. **Recovery:** Step 26's full-prefix comparison is an $O(n)$ teaching simplification; real systems use leader-epoch history to narrow the divergent-log search range.

A.6 Official References and Further Reading

The references below explain Kafka concepts and differences in real systems; they are not specifications for the course's custom wire/storage layouts. Version labels reflect the pages cited by this book; configuration and protocol details can change between versions, so consult each page at its listed version.

Topic	Official reference	Related chapters
Kafka design and persistence model	Kafka 4.2 Design	Official overview of logs, replication, and consumer positions; see the simplifications above for this course's differences.
Consumer positions and commits	KafkaConsumer 4.2 API	Steps 14–16: position, seek, commit, and thread constraints.
Minimum ISR configuration	Kafka 4.3 Topic Configs	Steps 22–24: <code>min.insync.replicas</code> and write acknowledgments.
Consumer-group protocol differences	Kafka 4.3 Rebalance Protocol	Steps 17–20: context for the classic and newer protocols; the course does not implement the full protocol.
Leader epochs and truncation	KIP-101	Step 26: why HW alone is insufficient to locate a divergent log.
JUnit test execution	JUnit 5.11.4 User Guide	Read actual Step behavior failures and test reports.
Tectonic compile options	Tectonic compile	The book uses XeTeX/CJK with the declared bundle and local cache; see the build notes below.

A.7 Typesetting and Build Instructions

The English entry point uses `book[openany,oneside,11pt]`; the Chinese entry point uses `ctexbook[UTF8,fontset=none,openany,oneside,11pt]`. Both use the shared preamble and then load their language-specific strings. `fontspec` and `xeCJK` select fonts by file from the TeX bundle rather than scanning system fonts. Latin Modern roman uses `lmroman10-regul`

`ar.otf` and its matching bold/italic/bolditalic files; sans uses `lmsans10-regular.otf` and its bold/oblique/boldoblique files; mono uses `lmmono10-regular.otf` and `lmmono10-italic.otf`, with bold using the regular file and bolditalic using the italic file. CJK serif uses `FandolSong-Regular.otf` with `FandolSong-Bold.otf`, and `FandolKai-Regular.otf` for italic and bold italic; CJK sans uses `FandolHei-Regular.otf` and `FandolHei-Bold.otf`; CJK mono uses `FandolFang-Regular.otf` for every shape. The build does not depend on Noto, fontconfig, xelatex, or latexmk. Code snippets use listings and show only ASCII Java identifiers and short interfaces; Chinese explanations are written in the text. The book does not depend on minted, shell escape, or embedded web pages. TikZ diagrams are drawn locally by TeX. EPUB conversion uses Pandoc 3.12.1 and the same expanded TeX sources; Tectonic renders the six TikZ figures, and Poppler's `pdftoppm` rasterizes them for embedding.

`./gradlew :setup` resolves the verified JUnit dependency; `./gradlew :setupBook` prepares checksum-verified Tectonic 0.17.0 and Pandoc 3.12.1 under `.tools/`, and checks for Poppler's `pdftoppm`. Poppler must be installed separately; `:setupBook` does not compile Java or resolve JUnit, and the Wrapper needs Java 17+ to launch it. `./gradlew :book` builds the English edition by default with `COURSE_LANG=en`; `COURSE_LANG=zh` `./gradlew :book` builds the Chinese edition. The English PDF/EPUB are written to `build/book/en/simpleKafka.pdf` and `build/book/en/simpleKafka.epub`; the Chinese pair is written to `build/book/zh/simpleKafka-zh.pdf` and `build/book/zh/simpleKafka-zh.epub`. Each command updates only its language's checked-in files under `docs/book/dist/`; readers can open those files without Java or build tools. TECTONIC can override the default repository tool, PANDOC must resolve to version 3.12.1, and TEX_BUNDLE can specify a bundle URL or local directory/zip/ttb. The cache is stored in `.tools/tectonic-cache`. The first build needs the online bundle, which supplies TeX packages and fonts. Build both language editions online first to warm the resources; afterward, `BOOK_OFFLINE=1` uses Tectonic's only-cached mode for PDF and diagram builds, while Gradle's `--offline` separately controls Gradle dependency resolution. The book tasks do not compile Java or use JUnit. The generated PDF and EPUB files are not promised byte-for-byte reproducible. The repository-pinned Tectonic engine is version 0.17.0; Pandoc is version 3.12.1.

Glossary

Term	Meaning and location in this book
Message/record	One item of data handed to the system for storage and reading; introduced in the course guide and given a byte format in Step 1 (Record encoding and decoding) .
Producer	The program role that sends records; batching is taught in Step 13 (Batched producer) .
Broker	The service role that accepts client requests and stores/provides records; single-broker requests are in Step 12 (Single-broker requests) .
Consumer	The program role that actively fetches and processes records; its read position is in Step 14 (Pull-based consumer) .
Topic	A named category of related records; registration and partition metadata are in Step 10 (Topic metadata) .
Partition	An independently numbered, ordered log within a topic; multi-partition append and reads are in Step 6 (Segmented log) .
Key/value	Optional association/routing bytes and required content bytes on a record; null and empty keys are distinguished in Step 1 (Record encoding and decoding) .
Timestamp	A time value stored with a record; this course requires it to be non-negative, but record order is determined by offset; see Step 1 (Record encoding and decoding) .
Append-only log	A log where new records are written at the tail and reading does not delete them; encoding and append start in Step 2 (Log append) .
Offset	A logical record number within one partition; append results use half-open intervals, as shown in Step 2 (Log append) .
Byte position	A physical byte location in one segment file, distinct from a logical offset; see Step 3 (Offset-based reads) .
LEO	Log End Offset: the next offset available for append, not the offset of the last record; see Step 2 (Log append) .
Segment	One file-sized slice of a partition log; segment rolling is taught in Step 6 (Segmented log) .
Sparse index	Rebuildable data containing selected offset/file-position pairs to shorten searches; see Step 5 (Sparse index) .
Retention	Deleting closed old segments before a threshold; this does not mean a consumer has read them; see Step 7 (Log retention) .
Truncation	Removing a log tail after a chosen next offset, unlike deleting the old head; see Step 8 (Tail truncation) .
Metadata	Information that tells a client a topic's partitions and their broker locations; see Step 10 (Topic metadata) .
Frame	One complete course request/response with a length boundary in the TCP byte stream; see Step 11 (TCP framing) .

Term	Meaning and location in this book
Batch	Multiple records validated, written, or sent together; storage append is in Step 2 (Log append) , and producer buffering in Step 13 (Batched producer) .
Consumer position	The in-memory partition offset from which a consumer next reads; see Step 14 (Pull-based consumer) .
Committed offset	The persisted next offset restored after restart; it is distinct from log LEO and HW; see Step 15 (Consumer offset persistence) .
Consumer group	Members that coordinate ownership of a topic's partitions; see Step 18 (Membership and rebalancing) .
Assignment	The current set of partitions given to each group member; basic round-robin assignment is in Step 17 (Partition assignment) .
Rebalance	Recomputing partition ownership when membership changes; see Step 18 (Membership and rebalancing) .
Heartbeat	A request by which a member periodically reports to the coordinator that it is alive; timeout boundaries are in Step 19 (Heartbeats and expiration) .
Generation	A version incremented when group membership changes; requests with an old version are rejected; see Step 20 (Consumer group fencing) .
Fencing	Rejecting requests from an expired role or member using an epoch/-generation; the group example is in Step 20 (Consumer group fencing) .
Leader/follower	Replicas for one partition: the leader accepts writes and a follower fetches its records; see Step 21 (Fetch-based replication) .
Replication	Fetching the leader's records and appending them to other replica logs; see Step 21 (Fetch-based replication) .
ISR	In-Sync Replicas: replicas meeting the course's synchronization conditions and participating in commit-boundary decisions; see Step 22 (ISR and high watermark) .
HW	High Watermark: the right boundary of the prefix visible to ordinary consumers; only offsets below HW are visible; see Step 22 (ISR and high watermark) .
ACK	The producer's choice of when the broker replies that a write succeeded; see Step 23 (Produce acknowledgments) .
minISR	The minimum ISR membership required for certain writes to succeed; see Step 23 (Produce acknowledgments) .
Deadline	The absolute time by which a wait may complete; a timeout does not prove an earlier append was rolled back; see Step 23 (Produce acknowledgments) .
Leader epoch	A version incremented when the partition's writing role changes, used to reject requests from the old role; see Step 25 (Clean election and leader epochs) .
Authority	The trusted in-JVM teaching control state that records the current partition role and replication progress; see Step 24 (Concurrent replicated broker) .
At-least-once	A failure between successful processing and progress commit may cause the same record to be processed again; see Step 16 (At-least-once processing) .
Outcome unknown	A request may have taken effect even though its reply did not arrive, so the caller cannot infer from the exception alone whether it appended; see Step 29 (Cross-layer consistency experiments) .

Term	Meaning and location in this book
Idempotence	Retrying the same batch with the same producer identity, epoch, and sequence does not append it twice; see Step 30 (Idempotent producer) .
Producer identity/epoch/sequence	Persistent identity fields that distinguish a writer, its generation, and batch order; see Step 30 (Idempotent producer) .
Event ID	A stable business-event identifier that recognizes a replay as the same effect even at a different Kafka offset; see Step 31 (Idempotent consumer effects) .
Side effect	A business or external state change made by a consumer callback, such as an account update; see Step 31 (Idempotent consumer effects) .
Transaction	A mechanism for committing a related set of operations as a unit or failing them together; this course has no atomic transaction across a producer, broker, and external business state; see Step 29 (Cross-layer consistency experiments) .
Exactly-once	A guarantee that one input produces one observable business result, not that a callback executes only once; this course makes no end-to-end exactly-once guarantee; see Step 29 (Cross-layer consistency experiments) and Step 31 (Idempotent consumer effects) .